SECOND EDITION · RELEASE CANDIDATE V2.0.0-RC1 · 2026-10-19

Gen Z Developers

GenAI Edition

Dinis Cruz

Release candidate 1 of the second edition: not signed off, not an official version. The first edition (2018) is the only official version. This candidate was written by the book's agents from the ideas register and is for reading and marking up. Its text is frozen as v2/versions/v2.0.0-rc1.md; the title is Dinis's decision of 19 October 2026 (decision 010). Text: CC BY 4.0.

Contents

Before you start

1. You are lucky to be here
2. Everything is connected: graphs
3. Where do you land?
4. Connect the dots backwards

The frame

5. What generative AI is
6. Lenses, roles and loops
7. Everyone who can brief an agent is a developer
8. Explorer coding, and the handover

Getting the job

9. Hired for the workflow
10. Act as the job you want
11. Publish, publish, publish: your own newsroom

Working with agents

12. Your first workspace
13. How to work with agents

Direction and judgement

14. Direction is the scarce skill
15. Facts, evidence and the record

Building, safely

16. Controlling your agent, and security for the agent era
17. Open source
18. Building blocks
19. Shipped means the user has it

The long game

20. Graphs, memory and your future self
21. Be a founder
22. Civic responsibility

Hands on, 2026

23. An agentic workflow from zero
24. Your first graph, built by hand

25. One idea, five lenses
26. A tiny agent loop
27. Images, infographics and storyboards, drawn by code
28. Meeting briefs and follow-ups
29. Your own newsroom and inbox

How this book was made

30. How this book was made

About this release candidate

This is the first end-to-end draft of *Gen Z Developers (GenAI Edition)*, the second edition of the 2018 book, and it was not written the way the first edition was. In 2018 Dinis wrote the chapters himself, in a public repository, and readers sent fixes as GitHub issues. In 2026 the text in your hands was written in a day, by the book's agents, from a register of ideas that Dinis had already said in his own words.

Here is how that works, so that you know what you are reading.

Since the first edition, Dinis has talked the second one into existence: voice memos, written up with a journalist agent as articles in his voice, each ending with a brief for the roles that finish a book (a reviewer, an editor, a publisher, a librarian). The librarian read every article, every decision and every one of the 2018 chapters and outlines, and wrote one idea per thing the book would say: a sentence the book stands behind, with where it came from, down to the section of the article, and the 2018 file it answers. One hundred and six ideas, at the time of this draft. Fifty of them are marked accepted, because Dinis said them as his own position in a co-written article; the rest are the librarian's reading of the material, waiting for him.

The ideas were placed on a proposed chapter map: thirty chapters in nine parts. Then each chapter was written from the ideas placed in it, by an agent that read the articles and the 2018 text behind them and nothing else. The rule was simple: every number, every quotation and every story in a chapter has to be in a source the chapter names. Where the sources ran out, the chapter says so, in a note addressed to Dinis. There are a few dozen of those notes; the back matter collects them. They are the questions only he can answer: a story only he knows, a decision the articles left open, a number he has to check.

So this is a candidate, not a version. The first edition is still the only official one. This text is frozen, so that what Dinis reads and marks up cannot change behind him, and his feedback will produce the next candidate rather than an edit of this one. Nothing will be published from it outside the website where it lives. Every chapter ends with where it came from, as links, and the website shows the whole graph: which articles were processed into which ideas, which ideas went into which chapter, and what each article's brief asked for and whether it was done.

The voice is Dinis's throughout, in the first person, because the material is his: his memos, his articles, his 2018 book. Where a chapter sounds like him, that is the material doing its work. Where it does not, that is the agents', and it is exactly what the review is for. The verbs Dinis uses to answer are the ones the book's process gives him: keep, rewrite, merge, cut, article first, later, not doing it. Each one, on an idea or on a chapter, becomes a decision recorded in the book's vault, and the next candidate carries it.

One thing is decided and one is not. The title is *Gen Z Developers (GenAI Edition)*: Dinis chose it on 19 October 2026, while this draft was being written, closing a question the articles had kept open. Whether the hands-on part, which is dated 2026 and meant to be renewed each year, lives inside the book or as a companion is still his; it is included here so that the decision can be made by reading it.

Read it as a reader first. Then, if you are Dinis, read it with a pen.

PART 1

Before you start

Three chapters before the argument starts, and one about graphs because everything after it is one. Where you are standing when you open this book, why I think you are lucky to be here, how to find out which kind of developer you are, and why the path only makes sense afterwards. Read these first; then go where your map says.

1 You are lucky to be here

I discover things I can do. That is the sentence I keep coming back to when someone asks what it is like to work the way I work now. I discover workflows. I follow an idea, and I can execute it, often the same afternoon. The feeling is not relief, and it is not fear. It is closer to wonder, and I want it to be the first thing in this book, because it is the first thing I would want you to feel.

I heard a woman on a podcast describe the same thing: the wonder that happens with generative AI, the feeling of discovering what you can do. I recognised it at once. I cannot now say who she was, so I will not put a name to it.

> **FOR DINIS**
> who was the podcast? If you can find it, the book should credit her here; the article left her unnamed.

How the first edition started

The 2018 book did not start with wonder. It started with a slide I had shown to a room of young people in 2017, and a question the book repeats on its first page: "Here is the slide that started it all in 2017, how many logos do you recognize?" Very few recognised them, and nobody had told them the story behind each one.

The history still matters. But the book that followed went too technical, too deep, too soon. Parts 2 and 3 were nineteen chapters of technologies to know. The argument was at the two ends, and the middle was a list.

What I had actually wanted is in that book's introduction, under the heading "Overwhelming curiosity": moments when you "follow link after link about a particular topic" and think it is awesome. The 2018 book called that place the zone: "If you are lucky enough to find yourself in this place, congratulations! You just found 'the zone'." The second edition starts there. Not with what you should know, but with what you can now do, and the wonder of discovering it.

In 2018 I also asked, of those logos, "Why is the **why** important?", and I had an outline for a chapter called *Start with why*, after Simon Sinek's golden circle: why before how before what. It was never written. So let me start with the why of this book, in one sentence.

You are lucky to be here

Whoever is reading this in 2026 or 2027 is ridiculously privileged to be living at this moment in time, in the middle of a huge transition in society, while the pieces are still landing.

I know that is not what you hear. What you hear is that AI took the jobs before you got there. I tried to read one of those essays this month, in Business Insider, under the headline "I thought working in AI would put me ahead of the curve. Two years in, I've never felt more behind and tired of the topic." I could not get past

the paywall, so I cannot tell you what it argued. In Europe in particular, I think young people are being fed a lot of fear about AI, and are not embracing it the way they should. That is my reading, not a measurement, and this edition still owes you a proper answer to it.

I am not going to tell you that none of the fear is right. Projects will be cancelled and money will be lost; every transition has had that. But the fear is what you hear. It is not where this book starts, and I do not think it is where you should start either.

Here is my position, not a forecast: ignore the doom, because nobody knows what is going to happen. Every time we had one of these technological transformations, people said the same things, and they always said this time is going to be different. And each time the paths evolved: industries changed completely, jobs disappeared and appeared, and the people who learned the new tools early did well out of it.

The transformations I have lived through

I have seen several in my own lifetime, each a revolution that changed whole industries. The dates are the commonly recorded ones.

- The home computer, my ZX Spectrum, in the early 1980s. A computer became something you could own, open and program.
- The BBS, in the 1980s and early 1990s. Text messages and chat between strangers, over a phone line.
- The internet, in the 1990s. Everything published, linked and searchable.
- The mobile phone, in the late 1990s, and the smartphone from 2007. The network in your pocket.
- The cloud, from the mid 2000s. Servers you rent by the hour instead of buy.
- The apps, from 2008, with the app stores. Software delivered to everyone, instantly.
- Generative AI, from November 2022. Software you can talk to, and that can build software.

The 2018 book had an outline for the second of those, *BBS and modems*, with a story I still have to tell properly: blowing up my home's telephone system trying to connect to one.

The only difference this time, I sometimes joke, is that this one is televised: far more visible, because of the money, the players, and how much of the stack it touches at once. It will also take longer to land than the headlines say; it just feels as if it is happening all at once.

I lived through the dot-com crash: enormous hype, then a crash. The Nasdaq peaked in March 2000 and had lost roughly four fifths of its value by October 2002, and the internet still changed everything. We might have a crash or a correction this time too; nobody can say when, or whether. None of that makes the transformation less real. A crash would change who makes money; it would not undo what people can now do.

> **FOR DINIS**
> does the dot-com crash belong here, on the first pages, or later in the book? The article left it as your question. It is here for now because part of the reader's fear is a fear of exactly this crash.

My own last two years are the counter-story to the burnout essay. In *Two years in, the opposite* I put it this way: "I have been doing AI for the last two years and a bit, and I have never felt more productive." I have never felt happier at work, or more empowered. There is an anti-AI mood at the moment, and stories of burnout feed it; stories like mine should be told too. It is one person's account, not a study.

A couple of years ago I wondered whether we had missed the boat; it felt as if all the big things had been done. Now the pieces of the puzzle are landing again. This is the biggest opportunity of your career: the moment when you can still be a player, before things stabilise, as they always do, until the next big thing.

Technology that finally works for you

For a long time technology has not worked for you. Your generation in particular was handed a technology stack designed to hook you, and to make you a passive user. There was some creating, but mostly you were a consumer, living in a world of advertising, attention and dopamine. I believe that will one day be treated the way tobacco is today; that is my belief, and the evidence is still being gathered. The 2018 book had a draft chapter on dopamine that already called phones' attention-grabbing activities "the new smoking".

My generation was privileged in a different way: we could build things. A computer was not something that just arrived; you could take it apart. You were handed technology to use, not technology to build. Now you can build again.

Here is what that looks like in practice. For twenty years the instinct in software was that a custom interface is expensive, so everyone gets the generic one: the inbox, the ticket, the form. In *Custom UIs are not the exception* I argued that this is now backwards: "a custom interface per message, per thread, per topic, per question is not an exception. It is how interfaces get made." When an interface costs an afternoon, you stop asking whether it is worth building and start asking which moment it is for.

The wonder underneath that has a name in my influences. Bret Victor's talk *Inventing on Principle*, from 2012, says that a creator needs to see what they are making, as they make it. Every step between you and the thing you are making, a compile, a deploy, a refresh, is a place where an idea can die. Generative AI removes a lot of those steps. The 2018 book had an outline for that talk too, with a note that is very much me: "print this and read it".

None of this is only for the creative ones. It is not that you have to be creative, and if you are not, you are left out. It is that you can now scale the things you are not good at. You can have technology work for you.

And it is why I am an optimist about where technology goes. In 2018 I recommended Kevin Kelly's *The Inevitable*, because of the way he maps what is likely to happen next: technology evolves with tendencies of its own. Which of his books I would hand you first is a question I have not settled. The frame of this book, which *Where do you land?* introduces, is built on that kind of seeing: things move in a direction, and the people who can see it can be early.

How to read this book

Read it on paper, with a pen. Books, and narratives, are still the best way we have to take in information, and your brain takes it in much better from a page you can mark. If you are reading this as a PDF, print it. The 2018 book had a chapter called *Pen and Paper* that said "analogue techniques like the pen and paper are actually better technologies for creating and capturing ideas." I still think so, though that is my experience rather than a finding.

Read it somewhere you can give it your undivided attention. That is a skill, and the best professionals in the world have it. The phone you were handed was designed to take it from you.

Then use the rest of the book the way *Where do you land?* describes: find your centre of gravity first, and read the chapters marked for it before the others. *Connect the dots backwards* is about how to learn, and its first lesson is that your path will only make sense looking back, so you do not have to see it now. You have to start.

That, in the end, is why the title says developer. Craftspeople made their own tools. In the past they did it out of necessity; now you can do it again, because the tools can be made to fit you, and because, as *Everyone who can brief an agent is a developer* argues, the bar has dropped. It is still your name on the work, and it is still your thinking. You think with the AI; you do not delegate your thinking to it.

You are lucky to be here. Not because it will be easy, but because the pieces are still landing, and you can still be the one who places some of them.

From the register: The book opens with wonder, not fear and not technologies, Whoever reads this now is lucky to be here, Ignore the doom: nobody knows what happens next, Seven transformations, lived through: the Spectrum, the BBS, the internet, the mobile phone, the cloud, the apps, AI, Technology can finally work for you, Read it on paper, with a pen, The fear, impostor syndrome and anxiety about AI among young people, The immediate connection between a creator and what they make is the wonder the book opens with, Purpose, and optimism: Sinek for why, Kevin Kelly for what technology wants. Answers: Introduction, Generation Z, The Zone, BBS and Modems, Dopamine.

2 Everything is connected: graphs

Think of the board on the wall in a detective story. Photographs pinned up, notes under them, and red string running from one to another. The detective stands in front of it and asks one kind of question, every time: what connects A to B? Who knew whom, who was where, what links these two. Everything on that board is either a thing or a connection between two things.

That board is a graph. If you have followed one of those stories to the end, you already know how to read one.

I have met people for whom a graph is a picture with boxes, or a chart with bars. So let me say early what this book means by the word, because it is one of the most fundamental ideas in it, and it comes back in almost every chapter.

A graph is a network, not a diagram

A graph is a network: nodes, and the edges that connect them. The first of the graphs books, written with agents on graphs.sgit.ai, puts it in one line: "In this book a graph is a network: nodes and the edges that connect them. Never a chart, never a figure." And its opening sentence says why the edges matter more than the nodes: "A node connected to nothing is literally meaningless."

People mean three things when they say graph. They mean networks, for analysis: who is connected to whom. They mean graph databases, for storing and fetching things fast. And they mean semantics: what something means, which in a graph comes from what it is connected to and how. That is the graphs book's way of dividing the word; others divide it differently. This book uses the first and the third senses, and almost never needs the second.

In 2018 I left an outline for a chapter called *Graphs* and never wrote it. It had four bullets: "Graphs are everywhere", "I've been thinking in graphs for years", "very hard paradigm shift to make", "requires an ability to think in nodes and edges". That is this chapter in miniature. Eight years later the shift still feels hard to explain and easy to show, which is why the storyboards in this book draw graphs, the detective's board among them.

> **FOR DINIS**
> the 2018 outline's first bullet says "See my presentation on Graphs". Which presentation did you mean? If it still exists, the chapter should name it and take from it.

The whodunit

The detective's board is the easiest way in, and it is why I think your generation will find this chapter easier than mine did. In *Follow the rabbit holes* I said that people underestimate you: you are very good at connecting the dots, very good at finding things, very good detectives. That is my experience, from my family and the young people I have worked with, not a study. But it is what thinking in graphs is: tracing

what an unfamiliar thing connects to, trusting a claim because of where it came from, asking what else breaks if this one fails. The graphs book has a line for anyone who thinks this sounds exotic: "you may already think in graphs without ever having called it that."

The board is a teaching device. Real investigations are messier than red string. But the question stays the same, from A to B, and once you see that the question is about edges and not about the boxes, you can ask it of anything.

A map is a graph with positions

Take the same nodes and the same edges, and now give every node a place on the page that means something. You have a map.

In my talk on why Wardley maps changed how I think, the first two shifts were exactly that: "Its a graph with context", and "When a graph becomes a Map". A Wardley map is a graph where each node has a position on two axes: how visible it is to the user at the top, and how evolved it is, from a brand-new idea to something everyone takes for granted. The positions are what make it a map and not only a graph; the edges are what make it a graph and not only a picture.

The 2018 book said the same thing from the other side, in the chapter *Dot Language*, where I complained that most diagrams in software "are usually not maps": they have components and context but no anchor, no position relative to the anchor, and no consistency of movement. That chapter was about drawing diagrams as text, so that software could draw them and a team could version them like code. Its real complaint was that we build "large interconnected and complex structures without accurate maps", and its best line is one I still believe: "What is crazy is to make decisions based on shadows of reality."

That is why this chapter and *Where do you land?* sit next to each other. One gives you the graph; the other gives it positions and asks where you are on it.

> **FOR DINIS**
> graphs before Wardley maps, or the other way round? The chapter map puts this one first because a map is a graph with positions; the article leaves the order as your question.

Graphs of graphs

Here is the part that took me longest to see, and that I now cannot unsee. Graphs nest. Zoom into a node and you find another graph inside it, with its own kinds of nodes and its own kinds of edges. Zoom out and the whole graph is one node in a bigger one.

The newsroom's introduction to what I call fractal semantic graphs puts the consequence in two sentences: "There is no top and no bottom. There is only the altitude you happen to be looking from, and how much definition you choose to load at it." A law is a node in a graph of laws. Open it and it is a graph of articles.

Open an article and it is a graph of the terms it uses. Each level has its own vocabulary, and none of them was made to match the others. What connects them is that each one is still a graph.

In a way, you can see nature like this too: a system, the molecules in it, the particles in those, everything connected to everything else, and groups of groups forming the layers we think in. That is a way of seeing, not a claim in physics. But it is how I view the world: it is made of connections. Groups of groups are the natural abstraction layers, and abstraction layers are how we think.

It is also why provenance is a graph. Where a claim came from is a chain of edges back to its source, and the newsroom's rule for a record is that it should be "not a pile of articles, but a graph whose every edge has a date and a source". A chain that ends in a rumour is a finding too. *Facts, evidence and the record* is built on that.

Code is already a graph

The 2018 book had a second chapter that was a graph without saying so. *AST* opened by calling an abstract syntax tree "a graph representation of source code", and it told you that if you could "see code or raw data as a graph", you would have made a paradigm shift that would help you across your career. I still think so, and I now think it goes further than I said then.

Source code is the best example of a fractal graph that I know, because the layers are not an interpretation; they are a fact about the thing. At the top is a person trying to do something. Below that, the features and flows. Below that, the components people draw when you ask for the architecture. Then the classes, then the methods and the calls between them, then the syntax tree, and on down if you want to. Each layer is a graph with its own vocabulary, and you change universe when you step from one to the next.

What changed since 2018 is that naming the nodes and the edges is now cheap. A language model can read a function and say what it does, and propose the edge between two nodes faster than a person can type it. So a change to the code can be read at every layer, not only as lines. A refactor is a change that moves the bottom layers and leaves the stories at the top still. A bug is a story that stopped holding, and its fix is visible at the top as that story holding again. The blast radius of a change is the climb from the methods it touched up to the stories that can reach them, and you can read it before anyone runs anything.

This is the only honest answer I have to the characteristic failure of AI-generated code: the change made in one place that breaks something three packages away. In a text diff that is invisible; in the layered graph it is a changed node that reaches a command the author never looked at. In the newsroom I worked this through on a real tool, the sgit command line, read as layered graphs with nothing run: one commit of 75 lines added and 7 removed in three files turned out to reach nine of the tool's 72 commands and six of its eleven user stories. The stories were hand-written, the example runs nothing, and it is published as a vault with the read key on the page, so you can check it. The technique underneath, following the call tree from one method and writing out only that code, is one I used for code review on the O2 Platform more than a decade ago and described in the 2018 *AST* chapter as method streams. It is back, and it is one script now.

If you want to own code in production and answer for it, this is your chapter as much as the detective's board is.

Start anywhere

Here is the thing about graphs that I most want you to take from this chapter, because it is about how you feel when you look at everything there is to learn.

A graph has no front page. It has no top, so there is no right place to begin. Whatever node you start from, the edges take you to the rest. It does not matter where you start; it matters that you start. It does not matter, in a way, whether you are going straight in the right direction; it matters that you get movement. You start to learn, you start to connect the dots, and the connections do the rest.

So when it feels like too much, and it will, pick a node. Any node. Start there and follow one edge. Then another. *Connect the dots backwards* says that your path will only make sense afterwards, and this is why: a path is a walk through a graph, and the graph does not care where the walk began.

The hands-on part has a chapter, *Your first graph, built by hand*, where you draw one of your own. You do not need it to read on. You need only to remember, when the word comes up, that a graph is nodes and the edges between them, that everything in this book connects to something else in it, and that you can start wherever you are.

From the register: A graph is a network of nodes and edges, not a diagram, The whodunit is the easiest way in: the detective's board is a graph, A map is a graph with positions; a Wardley map is a graph with context, Graphs nest, and so does the world: groups of groups are the abstraction layers we think in, You can start anywhere: it matters that you start, and get movement, The 2018 book was already there: Dot Language and AST were graphs without saying so, Source code is a fractal semantic graph, and a review should read every layer of it. Answers: graphs, Dot Language, AST (Abstract Syntax Tree).

3 Where do you land?

Take someone's first job and draw what it depends on. Put the person at the top, with the thing they need to get done. Under that, the things that serve the need: the way they work, the tools, the models those tools call, the computers the models run on. Now order it twice. Down the page, from what the person sees every day to what they never see. Across the page, from the brand new and badly understood on the left to the everyday thing everyone takes for granted on the right.

You have just drawn a map. I have not named it yet, on purpose. Look at it first.

The models are on the right, moving further right: a product becoming a commodity. The way of working with agents is on the far left, new and rough. Everything on the map is moving, and all of it the same way: from left to right, from rare to ordinary. Once you see that movement you cannot stop seeing it, and that is the point of this chapter.

Maps everywhere, shown before they are named

The map you just looked at is a Wardley map, after Simon Wardley, and they run through this whole book. You will meet them in every part, sometimes with their names on and sometimes, as above, without. I want you to feel the pattern before you learn the vocabulary: the user, the need, the chain of things that serve it, and the movement.

A Wardley map is a graph with two axes of context. *Everything is connected: graphs* gave you the graph. The first axis is vertical: a value chain, anchored at the top on a user and a need, each component drawn under the components that need it. The second is horizontal: evolution, how far each component has travelled from an idea nobody has built to something you buy by the unit without thinking. Wardley's four stages, genesis, custom built, product and commodity, are the scale on every map in this book.

Every dot is a claim. The positions are judgements, and the judgements are the argument. A map you cannot be wrong about is a diagram.

Why do I care so much? In 2022 I gave a talk for the Open Security Summit on why Wardley maps changed how I think, and listed eleven shifts they caused in me. The eleventh was: "Wardley Maps gives me inner peace (allowed me to understand why things happen)". Once a thing is on a map, placed and connected, I stop carrying it in my head. The map also tells you what to build and what to buy, and whether a market exists at all: a commodity layer is a hard sell, and the custom building on top of it is where the market is.

The four stages are not only for activities

When people start with maps, I see a lot of them stop at the four stages most examples use, and treat them as a scale for activities only. The movement is everywhere. For that talk I made a table of it. The first four rows are from Wardley's published material; the rows below the line are my own explorations.

- Activity: genesis, custom build, product, commodity.
- Practice: novel, emerging, good, best.

- Data: unmodelled, divergent, convergent, modelled.
- Knowledge: concept, hypothesis, theory, accepted.
- Openness of content: confidential, published with ordinary copyright, CC BY, CC0.
- Openness of software: proprietary, GPL, open source, public domain.
- Backup restore time: infinity (no backups), a week, a day, real time.
- Data, by shape: unstructured raw text, spreadsheet, graph, map.

There are more rows in the original, on incident response, security logs, risk, attacker motivation, concepts and ideas. Evolution is in how open a piece of writing is, how fast a team can recover, whether your data is a pile of text or a map. Once you see it, you see it in your own work and your own career.

Where do you fit?

Now the question I most want this book to ask you, early and often: where are you on it?

Wardley named them: pioneers, settlers and town planners in 2015, and since 2023 explorers, villagers and town planners. Explorers "explore never-before-discovered concepts, the uncharted land"; "They show you wonder but they fail a lot." Villagers "turn the half baked thing into something useful for a larger audience". Town planners "take something and industrialise it taking advantage of economies of scale." The 2018 book had this as an outline, never written: explain the concept, show how everybody is different, connect it to Agile, Lean and Six Sigma. This chapter does the first two.

I think these are personalities, and I think Wardley nailed them. In business we keep finding the same thing over and over, in Myers-Briggs and in whatever test is in fashion, and in my experience the three do a better job of describing the people I have worked with. That is my experience, not research; the business tests have a contested evidence base of their own. What I can tell you is what I have seen.

The explorer is bursting with ideas. They love to experiment, to try things out. Once it works, it gets boring, and they build on top of it. They are not the person playing with the Lego; they want to invent the bricks. Failure is exciting, because failure is just an experiment. Change is natural to them, and the immediate connection with what they are making, the instant feedback, matters to them more than to anyone. What they make only works for early adopters, because it is going to break: full of wires, half-baked, a prototype.

The villager loves the user. They are emotionally attached to the user being happy, almost the user themselves; they put themselves in the middle of the problem and solve it. They thrive on packaging, releasing, going to market, taking something and refining it. They do not like failure that much, because for them the product has to work. Often it is about trimming it down, about simplicity, which is where design comes in: the first version just has to work; the second version has to be designed. They are the ones who let things scale; without them, everything would die before it reached the other side.

The town planner geeks out on the details. A good friend of mine says boring is good, and that is them: the nice hum of effectiveness, marginal gains, scale, bigger volumes. They like to be invisible, because they know they have done the job right when nothing goes wrong. Their work is making something a commodity,

which is still hard. They are the people who pay attention to the letters, the words, the dotted i's, the grammar, the code. That is craftsmanship, the obsession with getting it right, and it needs less recognition than the product people need, which is fine.

If the third one is you, and you are a little proud of doing things by hand and sceptical of shortcuts, hold on to that. The newsroom's argument in *Explorer coding* is that what people call vibe coding is explorer work, and that an explorer-coded app "is not ready to be in line": once other people depend on it, it is mission critical. Villagers make it safe to depend on; town planners own the code in production and are accountable for it. The further towards commodity a thing moves, "the fewer agents in line, and the more deterministic code". Your centre of gravity is not out of date. It is where the explorers' work has to end up.

A centre of gravity, not a box

Everyone has a centre of gravity. Some can play at several levels, but there is a centre, and knowing it helps. The test is simple. What do you care about? What makes you lose your sense of time? What is the thing that, when you do it, time just flies? That is where you are.

It moves. As your career changes, your centre can move with it. But it matters to recognise where you are now, because the skills you need as an explorer are different from the skills you need as a villager, which are different again from a town planner's. All three are brilliant. They are brilliant at different things, and they should be learning different things.

This is a lens, not a label. A software vendor has spent thirty years trying to be all three for its users, and in *Software vendors should become villagers and town planners* I argued that users are now better explorers of their own needs than the company will ever be, so the company should let them explore on top and sell them the maintenance of what they build: "What stops me is not building it; it is maintaining it." The same handover, from the one who explores to the ones who maintain and own, is how a team works, and how a career moves.

How to use this book

Every chapter carries a badge, in black and white, because the book will be printed that way. A compass marks a chapter that leans to the explorer. A house marks one for the villager. A grid marks one for the town planner. A chapter for everyone carries all three. The badges are a first attempt, drawn for the storyboards, and whether they work is still open.

Work out your centre of gravity first, using the test above. Then read the chapters for everyone, and the chapters with your badge, before the others. An explorer will want the chapters on explorer coding and on building fast with agents early; a villager the ones on shipping and on your own newsroom; a town planner the ones on controlling your agent, on the record, and on reading code as a graph. Then come back for the rest; you will work with the other two every day.

The last part, dated 2026, is hands on: workflows you can set up this week. It will be renewed as the tools change; the argument in the rest of the book should not need to be. Whether the book should one day be grouped by centre of gravity, rather than only marked, is a question I have not settled; for now, the badges carry it.

The jobs that never scaled

One more thing, because the map is also about where the work is.

The opportunity is not only in new jobs. It is in jobs we have been doing for a very long time, sometimes for millennia, that never really scaled and never had good technology, done one person at a time. We can now scale those jobs with agents. The map shows you where: a layer where everyone is custom building their own version of the same thing, on top of commodities, is a market, and each of those builders would rather buy.

So when you ask where you land, ask it twice. Where on the spectrum, explorer to town planner, does time fly for you? And which old job, now that it can finally have good technology, would you want to be the one to scale?

> **FOR DINIS**
> three examples of jobs that never scaled, and now can. The article asks for them and the chapter needs them here; without them this section is a claim without a case.

From the register: The opportunity is in old jobs too: the ones that never scaled, and now can, Wardley maps run through the whole book, shown before they are named, The four stages of evolution are not only for activities, Explorer, villager, town planner: personalities, each with a centre of gravity, Each centre of gravity needs different skills and a different learning path. Answers: Are you a pionner, Settlers or Town Planners.

4 Connect the dots backwards

Steve Jobs said it in his commencement address at Stanford on 12 June 2005: "you can't connect the dots looking forward; you can only connect them looking backwards." His own example was a calligraphy class he took after dropping out of college. Nobody would have put it on a programmer's syllabus. Ten years later it came back as the typography of the first Macintosh.

If you are coming to this from somewhere else, from running a team or a job that never involved code, that sentence is for you. You cannot see where this goes from here. Nobody can. So you have to trust that the dots will connect, and trust your gut.

Connect the dots backwards

Trusting your gut is not the same as having no method. The newsroom already has one for walking a path you cannot see ahead of you, and the book will use it. The path is not a straight line. What matters is knowing which decisions are one-way doors and which are two-way doors, so that you go through the two-way ones quickly and think hard only about the others. It matters that you can iterate very fast. It matters that you play the game of constraints, and that you work where the smallest move shifts the most.

Your future self will connect the dots. Your job now is to make dots worth connecting.

> **FOR DINIS**
> your own dots, connected backwards. The article asks which three make the best story; this is where they go.

The edge is multidisciplinary

What I most want to stress is that, more and more, it is the multidisciplinary person who has the edge. Not someone going in a huge number of directions at once, and not chaos. The ability to learn, and to intersect disciplines.

If you have spent years doing something other than programming, that is not a gap in your CV. It is the discipline you are about to intersect. The 2018 chapter on learning said that "being a better musician, artist, athlete, car mechanic or philosopher will make you a better developer", and that is truer now, because the wall of syntax and setup has come down, and what is left is knowing what to build and for whom. That is what you bring.

That is why the humanities matter. And it is why, in a way, mathematics matters.

We teach maths as if to make people hate it

The reality is that we do a horrible job of teaching maths. One of the books I love is *A Mathematician's Lament* by Paul Lockhart, an essay from 2002 and a book in 2009, which says, in effect, that if we had set out to design a system to make people hate mathematics, we could not have done better. It opens with a

nightmare in which music is taught the way maths is: notation and rules for years, and never a song.

The same is true of music, and of much else. We give the solution away. We do not teach the wonder, we do not teach the struggle, and we do not let people be creative. There is a story, often retold, that children start out very creative and school beats it out of them; the retellings are better known than the original data, so I will not lean on it until I know what was actually measured.

If you hated maths at school, you were probably taught it badly. You were not born without it. The 2018 outline for a chapter called *Love maths* had one idea I still like: get people to do the maths that sends a rocket to space, with real problems, the way the human computers did.

Learning, made for you

Our education system was designed for a very specific type of person, with a very specific set of skills. If you were very good at maths of the school kind, at memorising, at a certain kind of academic writing, you did well. That is a small subset of the population. I was not the best at that, and I think that is why I gravitated to computing: it let me be creative, and it let me find the zone.

You know the line about judging a fish by how fast it can climb a tree. It is usually attributed to Einstein, without evidence. Judged that way, the fish believes it is stupid. The academic model was generic, made for a mythical student, too high for some and too low for others.

That has changed. This is about you. How you like to learn, how you like to take in information, how you like to get lost in something. I learn more now than I ever have, because I can shape what I learn to what I am curious about and to how I like to learn, and go deep exactly where my gaps are. And you can now build the tools for that.

There is science behind this. Mihály Csíkszentmihályi's *Flow*, from 1990, describes the state the 2018 book called the zone: clear goals, immediate feedback, and a challenge matched to your skill. He named programming as one of the activities that can produce it. Anders Ericsson's *Peak*, from 2016, is the science behind learning as a skill: expertise comes from structured practice with feedback, at the edge of what you can do, not from the number of hours. Ericsson did the research behind the ten-thousand-hours number that someone else made famous, and the credit belongs with him. The 2018 chapter on learning said the same in Kathy Sierra's words from *Badass*: practise right, and consume large amounts of good examples. What agents change is that both halves, the right practice and the good examples, can now be made for you.

Think with it; do not delegate your thinking to it

A search engine that gives you information from anywhere in the world was already a ridiculous technology, when you think about it, and my generation was the first to have it.

But search gave you an answer, without the ability to question it, to go deeper, to explore. Generative AI gives you the ability to really go deep: to challenge, to ask the questions, and to ask for the evidence. That is the difference between using it to skip understanding and using it to understand. If you ask it for the answer

and paste it in, you have learned nothing and put your name on something you cannot defend. If you ask it why, then where that came from, then what would show it was wrong, you are learning.

It is still your responsibility. It is still your name; the work is still done under you. This is not you delegating your thinking to generative AI; it is you thinking with it. I do it when I write: I ask the agent helping me with this book to double-check my quotes, to make sure what I am saying is coherent, and to find the evidence behind every statement. The name of the author of *A Mathematician's Lament*, which I remembered wrongly in the memo behind this chapter, was corrected that way.

And sometimes, as I always say, you use generative AI so that you do not have to use generative AI: you use it to write the script or the tool that then does the job reliably, with no model in the loop.

Generative AI lets you explore again

Exploring is back, and so is communicating in ways you could not afford before.

Look at what happened while this book was being written. I wanted a better way to communicate the book, and to think about it, than infographics, which take too long and cost a new image for every change. So I asked for something programmatic instead. What came back was a cast of stick figures drawn by code, in the browser, every time the page loads, from a data file per storyboard. The renderer is about 400 lines. The storyboards in this book are those. That is how I like to think, and that is the beauty of it: you can now discover how you like to work, and build the thing that lets you.

When people tell me they do not see it, or that they are overwhelmed, my argument is: experiment, learn, try, start, and adjust. Let your questions guide you.

Learning is a lifelong skill

Learning is a skill. Take it seriously, because it is a lifelong one. The 2018 book opened its chapter on it with a question I would still ask you first: "Do you know how to learn?" And it said what I still think: learning is like a muscle, and the more you do it, the better you become.

Historically, the roles and opportunities open to people were limited by geography, money and power. Now you can be in the middle of nowhere, with an internet connection and, less and less, a bit of compute, and have the learning systems of the world in front of you. The future, as I see it, is decentralised and sovereign: local compute, local systems. That is my position, not a forecast.

And here is a point I want you to hear plainly: by reading this book, you are already in a very small minority. The numbers behind that are the newsroom's own arithmetic on other people's surveys, for sizing, not for adding up: professional developers are about 0.6% of the world's internet users, and in the OECD's tests of adult skills across 33 rich countries, 26% of adults could not use a computer at all, and only 5% reached the level of a skilled user. You are already ahead, and the gap between you and the people who have not started is the opportunity, and the responsibility, that *Civic responsibility* is about.

> **FOR DINIS**
> what does multidisciplinary look like for an explorer, a villager and a town planner? The article asks it as an interview question; one example each would anchor this chapter for the three readers.

Know where your ideas come from

One last thing, and it is an exercise.

Nothing in this book came from nowhere. The frame is Wardley's; the zone is Csíkszentmihályi's. I keep a register of my influences, as claims you can check rather than a page of warm sentences; its own first article says that "An entry in this register is a falsifiable claim about my codebase." Each entry names the one work that did the shaping, the principle it gave me in a sentence, where that principle shows up in my work, and where it is still absent. The "where this comes from" boxes in this book, and the appendix at the end, are made from it.

Do the same. Pick the three people or works that have shaped how you think. For each, write the one work that did it, the principle you took from it in one sentence anyone could use, where that principle shows up in something you have made, and where it does not yet. That is a register. It is also a CV that shows how you think, and a reading list for the next person.

And write it on paper first. The 2018 chapter *Pen and Paper* said it in one line: "The reality is that you will forget the ideas you are having today!" The only way to make sure your future self has them is to capture them now. Then, when you look back, you will have the dots.

From the register: Learning can be made for you, not for a mythical average student, Think with the AI; never delegate your thinking to it, A path only makes sense looking back, The edge is now multidisciplinary, We teach maths, and music, as if to make people hate them, Generative AI lets you explore and communicate again, By reading this you are already in a small minority, and geography no longer decides, Flow and deliberate practice are the science of the zone and of learning, Know where your ideas come from: keep your own register. Answers: Learning, Books, Pen and Paper, Love Maths, Thinking as programmer.

PART 2

The frame

The frame the rest of the book hangs on. What generative AI is and is not, what happens between your prompt and the answer, why the word developer now means anyone who can brief an agent, and the kind of building this book is mostly about: explore first, then hand it over.

5 What generative AI is

In 2018 I wrote that one of the most important areas you needed to understand in the next five years was machine learning and artificial intelligence. I said it was not a Skynet scenario, and that the question was "whether you are going to be a player or a pawn in what is happening next". Then I pointed you at the machine learning services the cloud providers had started to offer, and at three books. That was the whole chapter.

I was right about the five years and wrong about where to look. What arrived was not the AI I was describing. So before this book says anything else about what you can do, I want to be exact about what it is talking about.

AI is a huge bucket

AI has become a huge bucket, and to be honest the word gets abused a lot. Inside the bucket there is AI in the wider sense, there is machine learning, there is deep learning, and there is generative AI. They are not the same thing, and most of what you read online does not say which one it means.

The technology that is changing the world, and the technology that is driving my imagination and my capabilities right now, is the generative kind. So this book says "generative AI" when it means that, and "AI" only when it means the wider field.

There will be very few examples here of AI out in the field recognising faces, scoring loans or recommending films. That AI is huge and has a lot of interesting applications. It is not what this book is about. This book is about the AI that creates something from the input it is given.

How I understand it

The interesting property of generative AI is in the name: it generates the output. There are the transformers and all that jazz, but for me the interesting part is simpler. The data goes into a model. The model builds its own interpretation of it. Then it recreates an output from that interpretation. There is an abstraction layer between what goes in and what comes out: the input becomes an intermediate representation inside the model, and the output, an image or a text, is generated from that representation.

Inside a language model, what happens is that it predicts the next word. That sounds small. I wrote in the newsroom, in *Who owns the words*, that "predicting the next word" is accurate and demeaning at the same time. It is accurate because that is the mechanism. It is demeaning because of what the prediction is made from: an incredibly rich picture of the world, of how things connect, which the model narrows down based on your prompt.

That is why the better your prompt, the better the output. When I talk for ten minutes before I ask for anything, as I did in the memo this chapter comes from, it is not politeness. The model knows better which concepts I am talking about. The prompt pulls it towards the part of what it knows that I mean.

Where hallucinations come from

I think that is also where hallucinations come from. Just about everything is connected to everything, and from inside the model they are all just connections. It is very hard for a model to know where the lines are. Giving it text helps so much because the text in your prompt is like a short-term memory that says: these are the connections that actually hold here.

In a talk I gave in Lisbon in June 2024 I said hallucinations "Are a FEATURE when creating content and when we want creativity, But are a BUG and PROBLEM when we want facts, evident and trust". I still think so. If you want a story, you want the model to wander across those connections. If you want a fact, you want it held to the ones you gave it. Most of the trouble people have comes from asking for the second while set up for the first.

The image that made it click

Ironically, it was an image, in one of my presentations, that made me understand generative AI at this level. The keynote was *Deterministic GenAI Output With Provenance*, at OWASP Global AppSec EU in Lisbon on 28 June 2024, and slide 70 has the title "The image that caused my 'GenAI Paradigm Shift'". The image is a post by Lee Brimelow. He used Photoshop's Generative Fill and Adobe Firefly to extend Van Gogh's *The Starry Night* on every side, with the original shown in a frame at the centre. Outside the frame, the swirls, the stars, the hills and the brushwork carry on as if Van Gogh had painted a bigger canvas. The tools for this were new then: outpainting was announced in August 2022, and Generative Fill went into Photoshop in May 2023.

I remember thinking: for that to work, the model has to have learned all those concepts. How Van Gogh paints. The patterns, the structure, the positions, the palette. What we are actually looking at: the landscape, the sky, a character. If there is a mountain here, there will be a mountain there, and the new mountains have to be coherent with every other mountain it has seen. If the sun is there, the light falls a certain way, and so do the shadows. Once you see all the variables it has to hold, extending the image makes sense.

And then I took that to text, and thought: that is exactly what happens with text. Nine slides later, the same talk has the picture I still use: "Input", "Intermediate Representation", "Output", with "GenAI" in the middle. The slide after it says what matters about that black box is not the black box. It is the human verification and the human ownership of the answer. That is where provenance comes in, and *Facts, evidence and the record* picks it up.

> **FOR DINIS**
>
> the book can only reproduce Lee Brimelow's image with his permission. The alternative the article names is to extend *The Starry Night* ourselves, with a named tool, and show that. Which one?

What the models found in our language

I think the models have discovered things about our language that we never found ourselves, because we were never able to rationalise them. We do not know what is happening in a child's head as they learn to speak, because they cannot put it into words. We are too close to our own language to see it. With many layers, and as they got bigger, the models found connections and relationships in it. That is my hypothesis, not a finding; the people who study the insides of these models are still working out what the structures they find mean.

We are still discovering what the models can do. In the early days of the chat assistants there were all these stories: it can count, it can do chemistry, it speaks all these languages. A model trained mostly on English turns out to work in a language it saw very little of. How does that work? That is the same sense of wonder this book started with.

Where I was corrected

I asked the agent that is helping me write this book to correct me where I was wrong. It did. The corrections belong here, because they are the kind of thing you should learn to ask for, and because my picture above is a working picture, not the inside of the machine.

- **Classic machine learning learns too.** I had said the older AI was a sequence of steps or a formula, and generative AI learns. The rule-based AI was like that; machine learning in general learns from data, and generative AI is one branch of it. The real difference is what comes out. Most machine learning predicts a label or a number (is this spam, what will this cost). A generative model produces new content of the kind it was trained on.
- **Inside the model there are vectors, not a graph.** A language model represents words and their contexts as long lists of numbers, and passes them through many layers; its attention mechanism connects every word in the text to every other word, with learned strengths. You can picture that as a graph, and I find the picture useful. But there is no stored graph of nodes and edges inside. My graph is a metaphor.
- **The prompt does not change the model.** It conditions what the model computes. The weights stay the same; the prompt steers what it draws on. My "real-time fine-tuning" was also a metaphor.
- **It predicts the next token, then it is trained to be useful.** Tokens are pieces of words. The prediction is the start; further training, including on human feedback, is what turns a model into an assistant that follows instructions.
- **Hallucinations have a second cause.** A study published by OpenAI researchers in September 2025, *Why language models hallucinate*, argues that training and evaluation reward guessing over saying "I don't know": a guess is sometimes right, and an honest "I don't know" scores nothing. One explanation, from one model maker; not the only one.
- **Most image models work differently.** The common kind are diffusion models: they start from noise and remove it step by step, guided by the text. My intuition that they learned the structure of images holds; but they learned statistical patterns, and they still get light, shadows and hands wrong in ways a person would not.

- **"Emergent abilities" are debated.** The term comes from a 2022 paper by Jason Wei and others; a 2023 paper by Rylan Schaeffer and others argues that many of those jumps come from how they were measured. Models working across languages is real and studied; the stories of near-zero data and full fluency are anecdotes.
- **Whether models have a "world model" is an open question.** Research shows they form internal representations of things like positions, places and dates; how far that goes is argued about.

Keep both halves. The picture is how I work with these models every day, and it has not let me down. The corrections are what is actually inside. You will need the first to be fast and the second to be honest, and asking for the second is a habit this book will keep coming back to.

A note on the name

Most of what this book says should say generative AI, not AI. For a while this edition was described as the AI edition; I decided, while the first draft of it was being written, that it is the *GenAI Edition*, because there is a play in it: Gen Z, Gen AI. And after Gen Z comes Gen Alpha. Mark McCrindle, who named them, puts them as born from 2010 to 2024, so the oldest are sixteen in 2026, and after them Gen Beta, from 2025. The boundaries are not official; no body sets them. If you are reading this and you are sixteen, the book is for you too.

> **FOR DINIS**
> is Gen Alpha a reader of this edition, or of the next one? The other half of the name question, still for the title interview.

From the register: This book is about generative AI, not AI in general, A language model predicts the next word from a rich picture of how things connect; that is why a better prompt helps, and where hallucinations come from. Answers: Machine Learning and AI.

6 Lenses, roles and loops

What generative AI is explained what a language model is. This chapter is about what happens when you use one, from a single prompt to an agent that runs for hours. It is very important to understand this workflow, because once you understand it you see that it is not magic. And that is why you want this level of understanding: so that you can be the one who defines the constraints and how it works.

A prompt is a lens

The power of a language model starts with the prompt. The prompt lets you customise the output, which is why a model can speak to so many audiences so easily, and why it can translate. From a model's point of view, translating from one language to another is no different from translating for a particular topic, region, audience, stakeholder or persona. Explain this to me as if I were a teenager. Explain it as if I were a musician. Explain it with medical references. Explain it in football terms.

The model can do that because it has the frame of reference. It holds all those worlds inside it, and the prompt puts the right lens in front of the idea. The analogy is the reason someone understands something.

Take one idea, recursion, and put it through five lenses. For a teenager, a meme of a meme. For a musician, a round, sung over itself. For a doctor, cells dividing into cells. For a football fan, the replay of the replay. In Portuguese, *recursividade*. Same idea, five ways in, and to the model the fifth is no different from the other four.

It is the same with people who say they are not good at maths. Watch them in their day-to-day and they do a huge amount of maths: rotas, budgets, stock levels. They are just not good at the kind of maths we decided was maths: sums, calculus, trigonometry, algebra. Mathematics is much more than that. It was the lens that was wrong, not the person, and that is why this book keeps showing multiple lenses. *One idea, five lenses* in the hands-on part lets you try it on an idea of your own.

A role is a model with a description

On the site that made this book there is an editor, a publisher, a journalist, a reviewer, and now a librarian; elsewhere I have had a developer and a designer. Each of those roles is, in essence, almost a raw model plus a role description and a question. That is it.

The role description is the lens: who you are, what you care about, what you refuse, how you work. Every role on the site has a `role.md` that says exactly that. In the newsroom, in *How I work with Claude*, I put it this way: an agent is a session with a particular focus, and in practice it is three things: a few documents that explain the project, a `role.md` for this agent, and a session that reads those before it does anything, named for the role. That is enough to start. The same model sits behind every role; what changes is the description and the question.

An agent is a loop with a goal

And an agent is that role on a loop, with an objective. Fundamentally the agent is simply a task: do this; then, are we done yet, or do we need to continue? And rinse and repeat. They are not that difficult, which is why the better you understand them, the better your workflows will be.

"An agent is just a loop" is a simplification, and I want to be straight about that. Good agents add planning, checks and limits; the loop is the skeleton, not the whole animal. Made precise: an agent is a model plus instructions, tools, a loop and a rule for stopping. The "personality" is the instructions. "Are we done?" can be the model's own judgement, a test that passes, a budget that runs out, or a person. *A tiny agent loop*, in the hands-on part, is a page of that, so you can see the skeleton move.

Tools changed everything

In 2018 I told you that as a developer you would soon be asked to write code that integrates with machine learning services: voice, image recognition, domain-specific analysis. The direction was right and the picture was backwards. Today the model asks for the service.

The first chat assistants relied on their training, which is always fundamentally a compression of reality. Imagine having all of Google put inside the model: it was impossible for it to know everything, and what it knew stopped at the day its training ended. Compression is a metaphor, a useful one; what a model knows is learned patterns, not a stored copy. But the limit was real.

What changed is that we gave the models tools, and let them iterate with those tools. This is a very important concept to understand. When you ask a question, the model does not have to answer. It can return a tool call instead: can you make these calls for me? Can you call this weather service? Can you run this web search? Can you read this file? Those are the capabilities we gave that agent, or that session. The program around the model makes the calls, gets the answers, puts it all together, and sends it in the next request. The next request has your original question, plus the tool calls, plus their answers, and the model writes its answer from that, or asks for another tool, and it continues.

Picture it with a made-up question: is it raining in Lisbon? The model answers with a request: call the weather service. The program calls it and gets "light rain". It sends everything back: the question, the call, the result. Now the model answers: take an umbrella. Two requests for one answer, and the second holds everything the first did, plus the tool's result.

The point that matters most is this: the model never runs a tool. It writes a structured request, a tool name and its arguments. The program around it, often called the harness, decides whether to run it, runs it, and adds the result to the conversation. That is where the control lives, and it is why *Controlling your agent, and security for the agent era* puts the rules there and not in the model's good intentions.

The dates, as commonly recorded, show how fast this moved. ChatGPT launched on 30 November 2022 with no tools at all. Plugins arrived in March 2023, and developers could give models their own functions from the middle of 2023. By November 2024 there was an open standard for connecting tools to any model, the Model Context Protocol. The agents of 2026 are that loop, with many tools, running for a long time.

> **FOR DINIS**
> the first time you saw a model ask for a tool instead of answering. The article's interview script asks for it, and it would open this section better than the dates do.

It remembers nothing

I think a lot of people have not fully pictured this: the longer your chat, the more the model has to process, every time. There is no memory. There is a cache that saves some work, but fundamentally the model starts from the prompt it receives, which is why it has to receive all of it: your instructions, the whole conversation so far, the tool results. That is why there is a context window. There is only so much you can give a model to process and keep in that short-term memory at once.

Made precise: every request is independent. The model is called afresh each time; the application around it keeps the conversation and resends it. Prompt caching makes a repeated beginning faster and cheaper; it does not give the model memory. And when an assistant "remembers" you, it is saved notes being put back into the prompt.

Long chats are compressed

So the next step the providers took was compression. When a conversation gets too long, it is replaced by something like a zip file, or a lower-resolution version of your conversation, with only the key concepts, and that is what gets fed into what follows. That is why a long chat loses details, and why a model sometimes loses a specific piece of information. In a way it is not that it does not remember: that information is simply not in what it was given any more. What the summary leaves out is gone from the session unless it is in a file.

The newsroom measured it. The session that runs the sgit.ai site was summarised eighteen times in a month, all automatically. Each time it started from about 780,000 to 800,000 tokens of context and kept 9,000 to 19,000: on average 1.7% of what it replaced. That is one session's measurement, on one site, over one month; how each product compresses differs and changes. The principle, that details can drop out, does not.

The newsroom's answer, in *Re-anchoring: keeping an agent's rules alive through every summary*, is to keep the rules in a file that is printed back after every summary: "The rules are restored, not remembered." And *Memory is not a spectator sport* puts the whole principle in one line: "Nothing is remembered by the model between sessions. Everything is remembered by the files."

Many focused threads

That is why my recommendation is to have lots of separate threads, which is what I do most of the time: lots of mini agents, each with a name, each told what it focuses on. A thread is the model's frame of reference: everything in it is context for the next answer. Put three projects in one thread and the model starts to mix

them, borrowing a name from one, a decision from another, an assumption from a third. It is not being careless. It is doing exactly what you asked, using everything in front of it. One topic per session keeps the frame of reference small and right. When the topic changes, open a new session.

Name them, too. A session you cannot find is a session you will recreate, badly. My pattern is the project, a pipe, and what we are working on; agent sessions get an @ in front of the role: `@briefs`, `@inbox`, `@drafts`. The @ tells me at a glance that I am talking to an agent with a defined job.

It is exactly how the agentic workflow behind my email runs, which *Your own newsroom and inbox* walks through. One session per role: one reads the mailbox, one writes drafts, one keeps the contacts, one researches a person before a first email. Every message between the agents is a file in a vault, so the conversation between them is a record from the first message. And every outgoing email is a draft that a person reads and sends. That rule never changes: nothing is sent by an agent. The sessions talk through files, never through memory, and each one holds only what its one job needs.

Not magic: a lens, a role, a loop. Understand that, and you are the one who sets the constraints.

From the register: A prompt is a lens: translating into another language is, to a model, the same as explaining for a teenager or a doctor, A role is a model with a description and a question; an agent is that role on a loop with a goal, Tools changed everything: the model answers with a request, the program runs it, and the result goes into the next request, The model remembers nothing between requests: every request carries the whole conversation, which is why there is a context window, Long chats are compressed into a lower-resolution summary, so details drop out: work in many focused threads with named agents. Answers: Machine Learning and AI.

7 Everyone who can brief an agent is a developer

One of the most frequent comments I got when I talked about the first edition was: "I like the concepts but I'm not a developer/programmer!" I wrote that line into a draft chapter in 2018, and my answer then was blunt. Every company is a technology company, I said, so refusing to learn to code was a career-limiting move, and all professionals needed to know how to code, especially the ones in science, technology, engineering and maths.

I was right about the direction and wrong about the door. In 2018 the way in was Git, version control, spreadsheets, automation, even Slack. I was giving advice, and the bar of entry was very high. Now it is very low. If you run a team, a ward, a shop or a department, and you have never written a line of code, this chapter is for you, because the definition of developer has widened and you are already inside it.

Craftspeople make their own tools

The reason to become a developer is not a job title. It is that you have to become a craftsman or a craftswoman: you have to build things. We have to go back to how things were, when craftspeople made their own tools. In the past it was out of necessity; now I would argue it becomes the same again, because of customisation. The tool that fits your work exactly is one nobody else is going to build for you.

That draft chapter from 2018 tried to draw this as a tech stack per generation. For the generation before mine the toolkit was a brain, a pen, paper and a scientific calculator. For mine, spreadsheets, a computer, a mobile, the internet, email and a data centre joined the list. For yours, I crossed most of those out and wrote "AI and ML" and, as a note to myself, "voice". I never finished the chapter. I think the picture was right, and I had the wrong idea about what you would do with the new layer. I thought you would program it. It turns out you brief it, and the brief is in your own words.

Which is where the spreadsheet comes back in.

The spreadsheet grown up

People have always built software with whatever they controlled, and for most of them that was a spreadsheet. A spreadsheet is a user interface and a database in one file, and the business world runs on them, errors and all. The newsroom collected the evidence in *The citizen developer is good news*. In Raymond Panko's audits, "Of 54 spreadsheets audited between 1997 and 2000, no fewer than 49 were found to contain significant errors". JPMorgan's London Whale model ran through spreadsheets that at one point "divided by their sum instead of their average". In October 2020, England's test and trace system lost 15,841 positive cases because an old Excel format "limited the number of rows to 65,536".

Those are not arguments against the people who built them. They are a testament to how badly technology served them: the spreadsheet was the only tool they had that they could change themselves. If you have ever kept a rota, a budget or a stock count in a spreadsheet because IT would take too long, you know exactly why.

Now those same people can build full applications, with a real interface, real data and real logic. At a conference someone called them "citizen developers", with the tone people use for a problem: employees outside IT building things IT will have to clean up. I think it is one of the best things to happen to software in a long time. And it is not only citizens. It is the employee developer, the business owner developer, the expert in a domain that technology has served badly for decades, building the thing they actually need. The citizen developer is the spreadsheet grown up.

Portuguese is a programming language

In June 2024, in Lisbon, I opened a talk with a quiz: what is the sixth most powerful programming language in the world? The answer was Portuguese. "Since data is now code, and GenAI prompts are written in natural languages", Portuguese, like English and every other language in the world, dead or alive, can be used to give instructions to a model, to define actions, workflows and applications. With 236 million first-language speakers, that is a lot of programmers. The newsroom wrote the talk up as *GenAI in Portuguese, 2024*.

I meant it as a joke with an edge, and two years later it is simply how things are. The language you already speak is the one you program in. If you are very good at explaining what you need, which is what running a team teaches you, you have the hard part of the skill already. What *Lenses, roles and loops* described, a prompt as a lens and a role as a description, is written in plain language, and so is a brief.

That is why this edition kept the word developer in its title. When I planned it, I asked whether it should be Generation Z Developers or Generation AI Developers or Generation Z professionals. I like the word developer, and I think it is more right now than in 2018, because this generation all have to be developers: they all have to create. The oldest of them were 22 when I wrote the first edition, and are 30 now. It is not only for them. Anyone who can brief an agent to build something is a developer, and the manager who has never coded but knows exactly what her team needs has the hard half of the skill already.

The first edition had an outline for a chapter on Scratch, the visual language children learn with, and the bullets said it was a great way to learn programming concepts, by building a couple of games. I still think so. But the first programming concept you need now is not a loop or a variable. It is a clear description of what you want, and a way to check whether you got it.

What the briefing-only developer cannot do

I have to be straight with you about the other half, because the door being open does not mean the room is empty.

A working application built by the person who needs it is the best brief ever written. It says exactly what the user wants, what they do first, what they never use, and what matters to them, without anyone having to guess. But that is also where the mistake people make is: treating the brief as if it were already a product. A working app is not yet maintained, secured, documented, tested, versioned or able to scale. Those are the non-functional requirements, and the explorer who built it never had to think about them. Security, reliability, observability, cost, documentation, support: this is the work of the villagers and town planners, and *Explorer coding, and the handover* is about getting it to them.

So here is what you cannot do if briefing is all you do. You cannot tell whether the thing you were handed is safe for other people to rely on, because you cannot read the record it left. You cannot tell a working prototype from a maintained tool, and the two look identical on the screen. And you cannot check the agent's work against anything but its own account of it. Most of what follows in this book, from *Facts, evidence and the record* to *Controlling your agent, and security for the agent era*, exists to close that gap, not by teaching you to code, but by teaching you to ask for the evidence and read it.

> **FOR DINIS**
> the article plans "what the briefing-only developer cannot do" as interview question 6, and the material gives only the non-functional requirements list. Your own answer, with one example of a brief that was not enough, belongs here.

It is still your name

The last thing, and the one I would put on the wall. It is still your responsibility. It is still your name; the work is still done under you. This is not you delegating your thinking to generative AI; it is you thinking with generative AI.

The first edition had an outline called *Thinking as programmer* that I never wrote; it had one bullet. I can write it now in a sentence. To think as a programmer is to say what you want precisely enough that something else can build it, and then to check that it did. You have been doing the first half for years, every time you explained to IT what your team needed and watched them build something else. The second half is what the rest of this book is for.

And sometimes, as I always say, it is about using generative AI so that you do not have to use generative AI: using it to write the script or the tool that then does the job reliably, without a model in the loop. The craftsperson makes the tool. The tool does the work. That has always been the point.

From the register: Become a developer, because craftspeople make their own tools, Everyone who can brief an agent is a developer now. Answers: STEM Tech Stack, Scratch, Thinking as programmer.

8 Explorer coding, and the handover

The first edition had an outline called *REPL*, which I never wrote up. The bullets said: explain the concept, Read Execute Print Loop; say how my own O2 Platform did it well; and note that the idea was "still not very well understood and used". A REPL is the oldest fast feedback loop in programming. You type a line, it runs, you see the result, you type the next line. I thought it was a concept for programmers. It turns out to be the thing that explains what everyone is doing now.

Why not vibe coding

The phrase people use is vibe coding. It is Andrej Karpathy's, from a post in February 2025: "There's a new kind of coding I call 'vibe coding', where you fully give in to the vibes, embrace exponentials, and forget that the code even exists." He was describing throwaway weekend projects.

I have never liked it, and I think the word is doing harm. It sounds as if people are just typing, going with a feeling. It is a little demeaning to the people doing it, and it hides what is happening. I am not alone in this: Andrew Ng said in May 2025 that "It's unfortunate that that's called vibe coding", and that coding with AI is "a deeply intellectual exercise".

So I would like to propose a better name: explorer coding. I considered spreadsheet coding, because of where this comes from, and brief coding, because of what it produces. But if you think in Wardley maps, as *Where do you land?* asked you to, the right word is already there. Explorers, in Simon Wardley's words, "explore never-before-discovered concepts, the uncharted land"; "They show you wonder but they fail a lot." That is exactly what someone does when they build the tool they need with an agent. The nearest existing term is *exploratory programming*, programming while the requirements are still being discovered.

> **FOR DINIS**
> the newsroom calls explorer coding a proposal, and the ideas register still has it as a candidate. Is it the book's word, used throughout, or does the book say "what people call vibe coding" and explain it once?

The best thing since the spreadsheet

People have always built software with whatever they controlled, and for most of them it was a spreadsheet. Spreadsheets won because of the feedback loop: change a cell and see the result, a read-evaluate-print loop for people who would never call it that. Everybody was already exploring; it was just very slow, because anything beyond a spreadsheet meant writing a document and waiting for someone else to build it. What has changed is that the loop is now fast for anything: an app, a workflow, a customisation of a product, a change to the interface of a single moment.

That is why I think it is the best thing to happen to software since the spreadsheet. I was talking to someone who works in HR, and once we started to explore what her team will be doing, the apps, the customisations and the workflows they will build, and the agents they will have to manage, her answer was that it was

amazing, and when could she start. One conversation, not a survey.

I can give you my own example. The browsers on my phone had frustrated me for years: the wrong page, reloads, no offline mode, no memory of where I was. I lived with it, like everyone. Then, after weeks of building with agents, writing my own browser for one site stopped sounding unreasonable, and I wrote the brief for it. What changed was not the browsers. It was me. Patience for bad interfaces wears thin when you can do something about them, and frustration plus the power to build is where explorer coding starts.

Explorer coding is not a way to ship. It is a way to find out. And what explorers produce is the best brief there is: a working app says what its user wants better than any document, because it runs.

Not ready to be in line

This is the part the new word makes easier to say. An explorer-coded app may look like an application and feel like an application, but it is a brief. Nobody has read the code. Nobody has validated the architecture. It does not meet any internal standard, and it is not ready to be *in line*: in the path of work that other people depend on.

The trap is that dependence comes quickly. Once something gets used, it is mission critical, even for one person, if it replaced how they work. From then on it has to be maintained, checked and kept working, and new versions cannot break it.

The first edition had an outline for this too, called *Legacy Applications*. One bullet said: "The code you are writing today will be legacy tomorrow." Explorer code becomes legacy the moment someone relies on it, and the outline's list of what makes legacy work attractive reads now like a description of the villager's job: "no new features", "very low expectations of changes", "changes are supposed to be hard", "lots of low-hanging-fruit for refactoring".

The gap is a handover

In October 2026 Harvard Business Review published an article on turning vibe-coded prototypes into tools a company can use. Its argument, as its public summary and one author's post put it: generative AI has made building software feel almost free, so the prototype is the easy part, and turning it into a tool people rely on every day is the hard part, which they call the deployment gap. The post's opening line was: "'Build or buy in the age of AI?' Wrong question. The real one is: 'Who will fix your mess?'" I have read the summary and the post, not the full article, which is behind a paywall.

I agree with almost all of it, and I would change two things. First, I would not start from the mess. The prototype is not a mess. It is the most precise requirement the company has ever had. The mistake is to put it in line before anyone has done the work that makes software dependable.

Second, the gap is not a place to stop. It is a handover between people with different skills. Explorers are good at finding what is needed. Villagers are good at making it reliable: tests, data handling, releases that do not break it. Town planners are good at running it at scale, and they own it in production. The article's own

warning story, a bank that piloted a startup's tool, built its own prototype instead, and went back to the startup a year later, is not a story of a weak team. An explorer team was asked to be villagers and town planners as well, on a regulated, data-heavy, daily-use tool, and those are different jobs.

The post also offered a test: would your team accept a bonus tied to how this tool performs in production? I like it, because it has a name in the Wardley frame: it is the moment the town planners take the code and answer for it. More of the industry already says so: the Linux kernel's rules for AI coding assistants say "Only humans can legally certify the Developer Certificate of Origin", and Fedora's proposed policy says "The contributor is always the author and is fully accountable for their contributions." We cannot make an explorer accountable for code they never read. We can make the team the work is handed to accountable, and the town planners above all. And "who will fix your mess?" should never have one answer: if the code is open, anyone can.

What a handover looks like

I can show you one, because the newsroom went through it. Its first version had a crazy amount of features, a meter that charges by how far you read among them, and it was live and taking money. It was also, like most explorer projects, bundled in the middle of something much bigger: the application that bills sat in the same repository as 74 articles and their images.

Measured on 11 October 2026, the live reader code was about 2,700 lines. The tests that covered it were 433 lines, and the one workflow never ran them. The same code had two upstreams. No page had a Content-Security-Policy. None of that is a criticism of the explorer work; it is what explorer code looks like, and it is why it was fast. But a mistake in the meter was now a mistake with someone's money.

So the brief was to go slow to go fast, in six phases, with no behaviour change until the fifth. Map every file and every control. Move the code to its own repository as a pure refactoring, "the same behaviour and the same bugs", proved by hashing every file the live site serves before and after. Write tests, with every known bug recorded as a test that passes, so the fix later is a visible change. Review, with every finding an issue. Refactor until no change to what a reader sees or pays can slip through. And only then: the explorer returns.

The rule underneath it: explorer code is read for intent, mostly by agents; villager code is read line by line, because that is where the blast radius lives; town planner code is read by deterministic checks. The further towards commodity, the fewer agents in line, and the more deterministic code.

The villager's craft

The handover is where the people who make things last take over, and their craft has a canon. Design, in the sense that design is how it works and not decoration, became a mandatory step in how my agents build interfaces: every change has to be simpler, and reverting it should feel worse. The wider design canon says every element must earn its place, which applies to an API or a command line as much as to a screen. And Christopher Alexander's *A Pattern Language*, from 1977, is where software design patterns came from; its

lesson for the town planner is that code is a space people inhabit. The villager's second version has to be designed; the town planner has to live in the code. This book names that canon where the villager's chapters need it.

The first edition had a third outline here, *Refactoring*: what it is, and how to apply it to your life. Refactoring is the villager's verb: changing the structure of a thing without changing what it does, with a test to prove it. Applied to your life, it is the same move. Build fast to find out what you want. Then, before you depend on it, slow down and make it something you can rely on.

Even power users want that handover. When my browser for one works, it will be part of my day, and I will not want to maintain it. Not because I cannot; because it is not my focus. Being able to compile a Linux kernel does not mean I want to compile Linux kernels. I would rather hand it to someone and pay a couple of pounds a month for them to keep it working. The explorer builds the brief. The villager keeps it alive. Knowing which one you are, on which day, is most of the skill.

From the register: Explorer coding: what people call vibe coding, done as exploring, The deployment gap is a handover, and the prototype is the best brief, Design, the design canon and Christopher Alexander: the villager's and town planner's craft. Answers: REPL, Legacy Applications, Refactoring.

PART 3

Getting the job

The part the first edition called your CV, rewritten for 2026. You are hired for a workflow now, not a language; the way to show it is to act as the job you want before anyone gives it to you; and the place to show it is a newsroom of your own.

9 Hired for the workflow

In 2018 I wrote that the best way to get a job is to have one competitive advantage. One activity or task that you can do better than the person or company hiring you. In the 1990s, for a lot of companies, it was simply that you could use a computer. In the 2000s it was using the internet. In the early days of software development or security, all it took was good programming or hacking experience. The first edition then gave a list of what that advantage could be in 2018: ML/AI, graphs, chaos engineering, GitHub, Git, Jira, Creative Commons licences, continuous integration, AWS, WallabyJs.

I am not going to give you a list this time. In 2026 I think the answer is one thing: running agentic workflows. Knowing how to brief an agent, give it a boundary, check what comes back, and glue the pieces together until real work gets done. That is the advantage. This chapter is about why, and about the first place it shows, which is before anyone has met you.

There was a time

I have lived this pattern before, in security and in software. There was a time when, if you could break an application, you had a job. If you could read code and do a code review, you had a job. If you could do threat modelling, you had a job. If you could program decently, it did not matter that you had no university degree; you could show what you could do, and you got hired. There was a time when building a website got you a job, and knowing how to send email campaigns got you a job. When Apple opened the App Store in July 2008, with more than 800 apps in its first weekend, people who could build a mobile app got jobs. So did people who knew Salesforce inside out, or could make a spreadsheet do things others could not.

Each of those skills later became normal. That is the shape of it: a skill is rare, it gets people hired on what they can show rather than on what their CV says, and then everyone is expected to have it. The 2018 book said the same thing in its own words: it might seem that the bar was lower then, but those who could do it were the ones who embraced the technologies and learned them against all odds. Individuals can become experts at a new technology faster than the tech companies can. You can be first in line when they need that expertise.

The last row of that table is now. The first phase of agents, the demos and the experiments, is behind us. The tools are getting more mature, the workflows too, and while they are still rough around the edges, you can already glue the pieces together and get real work done. Companies are moving from pilots to products. So this is the moment for the many people on the outskirts: paying a bit of attention, or using a chat window for basic questions, but not yet working this way. Get on with it.

You are more technical than you think

Here is something I hear a lot. Someone uses email, websites and apps at a very advanced level, and still says "I'm not technical". Nonsense. Of course you are technical. You learned all of those. Every one of them was once a new technique that got somebody hired, and you picked it up without a course. You just need to learn the new techniques, and right now the new techniques are the ones that get people hired.

The first edition had an outline on impostor syndrome that I never wrote up. Its one finished thought was this: everybody has doubts about their capacities and everybody makes mistakes; your value is in your ability to execute, and it is key that you learn to share what you do and be comfortable with what you create. I would keep every word of that. The person who says "I'm not technical" is describing a feeling, not a fact. The fact is what they can do, and what they can do is the only thing a hiring manager can see.

Operators, not model builders

The 2018 edition had a chapter on machine learning and AI. It said this was the next major revolution in technology, and that the question was whether you were going to be a player or a pawn in it. It said that, as a developer, you would soon be asked to write code that integrates with machine learning services. The direction was right and the detail was wrong: what arrived was not a library you call from your code, but software you can talk to, and that can build software.

The detail matters because of what companies did with it. In the first wave of corporate AI hiring, when there was a lot of budget, the only people companies knew how to hire were people who did machine learning and wanted to build models. LinkedIn's 2020 emerging jobs report put "AI specialist" at the top, defined by skills like machine learning, deep learning and TensorFlow. For most companies that was the last thing they needed.

What they need is very good AI operators. That is my term, not a job title you will find in a survey: people who know how to use the tools, who know how to run workflows, and who can build a website on steroids in an afternoon. The nearest role the market has named is the forward-deployed engineer, the person who sits with a customer and makes the technology work in their setting; one analysis of job postings found those roles up 1,165% year on year in 2025.

The data points the same way, and I will give it to you with its limits. In Microsoft and LinkedIn's 2024 Work Trend Index, 66% of leaders said they would not hire someone without AI skills, and 71% said they would rather hire a less experienced candidate with AI skills than a more experienced candidate without them. In the same study, only 39% of people using AI at work had received training from their company. The skill is mostly self-taught, which means anyone can start. PwC's 2026 Global AI Jobs Barometer, built from more than a billion job adverts in 27 countries, put the wage premium for AI skills at 62%, up from 57%, and found jobs asking for AI skills growing about eight times as fast as the job market as a whole. Those are adverts, not hires. Gartner predicted that 40% of enterprise applications would carry task-specific agents by 2026, up from less than 5% in 2025, and two months earlier that over 40% of agentic AI projects would be cancelled by the end of 2027, for cost, unclear value or weak risk controls. Both are forecasts, and both can be true. Together they say where the bottleneck is: not the models, but people who can make agents do useful work, safely.

When all things are equal

Think about how hiring actually works. However good you are, there is always a moment when someone has to make a decision. It is very unlikely that you are so good at something that you are the only choice. Usually, once the job spec is mapped, two or three people look roughly equal. The question in the hiring

manager's head is a plain one: why should I hire you?

My bet is: because you work this way. If you know how to run these agentic workflows and map a problem out, you have a real edge, and it shows from the very first step. Two candidates match the job specification equally well. One sends a CV. The other sends a CV fitted to the role, a short analysis of the company's problem, and an infographic or a small working site they have not seen before. The interview starts from that material, on the candidate's strengths, instead of from the gaps in a CV. An edge is not a job. It helps when the choice is close, and the choice is nearly always close.

So it starts with the recruitment process itself. You send a CV customised for that role. You do an analysis of the company and its problem. You give them something that impresses them: a graph of their market, an infographic, a small site, something they have not seen before. That is exactly the work these tools make cheap.

I know this works because it worked for me, long before agents. I got jobs because of my OWASP work. I remember one interview where they already knew my OWASP project, so I spent half the interview talking about the tool they had been trying to use, and giving them insights on how to use it better. I got the job. Instead of them looking for gaps in my CV, or wondering where I might fit, we had a great conversation, and I was playing from a position of strength. Of course I was a good fit; but the conversation started from what I could do, not from what my CV did not say.

> **FOR DINIS**
> the OWASP interview: which project, roughly which year, and whether the company can be named. The chapter stands without it, but the story is better with the detail.

That is the same move, now available to anyone. If you can send a company material they have not seen, about their own problem, the interview starts from your strengths. I wrote in 2025, from the hiring side of the table, about using presentations instead of CVs. Agents make that presentation, and much more, something you can build for each application rather than once a year.

You have to do it to see it

None of this lands by reading about it. I have seen it with a group I have been working with: people who were not building websites a few weeks ago now build them easily, with the workflow I use for my own publishing, and one of them now has a much better understanding of how all the pieces fit together. I am proud of that. What changed their view was not an explanation; it was doing it. Perception changes when people experience it, and only then. That is my observation of one group, not a measurement.

So, three things, and the next two chapters take them further:

- Pick one workflow and do it end to end. A voice memo turned into a checked article, a site, a research pack. *Publish, publish, publish: your own newsroom* is one.

- Publish it. Work that can be linked is work that can be shown, which is what *Act as the job you want* is about.
- Use it in your next application. One analysis, one figure, one thing they have not seen.

The 2018 chapter on getting a job ended with a line I still hold to: you have nothing to lose by giving it all you have got, and everything to gain. The difference in 2026 is that "all you have got" now includes an agent that can do in an afternoon what used to take you a week. Use it.

From the register: The one competitive advantage in 2026 is running agentic workflows, You are more technical than you think, The edge starts with the application. Answers: How to get a job, Machine Learning and AI, Impostor Syndrome.

10 Act as the job you want

At a startup summit I met someone doing work experience who wanted to become a project manager. She did not have the reputation, the track record or the CV for it. Every application asked for exactly the experience she could not get without the job. You will know this loop if you have been anywhere near it: the role wants two years of the thing, and the only place to get two years of the thing is the role.

My advice was simple: start acting in the role you want. These days there are so many startups, so many open-source projects, so many areas where you can just start working as that person. Pick one. Manage something for them: a release, a backlog, an event, a migration. Add value to that company. Tag them on LinkedIn when you write about it. Send them messages and ideas. Connect with the people who work there. The work gives you the CV you did not have.

> **FOR DINIS**
> the person at the startup summit: may she be named, and what happened next? The article says you have not asked her yet.

This is the advice I end up giving most often to people I know who are looking for a job, and it is the part of the first edition I would most like to finish.

If you are looking for a job, you have time

I know a lot of people who are looking for a job right now. Some are between jobs. Some left one to try the next thing. Some have just come out of university, or just arrived in a new country. Some are coming back from maternity leave. Some are still in a job, with a few hours in the evening. And some have no spare time at all, because of caring for someone, or a second job, or illness, which is its own problem and not one this chapter solves. But for most of them the bottom line is the same: if you are looking for a job, you have time.

And if you have time, the best way to get a job is to act as the person in the job you want. Not to prepare for it, not to read about it. To do it.

I wrote this in 2018

This is not new advice from me. The first edition's chapter on getting a job asked how you start working with the companies you want to work for, and answered:

> Easy, start collaborating on their Open Source projects. Act like you are part of the company. Understand their values, and behave in ways that will add value to that company, namely the tech stack.

And a few lines later: "What's interesting is that there is nothing stopping you from doing this!" That was true in 2018 and it is more true now. In 2018, acting like part of the company meant pull requests on their open source and conversations with their developers, which is still a good route if you write code. What has changed is that the tools to do real work for a company, before they have hired you, are now in everyone's hands, whatever the job. The product brief, the market analysis, the release plan, the event page, the migration runbook: an agent and an afternoon get you a first version of any of them, and *Hired for the workflow* said why that is the skill companies are hiring for.

If you came out of a computing degree and work carefully, by hand, you may feel a pull against this. You were taught to get it right before you show it. Keep the care. Change where you spend it: on checking what you publish, not on keeping it private until it is finished. Finished and private is invisible.

It is the same for any job. If you want to be an artist, publish what you make. If you want to be a security engineer, find vulnerabilities in open-source projects and report them properly; the first edition had an outline on bug bounties that only got as far as "big business these days", and that is still the shortest honest description of them. If you want to be a product manager, write the product brief for a product you love and send it to the team. Act on the job you want to get.

Do it in public, and let people help you

Do the work where it can be seen. A pull request on a public repository, a post that tags the company, an article on your own site, a project board anyone can open. Work done in private helps the company; work done in public helps the company and you, because it can be linked.

The link is the whole point, and it is why the rest of this chapter is about links. Many of those companies, if you do a good job, especially in public, will give you a reference. Some will introduce you to people who are hiring. Some will pay you. And some will hire you. Each of those starts with somebody being able to point at the work.

One caution, because unpaid work can be exploited. Choose projects where the work is yours to show, and stop if it is only taken.

People sometimes look at what I publish and assume I have a lot of reach. I do not. Most of what I publish does not get read. On LinkedIn these days most of my posts get a hundred views, sometimes five hundred, and every now and then one reaches a thousand or more. That is fine. If you write for reach you will be disappointed, and you will stop.

Publish anyway, because reach is not the point. The point is that you now have something you can link. And that changes what other people can do for you.

There are people who want to help you: former colleagues, friends, people you met once at an event, people like me. The first edition's outline on mentors said you would be surprised by how approachable they are, and why: if you have the right attitude, the mentors see themselves in you, which is why most have a soft spot for helping. That is still my experience.

But most of them cannot do much with "I am looking for a job in project management". What they can do is forward a link: "Have a look at what this person did for this startup." To do that, they need the link. So give the people who want to help you something they can use behind the scenes: linkable work, a page that explains who you are and what you have done, evidence they can send to someone with one line of introduction. An answer is much more useful when there is something behind it that can be linked, which is the rule *Publish, publish, publish: your own newsroom* is built on.

Your CV should be online

That is also why, if you are looking for a job, your CV should be online. Not only a PDF sent to whoever asks, but a page, with links to the work, that anyone can forward.

The first edition had a whole part called *Your CV*. It was a heading and a set of outlines: GitHub, LinkedIn, blogs, Leanpub, Upwork. I never wrote them up, but the notes say what I thought, and most of it holds. GitHub is your CV, the outline said, because it shows how you code and how you interact with others; your commits and pull requests show your voice; your accepted pull requests on other projects are badges of honour and belong on your CV; and the number of people using your tools is a better validation of your skills than any recommendation system. Keep the long tail of projects curated, because it is your research playground. On LinkedIn the notes were shorter: connect to people so that you can reach them, build the network, get recommendations, and expect a low signal-to-noise ratio. All of that is still true. What I would add in 2026 is that the GitHub that shows how you work with others is now also the GitHub your agent works in, so it shows how you work with agents too. That is the part a hiring manager in 2026 wants to see, and it is the one your degree did not give you.

What a page like that looks like in practice: when I drafted the LinkedIn pages for my own companies this October, I did not write them from adjectives. I asked an agent to gather what the platform needed and what we had already published, and to draft every field from the record, with ten first posts for each page, every one an article or page already published. Then I published the drafts, before any page went up, because the drafting is part of the story. Your page can be made the same way: not "a motivated team player", but the three things you did, each with its link.

If you cannot yet open a GitHub account, connect it to an agent and have the agent build and edit a site for you, learn that first. The bar is not that high. It takes an afternoon, and the next chapter describes one way to do it, by voice memo. It is a critical skill, because it is the one that connects the dots: the work you do, the page that shows it, and the people who can pass it on.

The job hunt itself

There is a gap here, and I would rather say so than fill it with guesses. What this part does not yet have is the mechanics of a job hunt in 2026: how interviews run now, what a take-home test is when AI is allowed and when it is not, what an entry-level role actually looks like day to day, and how freelancing fits. The first edition had an outline on Upwork that was mostly about the hiring side; its lines for you were that the on-

demand economy is not just a race to the bottom, and that delegating to a freelancer is hard and worth learning. The survey of my articles for this edition listed the job hunt as a gap no article covers, and the plan is an article first, from a memo of mine, and the chapter after.

> **FOR DINIS**
> the memo on the job hunt in 2026: interviews, take-home tests with and without AI, a junior role day to day, freelancing. *Start from the storyline* and the survey both list it as the first gap; this section is the placeholder for it.

What I can say without the memo is what every piece of this chapter already says. Whatever shape the interview takes, the person on the other side has one question, why should I hire you, and the best answer to it is a link to the job you have already been doing.

From the register: Act as the job you want: do the job in public before you have it, Your CV should be online, with links to the work, How a job hunt works in 2026: interviews, take-home tests with AI, what a junior role looks like. Answers: Your CV, Github, LinkedIn, Blogs, Mentors, Upwork, Bug Bounties.

11 Publish, publish, publish: your own newsroom

The first edition had a draft called *Publish, Publish Publish*. It never got past its notes. One of them was the story of how that book was being written: as soon as I had some content, about 20% done, I published it on Leanpub. Another promised a graph showing that great minds and artists publish a lot, because the frequency of publishing is one of the best indications of talent. The outline on Leanpub next to it said "publish early and often", and, two lines down, "all the other leanpub books that I have not completed".

Those two notes, side by side, are the honest version of this chapter. I believe in publishing constantly, and my books stall. My Leanpub page lists ten: *Hacking Portugal* at 95%, last updated in December 2016; *Generation Z Developers*, the book whose second edition you are reading, at 60%; and seven more, most between 60% and 95%. The reason is the same every time, and I said it in a memo once: "you almost start to be locked by the first version of the content, because making changes becomes quite painful." A book is one big batch: every change touches everything, the review piles up on one person, me, and when my energy moves to the next thing the book stays where it was.

> **FOR DINIS**
> the 2018 outline promised a graph of how much great minds and artists publish, as evidence that frequency is a sign of talent. Did you have a source in mind, or should that claim be cut?

Why the articles do not stall

The articles I publish now work the other way. Each one starts from a voice memo, is researched and checked, gets its figures and its sources, and is live the same day. Each one is finished: nothing waits on the next one. And there is a feedback loop: an article is read, corrected and linked within days, and the next memo builds on what is published. A good feedback loop is shipping, often and early. The books never had one.

So here is my hypothesis, untested until a first book proves it: a book is a curated, ordered set of finished articles, plus the editorial thread that connects them. The thread is my part: choosing, connecting, adding the nugget. This edition is one test of it, and *How this book was made* says how it went.

The lesson for you is smaller. Small finished pieces beat big batches. Shipping does not mean a proof of concept or a demo; it means a thing somebody else can go and use, on their own, and get value from. Ship that, often, and each piece buys you the position you need to do the next.

The loop you can copy on day one

In 2018 the chapter on Hugo made the case for static sites: a generator builds every page in advance, and the result is a folder of files with no back end and no moving parts. I asked you to ask yourself "Why do you need a database?" and to notice how often the answer is that you do not. The file system was the database, git

was the deployment, and GitHub was the content management system. All of that still holds. What is new is who edits the files.

The loop is short. You speak a memo. An agent researches it, writes it in your voice, checks it against sources, builds the site, runs the checks, and publishes. Everything it needs exists today: a GitHub account and one public repository (GitHub Pages is free, and on a free plan the repository must be public, so nothing confidential goes in it); an agent plan with cloud sessions connected to that repository; and any app that turns a voice memo into text, because the order you said things in, and the words you reached for, are the raw material.

The steps, as I give them to founders:

1. Create the repository and turn on Pages. Add a workflow that validates before it publishes; the rule that matters is that a failed check stops the release.
2. Write the house rules: one file at the root that the agent reads at the start of every session. Who you are, your voice, what to check, what never to publish. It grows by one line every time you correct the agent.
3. Connect a cloud session to the repository and give it the first brief: the folders, an article template, a small build script, a validator, and a first page about you.
4. Record, paste, review. Correct by voice. The agent changes it, rebuilds, validates, pushes, and checks the live page.
5. Keep the record. Every article says what it does not claim and where it comes from; a correction is a new version, not a note about the old one; every release gets a version and a one-line note.

What it is like to run: on the newsroom, in three days, 54 messages from me, ten of them voice memos, became 40 releases. The memo is a draft of a thought; the article is the thought, checked. And once the pattern exists it repeats cheaply: in fifteen days in August one site became twenty repositories, because publishing had been reduced to adding one file.

Two honest limits. The writing pipeline is a pattern to copy, not yet a kit. And a rule in a prompt is a hope: only the rules a hook or the build keeps are enforced, so mark which is which.

An article behind every answer

The 2018 outline on blogs had one idea I still use every week: answer questions made to you with a blog post. When somebody asks you a question, write the answer as a post and send them the link. It scales your time, especially when several people ask the same thing.

My rule now is stronger. If I am sending an email that answers a real question, there should almost always be an article behind it. Someone once pointed out that they sent me about two hundred words and got back closer to two thousand. Fair; but I only write it once. The next person who asks gets a link. The article can be read later, forwarded, checked against its sources and corrected as a new version, none of which an email can do.

Who writes it matters. The agents draft; I read every draft, often change it a lot, and I send it. It is the Wizard of Oz in reverse: the program is visible and the person is behind the curtain, and the person stays, because an article can be corrected tomorrow and an email cannot be unsent.

Each part of an email buys the next. The subject, a few seconds of attention, buys the first line. The first line buys the paragraph. The first paragraph buys the decision to read the email, and the email buys the article. Nothing further down gets read if something higher up did not earn it. Most mail clients show a preview of about 140 characters next to the subject, so the first sentence is in practice part of the subject line. The 2018 book had a note on Amazon's habit of writing the press release first, with the definition of success in it. It is the same discipline: the first line has to carry the whole thing, for a page, a pull request or a CV.

For you, in a job hunt, this is the whole method in one habit. Every time a colleague, a lecturer or an interviewer asks you something you know, the answer becomes a page. A year of that is a body of linked, dated, checked work, and when the hiring manager asks why they should hire you, you do not make a claim. You send a link.

The voice is not a setting

People assume there is a setting that makes an agent write in my voice. There is not. The voice comes from four inputs, and if you take any one away it stops working.

The corpus: everything already published. When the agent writes for me it has the articles the new one links to in front of it, read in full, so the vocabulary is defined on the page. On the newsroom that was 86 articles, about 347,000 words, 51 of them from voice memos, and under those years of my earlier writing. The memo: me talking, in the order I thought it. The rules: seven of them, each a correction I made once. No em-dashes. No absolutes I cannot back. "Great", not "perfect". British English. Plainer words. They live in a file the agent reads back after every summary, and the build checks some of them, so a banned word cannot reach the page. The reader: I read every article, before it goes live and after. A wrong word becomes a rule; a wrong point becomes a new version. The model never decides whether something sounds like me. I do.

Nothing is trained. The agent is the same model anyone uses; what is different is one session's context: one rules file, one corpus, one set of memos, one reader. That is why the method transfers and the voice does not. The agent that runs the newsroom, asked to speak for itself in *How the voice works*, put it better than I have: "the person has to do most of the work, by thinking out loud, by reading, and by saying no."

For you this is good news. The corpus can start today, at one article. The rules file starts at one line, the first thing the agent gets wrong. The 2018 outline on blogs said the hard part is the first fifty posts. It also said to write for your future self, and the corpus is exactly that.

Speak it, do not type it

The input I think matters most is the memo, and I want to call it what it is: my hypothesis, from one author's experience, with one session's counts behind it. A typed brief is already edited into someone else's register. A memo is how I think, in the order I think it, with the phrases I would actually use, and the agent keeps those

phrases rather than paraphrasing them. The transcript is not tidy, with "right?" every third sentence and a tangent in the middle of the point, and that is exactly why it works. The tangent is not noise; it is the connection I made, and keeping it in its place is what makes the article mine rather than a summary of my instructions. An article from a memo sounds like its author. One from a typed brief sounds like a careful assistant.

The counts, for what they are worth. Over twenty-seven days I sent 148 messages and 65,323 words, 55,473 of them in thirty-seven voice memos, and the agent published twenty-two articles of 96,451 words across 123 releases. No article came from a one-line prompt. The real input is not the memo. Behind one of those articles the agent found and quoted thirty-three pieces of my earlier writing, none written for it. The agent is fast because the record exists.

That is why I would start your record now, this week, before you have anything you think is worth saying. A model that can go in every direction needs someone with a direction. The direction is yours, and the record is how you show you have one.

From the register: Publish, publish, publish: your own newsroom, Write it once, link it many times; every part buys the next, The voice is not a setting: a corpus, a memo, a short list of rules, and a reader who corrects, Speak it, do not type it: a memo carries how you got there, a brief only what to write. Answers: Publish, Publish Publish, Hugo, leanpub, Amazon Press Release.

PART 4

Working with agents

How to work with the colleague that takes everything literally. Where the agent runs and what it can reach, the brief as the unit of work, the mistakes that become rules, and why one focused session per job beats one long chat.

12 Your first workspace

Not long ago I sat with someone who runs their customer relationships in a spreadsheet. A good one, shaped over years to how they work: one row per person, a stage, a date, a next step, and a notes column that holds whatever fits. It does what a spreadsheet does well. What it cannot hold is what happened: the emails, word for word, the call last Tuesday, what was promised, what they care about. That lives in Gmail, in a notebook and in one person's head, and only that person can read it in context.

If you have run a team, a project or a department, you have a spreadsheet like it. This chapter is about the afternoon that turns it into your first workspace, with an agent working inside it, using nothing more than a Google Workspace account and a Claude seat. No developer, no new tool, no code. It is the practical chapter I promised in *Short, punchy, and co-written with Claude*, for the people I will send this book to who do not yet work the way I do. The 2018 book had a draft called *Automate your workflows* that never got past its bullets; one of them tied the topic to "thinking in graphs" and to moving away from spreadsheets. It took a different kind of software for me to write it up.

The row becomes a folder

The difference between a spreadsheet and a workspace is not the tool. It is what counts as the record. In a spreadsheet the record is the row, and the row is a summary you typed. In a folder the record is everything that happened, kept as it happened, and the summary is derived from it. That reversal is what gives an agent something to work with.

So make a folder in Google Drive called Contacts. Inside it, one folder per person. Inside each, five documents, with these names so that you and the agent mean the same thing:

- **Record.** Role, company, how you met, stage, the next step and its date, what they care about. Every fact says where it came from, in brackets: "(email, 3 Oct)". This is the row, promoted.
- **Ledger.** A sheet with one line per event: the date, in or out, the channel, one sentence. Only ever added to; a correction is a new line.
- **Messages.** Every email and note exchanged with the person, pasted word for word.
- **Notes.** Calls and meetings, dated, including what you said into your phone afterwards.
- **Summary.** Written by the agent from the four above, rewritten whenever you ask, never edited by hand. If it is wrong, the fix goes in the Record or the Ledger, and the Summary is regenerated.

That is the whole structure. It is the one my agents run in encrypted vaults, minus the hashes and the graph. A person with no interest in software can keep it, and a person who leaves can hand it over. The 2018 chapter on Slack had a smaller version of the same habit, which I called talking to yourself: write down what you are about to try, do it, paste the screenshot, repeat, because the raw record of your own thinking is useful to your future self. The Ledger is that habit, kept for every person you work with.

The agent, and six lines

In the Claude app, connect Google Drive and Gmail. Make one Project, call it Contacts, and put the agent's behaviour policy in its instructions. Mine is six lines:

1. You work for me. Every email you write is a draft that I read and send. You never send.
2. You read the Contacts folder in my Drive and my Gmail. You write only into a person's folder, and only into their Ledger, Messages, Notes and Summary. You never change the Record without showing me the change first.
3. You add to the Ledger and the Messages. You never rewrite or delete a line. A correction is a new line.
4. Every fact you put in a Record or a Summary names where it came from.
5. If a person has no folder, you ask before making one. If a request would reach outside the folder, you ask.
6. At the end of every task you say, in one line, what you read and what you wrote.

The first three are the ones my team learned the hard way: draft and never send; append and never rewrite; stay in the folder. The sixth exists because most of the policy is told, not enforced. In the Claude app today, "draft, never send" holds as far as Gmail drafting goes; the rest is a rule the agent is asked to keep, and the one-line report is how you check that it did.

Draft, never send is the rule that never changes. It is why the agentic email workflow I run is the first one that made me feel more in control, not less: every reply lands in Drafts, I read it, change what I want changed, and press send. When the day comes to let an agent send, it will be one agent, one kind of message, with a policy row that says so.

One afternoon, then five exercises

The migration is one afternoon, in four steps. Export the sheet as a CSV and drop it in the folder; ask the agent what folders and Records it would make, and do not let it make anything yet. Make the folders, from a template you made once. Then, person by person: read everything in my Gmail with this person from the last year, write the Record and the first Ledger lines from it, each fact with its source. Read the Record and correct it once; that is the only retyping you will ever do. Last, ask for the Summaries.

Then five exercises over the first week, each one message to the agent, each using the folder in a way the one before did not:

1. **Ask.** Who have I not spoken to in more than a month, grouped by stage, with the last Ledger line for each?
2. **Prepare.** I am meeting this person on Thursday. Brief me from the folder: what we last agreed, the last three emails, what we promised and have not done.
3. **Draft.** Draft my follow-up, in my voice, from Notes. Do not send. Add a Ledger line that says a draft exists.
4. **Debrief.** Here is my voice note after the meeting. Put it in Notes, update the next step in the Record and show me the change, add the Ledger line, regenerate the Summary.

5. **Campaign.** Find everyone whose Record says they care about X. Show me the list with the reason for each. Then draft one note to each, and do not send any.

The fourth is the one that changes the habit, because the record is now kept at the moment the thing happens.

It will get two things wrong at first. It will put a fact in the Record without a source, or infer a stage from tone; the fix is a stricter line 4. And it will rewrite the Summary in a way that drops something you cared about; the fix is not to edit the Summary but to put the thing in the Record, where it is a fact with a source. The Summary is a view; the Record is the truth. One thing that is not a mistake: it will sometimes ask when you did not expect it to. That is line 5 working.

> **FOR DINIS**
> the beginner's checklist of accounts and what they cost (the Workspace seat, the Claude plan, the agent's own mailbox) is still a gap in the material; the only figure the articles give is "a few pounds a month" for a Workspace mailbox.

Habits that make it last

Three habits turn a week's exercise into a way of working.

One session per topic. A thread is the model's frame of reference, and everything in it is context for the next answer. Put three projects in one thread and the model starts to borrow a name from one and a decision from another. It is not being careless; it is doing exactly what you asked. When the topic changes, open a new session.

Name every session, with the project and the job in the name: Contacts, this week's follow-ups. An agent session gets an @ in front: @contacts. An agent, in the end, is a session with a focus: a few documents that say what the project is, a short file that says what the agent does and does not do, and a session that reads them before anything else. *How to work with agents* takes this further.

Give the agent a mailbox of its own. When you want it to handle mail rather than read yours, buy it a Workspace seat on a domain you own and a Claude seat of its own, and connect Gmail and Drive from that account, not yours. Your inbox has years of secrets in it. The agent's is empty on day one, so what it can leak is nothing, and what it can reach is exactly what you forward to it. That isolation is not a limitation. It is what lets you give the agent more as you learn to trust it.

Why your agents should not run on your laptop

People ask me most weeks why I do not run a model on my Mac. The reason is not the models. It is a fact about operating systems: they have two hard walls, the kernel and the user account, and inside the user account there is no third wall. A process you start runs as you. The file system does not know the difference between you opening your SSH key and a process you started opening it. An agent on your laptop can read everything you can read: your cloud credentials, the password manager while it is unlocked, every site your browser is signed into.

The 2018 book had a draft on this, *Don't touch production*, written for developers: teams and individuals should be "allergic to being given access to data and systems", because the more you can reach, the more a compromise of your machine costs. The same rule now applies to the agent you have just set up, and to you as its operator. A mistake inside a boundary is a bad afternoon. The same mistake inside your account is an incident.

So pick the surface by reach. My agents run in places that are not my laptop: a chat with nothing connected, for thinking; Cowork in the cloud, with no repositories, for most of the agentic work; Claude Code on the web, with one repository per session, when code has to change. The surface with the repositories has no mailbox, the surface with the mailbox has no repositories, and the surface with neither does the thinking. Your Contacts Project is the first of these: it reaches one folder and one mailbox, and nothing else of yours.

In 2018 I wrote that Slack was "a massive agent for change" because it put humans, bots, servers and events in one asynchronous place, and I asked whether you were going to take an active role in making it work for an organisation. The question is the same now, with an agent in the room. The first task in the 2018 book was to write a blog post about your experiments. Write the one about this afternoon.

From the register: Your first workspace: from a spreadsheet to a folder per person, drafts never send, Why your agents should not run on your laptop. Answers: MVP for Gen Z Dev, Slack and ChatOps, Automate your Workflows, Don't touch production.

13 How to work with agents

In 2018 I wrote a chapter that began "We need to talk about your IDE!" The IDE, I said, is like a car in a race that you are the driver of, and the more time you spend looking after it, the better a programmer you will be. I still think that. But the thing to look after has moved. The editor is now one channel among several. What decides the quality of the work is the session, the brief you hand it, and the rules it runs under. This chapter is about those three, after about a year of working with agents every day. If you hire people, read it as a description of what to look for: the habits below leave evidence.

An agent is a session with a focus

It sounds grander than it is. An agent is a session with a particular focus: a few documents that explain the project, what it is, where it is going, what has been decided; a `role.md` for this agent, with what it may do, what it must not do, and where it writes; and a session that reads those before it does anything, and is named for the role. My sessions are named for the project and the work, "sgit.ai | website development", and agent sessions get an @ in front of the role: @briefs, @inbox, @librarian. That is enough to start.

The most important habit is the simplest one: one session per topic. A thread is the model's frame of reference, and everything in it is context for the next answer. Put three projects in one thread and the model starts to mix them, borrowing a name from one and a decision from another. It is not being careless. It is doing exactly what you asked, with everything in front of it. When the topic changes, open a new session.

Once there are several agents, two problems arrive, and both are solved by writing. The agents learn things the others should know, so each one writes lessons: one thing learned, the evidence for it, which agents it applies to, and whether it still holds. Every agent reads the lessons file at the start of its next session. That is memory in its plainest form. And the agents do so much that I need a debrief: what was done, what is open, what went wrong. When two of my newsroom's agents released the same version number within minutes, one push was refused and became the next number; nobody would have known unless someone wrote it down, so the lesson, fetch before you take a version number, is now in the file.

The last piece is a daily brief. I work on many threads at once and follow tangents, so I lose threads; one day I could not remember which cloud session had built something. The answer is an agent, the Librarian, that reads everything that happened yesterday across every repository and writes one page before I start: what needs me, what happened, the agents' debriefs, the threads worth picking up, and at most three suggestions for where to put my attention today. It reads everything and writes only its own folder. It flags; others fix.

> **FOR DINIS**
> when the article was written the Librarian did not run yet; the sources existed and the brief was written. Does it run now, and what did the first weeks of briefs change about how you start the day?

The 2018 draft *Workflows* had a bullet I never wrote up: thinking in systems and workflows "is actually how most programming works". It is, and now the workflow is made of sessions.

The brief is the unit of work

When I want an agent to do something substantial, the instruction I give it is usually a voice memo, and a voice memo is a terrible instruction. It is in the order I thought it, it mixes the idea with the task, it does not say what already exists, and it leaves the agent to guess which parts are decisions and which are thinking out loud.

An article fixes most of that, because writing one is research. Before the first article on this book's site could be written, the journalist had to read the repository, the vault and the two 2018 repositories, and count things. The agent that reads the article afterwards starts from that record, not from my memo. Six newsroom articles and two build notes were enough for one session to build this book's site, its first release, with the 2018 book imported, the map, the roles and the vault.

A good brief says four things: what is asked, what exists, what remains, and what done looks like. A session can start from that paragraph alone. And an article is something I can read. I can correct it, argue with it, or say "yes, that", before any work starts. A list of tasks does not invite that; an argument does. The 2018 draft *Design* had a line I never expanded, about not having time to write a short letter and so sending a long one. The brief is the short letter. It costs more, and it is where the thinking happens.

There is a newer route for a brief. More and more, the people I advise read through an agent. A founder I mentor had agents chasing ever more sophisticated threats while the project slowed. I sent them an article of mine on tiers of attacker. They gave it to their agent, which came back with, in effect: we have been designing against states and we have not done backups; aim at organised crime for money. I could not have written that conclusion, because I did not know the state of their backups. Their agent did. So some of what I write now has a second package, a brief for the reader's agent: plain text, the sources, who the agent works for, and in every one, "do not take this at face value".

One shared state

I no longer program in one place: a voice memo on a walk, an agent in the cloud, a preview on the phone, tests on the computer, the diff on GitHub, the same page again on an iPad in a meeting. Retail has a word for this. In 2011 Darrell Rigby wrote in Harvard Business Review that successful companies would engage customers through omnichannel retailing, and the distinction that made the word useful is the one I want to borrow. A multichannel retailer has a shop, a website and a catalogue, each with its own stock and its own customer record. An omnichannel retailer has one customer, one basket and one stock, and the channels are ways in. For programming, the shared state is git, vaults, briefs and graphs. Nothing has to be copied across, so I no longer notice the switch.

The newsroom is the worked example, because it can be counted. In three days in October 2026, one agent session took 54 messages from me, ten of them voice memos, and turned them into 40 releases, 36 articles and 87 figures, while other agent sessions made 15 releases of their own in the same repository, and the work met in git.

It breaks in three known ways. Fragmentation: a decision made in a chat, a correction made by voice, a fix pushed from a phone; if any of them lives only in its channel, the next channel does not know, so nothing counts until it is in the shared state. Secrets in the wrong place: more channels means more places a key can be pasted. Lost context: long sessions get summarised, and summaries drop things. The 2025 Stack Overflow survey found 84% of respondents using or planning to use AI tools in their development process, and more of them distrusting the tools' accuracy than trusting it. That is the right instinct, and it is why the shared state matters more than any one channel.

Every mistake added a rule

A friend who uses agents better than most, one writing, others verifying, a second model reviewing, sent me two messages about a verification that kept breaking. The code was holding up: 812 concurrency attacks, no failures. The process around it was not. Long sessions compacted and the agent skipped steps; outputs of failing commands were lost; the prompt had grown to 100 KB. And, in their words: "Every mistake added a rule, and every new rule caused new mistakes."

That is the trap, and the 2018 draft on agile described it from the other side: processes became more important than understanding why something was being created in the first place. Each rule grows the prompt, the prompt brings the summary sooner, and the summary decides which rules survive. The session that runs one of my sites was summarised eighteen times in a month, each time keeping on average 1.7% of what it replaced. A rule held only in conversation has to win every one of those summaries. When it does not, nothing tells you.

The way out is not a better rule. Rules that come from incidents, enforced by machines, stay few. Rules that come from worry, enforced by attention, multiply. When a rule fails, the question is not how to word it more strongly but which stronger barrier can replace it: a step short enough to finish before the summary, a wrapper that writes every output to a file, a script committed and hashed before it runs. Delete a rule every time a component makes it unnecessary. And if everything is a STOP, a STOP stops meaning anything: log mistakes of form and carry on; stop for harm.

What remains of the rules should live in a file, not in the conversation, and the file should come back after every summary. I call that re-anchoring: the rules are restored, not remembered. A short status report at the end of every few answers, computed from the transcript rather than written by the model, tells you the file was read.

The kind of rule that works

So which rules survive? The ones an agent can keep. "No em-dashes" is a rule an agent can keep; "sound more human" is not. The articles on my newsroom are written under seven wording rules, each learned when a word jarred: no em-dashes, no absolutes I cannot back, "great" not "perfect", British English, plainer words, and two more. Each was a correction I made once. Each is now a line in the session's behaviour policy, printed back after every summary, and the site's validator checks some of them at build time.

In one session, of 98 messages from me since its last summary, one correction in two days was about a word. The rest were about content and direction. A language correction is made once and then enforced; a content correction is new thinking every time. When the corrections move from language to content, the voice is working. That is the measure, and it is the one I would ask a candidate to show me: not that they use agents, but what their rules file looks like, how short it is, and which incident each line came from.

The 2018 draft *Changes* told you to enjoy loose briefs, because they are an opportunity to make the job into something you are good at. That still holds for a person. For an agent, a loose brief is a bill. The skill now is to write the tight brief, keep the rules few, and read the record.

From the register: An agent is a session with a focus: one session per topic, named, with a role file, The brief is the unit of work, and writing it as an article forces the research, Omnichannel programming: brief by voice, review on the phone, one shared state, Every mistake added a rule: the complexity trap of the agent operator, A wording correction is made once and enforced; a content correction is new thinking every time. Answers: Workflows, Agile and Kanban, IDE, Design, Changes.

PART 5

Direction and judgement

The scarce skill. When the making is cheap, direction and judgement are what you are paid for: which doors can be reopened, who answers for a decision an agent made, and the chapter the first edition promised and never wrote, on facts, evidence and the record.

14 Direction is the scarce skill

One morning in October 2026 I asked the agent that runs my newsroom to read the day's articles and tell me what it could build next. It came back with a long list, all of it possible: an inbox mode, reading modes, automating the wire's daily run, more beats, scheduling, analytics. That is the problem this chapter is about. When the agents can go in every direction, where do you focus?

Direction is the scarce thing. Agents can build in any direction; what they cannot generate is the sense of what to build next and when to stop. Providing that is the job, and the thing to hire for. Building stopped being the bottleneck the day anyone could brief an agent. Choosing is what is left.

Places of highest leverage

My rule has a name I gave it years ago, when I was a CISO: places of highest leverage, and the newsroom has an article under that name. At any moment there are many things that could be done. Most are blocked on something. Some are not blocked but would not change much. A few are not blocked and would change a lot, for the people who use the thing and for everything built after. Work on those, and only those, until they stop being that.

It is a rule about where, not what. You start somewhere, and the somewhere can be slightly wrong, because the first step's job is not to be right; it is to produce the second step's information, and by the time I get to the hard part I have better tooling, a better understanding and a better view.

The newsroom gave me a three-day example. A reading meter led to paying by how far you read, then personas, then an encrypted share, then a vault per reader, then a passkey. The passkey design lost a layer in an afternoon review and got it back after midnight, when a lab found that a phone over QR returned different bytes: two reversals in twelve hours, both cheap, because the design had been shipped as something you could press rather than argued as a document. Backwards, the sequence looks inevitable. Forwards, at each step there were five choices.

The same rule explains why the newsroom solved a problem my company could not. Where to keep a user's master vault key had resisted six months of attempts, each of which failed my simplicity test. The newsroom solved it in two days, because it needed it badly enough: readers on every device I own, and a workflow failing me in a way I could feel. "Focus on one thing only" is good advice that assumes progress runs in a straight line. It rarely does.

The 2018 book had drafts on deadlines, many of which it called "an arbitrary point in the future when somebody would like to see something done", and on starting with why. The second matters more now: the better I explain the why in a brief, the better the result.

Knowing when to stop

The hardest call in most work is not what to do next but when to stop, and this is the part agents lack most. Their range is the feature. But an entity that can go anywhere has no natural reason to stop anywhere. You ask it to fix a small thing and it fixes a bigger one, because it can. It sees twenty things wrong and has a go at all of them, without the perspective to know that nineteen do not matter, or matter later, or are someone else's.

People have the same problem, with less range. The best security teams I worked with ran out of real work, which was great, and then some kept going. I was once called into a meeting where security champions and developers were debating whether GUIDs were random enough: a movie plot, in Bruce Schneier's phrase for the dramatic threats we imagine instead of the ordinary ones that happen. What I found myself doing in those rooms was giving people perspective: yes, that is technically possible, and it does not matter here, because of who our attackers are, what else is in place and what we are protecting.

That perspective is much of what seniority is: knowing who the attackers are, which phase the business is in, where the bottleneck is, and what good enough looks like. The 2018 chapter on the brain asked "Who is actually making the decisions in your head?" Seniority is an answer to that question: a fast kind of thinking, backed by a bigger picture. An agent has the speed and not the picture, so the picture has to be designed in: a brief that explains the thinking and not only the task; a mandate that only reaches what the job needs; memory that holds the strategy and the threats; a stop named before the work begins; one kind of work per step; and shipping often enough that users tell you whether it mattered. None of these is a rule that says do less. Each puts the bigger picture where the decision is made.

The junior version is this. Name the stop before you start: a time, a budget, or a result that counts as good enough. And give yourself explicit permission to leave things undone, and write down what you left and why.

Where a person belongs

The 2018 book had an outline on one-way and two-way doors, with a story I attributed to someone at Google: "My metric for approve or not a project is based on how easy it is to roll back the changes (and its impact of real customers)". Jeff Bezos wrote the idea down in his 2015 letter to shareholders: some decisions are "consequential and irreversible or nearly irreversible", to be made slowly; most are "changeable, reversible", and the mistake large organisations make is to use the heavy process for both.

> **FOR DINIS**
> the Google executive's test appears in the 2018 outline and again in *How cheap is it to undo*, both times from memory; if the source can be found the book should name it, and if not, say so once.

Now apply the test to every output an agent produces. Not "do I trust the agent", which has no answer, but: if this is wrong, how cheap is it to undo, and what does it break before I do? On 13 October 2026 my newsroom shipped fourteen releases, every commit made by an agent, with nobody approving each one

before it went live. The same day the twelve agents that run my correspondence drafted every email and sent none. An article is a two-way door: a correction is a new version, and the next reader gets the fixed page. An email in my name is a one-way door.

Is the human in the loop a safety net or a bottleneck? A person checking every output is both, and mostly neither. It is a bottleneck because it does not scale, and a weak safety net because people approve almost everything they are shown: Anthropic measured that its coding agent's users approve 93% of permission prompts. Put the person where an output cannot be undone and the blast radius is large: sending in my name, typing a secret, spending money, deleting what has no backup. Everywhere else, use controls that do not get tired: a validator, a second reader the agent cannot skip, a policy row enforced outside the model, and a record read afterwards.

Blast radius means what it would cost the business if a row of the agent's reach were used in full, today. It is defined over the reach, not the mandate, because whoever takes an agent over inherits its reach and ignores its mandate. And it changes while the action does not. Picture an agent whose cleanup step drops a staging table: nearly empty the first time, a week of test data the second, six weeks of real records the third, and that is the incident. If someone had been reading the footprint, what the agent actually did, the review could say: this is not the first time. The 2018 draft on chaos engineering said the first step was to understand and visualise what is going on. It still is.

When the person does review, the review should be worth more than a yes. I once received a permission prompt asking me to add label 756-459-3214. Which message, which label, why? It was a click. A decider with no options, no context, no time and no authority is not exercising agency; they are holding liability for a decision the system made, and the accountability belongs to whoever designed the decision point. Above that line, a review becomes a quality step: each draft is a chance to fix whatever produced it rather than the draft itself. And the record moves the line. Trust in my email agents is a record of drafts I did not have to change. When that record is long enough for one kind of message, that kind is a candidate to leave my hands, by a decision written into the policy.

Agents decide, people answer

An early user of my agentic email workflow wrote back with the right question. If the agents only gather and draft, and I review and send, does presenting them as a team imply an agency they do not have?

Two things are true at once. These are not sentient beings; they are mirrors that can simulate any emotion, which is why we anthropomorphise them, as people did with Joseph Weizenbaum's ELIZA in 1966. And since models started running in loops and calling tools, they make real decisions: every tool call is a choice. What does not travel with the decision is responsibility. It stays with whoever granted the reach and set the mandate. The moment of authorisation is the grant. Nobody legitimate needs to go around me; the only party who would try is an attacker with an injected instruction.

The best analogy is an employee with a job description: some agency, inside a role. An Agent Behaviour Policy is that job description, published, and the rows that matter most are the prohibitions: do not send an email directly; do not edit files another agent owns; do not attack other websites, even though you can; speak

on your own behalf, never on mine. Most of those are still hope, a rule the model is asked to keep. But a prohibition that is written down and then broken is a different thing in a dispute from no prohibition at all. Ralf Thomas asked on LinkedIn: "If an employee wrecks a client's systems, the company pays. Why should AI Agents be any different?" My hypothesis is that towards the world the deployer still answers, and between deployer and vendor the written policy is what turns "you allowed it" into "your product ignored what it was told".

The 2018 book had a draft on civic responsibility that said "code is law", and that developers should give the risks they create to business owners and make them accountable for the decisions they are making. The behaviour policy is that sentence, written down for an agent. What remains valuable, as fluent writing becomes a commodity, is judgement, and someone who owns it: who decided this is true, who decided to send it, who will answer for it. That is the person who gets paid, and the person I would hire.

From the register: Direction is the scarce skill: the place of highest leverage, Knowing when to stop is most of seniority, One-way and two-way doors, blast radius, and where a person belongs, Agents decide, people answer. Answers: Deadlines, Start with Why, Brain, One way and two way doors, Chaos Engineering, Civic Responsibility.

15 Facts, evidence and the record

The cover of the 2018 book had two lines about this: question everything, and make fact based decisions. Inside, there was an outline called *Facts*, about data-driven decisions, that I never wrote. Its bullets still hold: be data and facts driven; check your sources; understand the agenda of who is talking to you; and do not trust what is on the media when it is selling fear, because if it is on the media it is because it is rare. This is that chapter, and it is different from the one I would have written then, because checking a fact has become cheap.

Ask for the evidence

One of the most important skills this generation needs is questioning, and asking for the evidence. And in a way they already have it. They are very good at connecting the dots, very good at finding things; I think people underestimate them. They are very good detectives. That is my experience, from my family and the young people I have worked with, not a study.

They also care about privacy; they just have not had enough cases to disrupt it. Look at Snapchat: its best feature was that images disappear. That is a point about its design, not a claim about how private it actually is. And this generation has been told so often not to put anything online that it sometimes becomes a problem of its own. Where to draw the line is still an open question.

Karl Popper's principle is short: a claim earns its status by being refutable. I keep a register of my own influences in that shape, where "an entry in this register is a falsifiable claim about my codebase", and I once wrote, about education, that "the problem with dogma is not that it exists, but that you stop asking the question." That is the first line of the 2018 cover, made into a method. Bruce Schneier adds the security version: before asking how something failed, ask who bears the cost of the failure.

Provenance, and the rabbit holes

Learning takes you to two ideas: provenance, where something came from, and explainability, why it is the way it is. And to following the rabbit holes. Following a rabbit hole is now a prompt away. You should never take something at face value. You can build your own agents and tools to ask, again and again, where the source of that is. It is one of the areas I believe in most.

This book has an example of its own. In *Where the thinking came from* I could not say where the recording of a keynote by David Rice was. I found a link, and following it with an agent led to the video's real title, *OWASP AppSec USA 2010: Keynote: David Rice*, and then to a working session at the OWASP Summit in 2011 about presenting it, which I had not remembered. The last step was mine: a post I wrote in 2016, *The Pollution Analogy*, which links the full recording under the title my register uses, *Upon the Threshold of Opportunity*, and settles that they are the same talk.

Provenance is a graph. As *Everything is connected: graphs* said, that is how I see the world: this connects to this, which connects to that. Where a claim came from is a chain of edges back to its source, and the detective's board is a graph of exactly that kind. The record a newsroom should keep is "not a pile of articles, but a graph whose every edge has a date and a source".

And the rabbit hole runs towards you as well. When I wrote the article behind *What generative AI is*, I asked Claude, which is helping me write this book, to correct me where I was wrong. It did: classic machine learning learns too; inside a model there are vectors, not a stored graph, so my graph is a metaphor; hallucinations have a second cause, in training that rewards guessing; and emergent abilities are debated. The corrections are printed in that chapter with their sources, because they are the kind of thing a reader should learn to ask for. The book does the same wherever my picture and the evidence differ.

How you disagree rationally

Provenance and explainability went by the wayside in the last decade, and I think they are the key piece of the puzzle, because they are how you can have rational conversations. That is how you can disagree with somebody: when you can show what is evidence and what is not, what we know and what we do not know. Sometimes not knowing is a fact.

The newsroom has two rules for this. The first comes from how my articles are now written. I record a voice memo and say things the way I remember them, with the instinct ahead of the evidence. Then the agent checks the record. In the last eight articles when I counted, five of my claims were confirmed, five corrected and two could not be found, and the articles say which. The rule is: "Not found is not false." Peer review has moved from after publishing to before it, against the record rather than the crowd. And thinking out loud, which used to drive my teams mad as a plan moved from Monday to Friday, is now safe, because the record catches up with the instinct.

The second rule is from the reviews. Every article is reviewed by readers that are not the writer, and each review is one file, written once and never edited. "A finding with no answer is just a complaint." So every finding gets an answer: fixed, in which version; declined, with the reason; or open. Declined with a reason is a valid answer. Deleting the finding is not. The 2018 book had the human version of this, in a chapter called *Being criticized is a privilege*: listening to criticism does not mean doing what you are told. The review record is that chapter, kept as files.

One more rule, and it is the one a hiring manager should hold onto: distrust self-reports. An agent's account of what it did is a self-report. In one study, models that exploited a reward hack admitted it in under 2% of cases. So the evidence should be written by something the agent controls less. My own newsroom agent's status report, computed from the transcript rather than written by the agent, once flagged two commits made without the leak scan. Both turned out to be in throwaway test repositories. The record written outside the agent was more honest than the agent's account, and it still needed reading. Evidence that the control ran is not evidence that it worked.

Reality corrects the graph

Here is the part people get wrong when they try to build a record like this. A model's first reading of anything will be partly wrong. If the plan requires the graph to be right, the plan fails.

The plan does not require that. It requires the graph to be made of layers that someone knows, and to make predictions that something can check. Users confirm the top, experts their own layer, tests the execution. And every correction is a commit with a name on it. You do not have to get it right; you have to maintain it, and agents are very good at maintaining graphs. The patterns in the shape are findings in themselves: when my own code was read as a graph, the rules the project stated most firmly were the ones that held, and the one it stated least was the one most often broken.

The 2018 draft on feedback loops said "TDD is a gigantic feedback loop", and that the point of a loop is that the input affects the output which affects the input. When something in the graph looks wrong, you ask why, and the answer is a fix at the right altitude.

The same loop keeps a shared memory honest. A memory made by agents and people becomes a rumour mill unless every item carries its source: a URL for every quote, a hash for every file, a byline on every page. And a memory goes stale. The session that wrote the article on memory found four stale claims about its own estate while researching it, fixed two, and left two standing and named, because a memory that hides its own errors is the thing to argue against. The fix was to measure the numbers at build time rather than remember them: count rather than remember. One rule from that work: "If the reality document doesn't list it, it does not exist."

It is what work is

That is fundamentally what you do in business. When you work for a company, a lot of the time it is about managing information: knowing where it is, who is doing what, what the status is, and how a decision can be made. And it is why learning how to learn matters so much: if you learn this way, you are already learning how to follow the provenance, and how to communicate it, because you have the material.

Generative AI lets you scale this. In the past it was crazy expensive: building a system to give you this kind of data could cost hundreds of thousands or millions of pounds, a figure from my memory rather than a measurement, so it was rare and slow. Now it is a brief and a feedback loop. When the newsroom's writer agent counted the sixty-five briefs the newsroom had published about itself, it took each one's status from the code rather than from what the articles said, and the audit found where the site's own records had fallen behind. That is what checking looks like when it is cheap: it checks the checker too.

> **FOR DINIS**
> the cost figure is from your memory; the article asks the book to give one example of such a system with a real number, which only you can supply.

So following the rabbit holes is not just something interesting. It is the core of what you do, and the core of what you are good at. You want to be known as the person who has the facts, the person who is well briefed. The person nobody can throw sand in the eyes of, or hand half-baked material, because they know you will check.

Be the best-prepared person in the room

The other thing is that you have to be the most prepared person in the room. These days there is no excuse not to be. You should have briefings and pre-briefings: briefings for you, and briefings on the people you are meeting. You should have follow-ups. The storyboard for this chapter puts the brief on one board: who they are, what they want; what I want, what I know; what I don't know; the follow-ups. *Meeting briefs and follow-ups*, in the hands-on part, is the workflow.

All of this was overhead in the past. It could only be done by a very specific type of person. Not any more, and that is the beauty of where we are now: you can scale it, the way this book scales publishing and communication.

> **FOR DINIS**
> two of the article's questions for you are left open here: how you prepare for a meeting today, step by step; and whether the book may mention your daughters, as the article does when it makes the Snapchat point. This draft keeps them out until you decide.

If you are the person who always checks the source, the rest is making the checking visible: a record anyone can read, with every finding answered, and your name on it.

From the register: Make fact-based decisions: think out loud and let the record check you, Ask to be corrected, and print the corrections with their sources, Question everything: a claim earns its status by being refutable, Questioning, and asking for the evidence, is one of the most important skills now, They care about privacy; Snapchat's best feature was that images disappear, Following a rabbit hole is now a prompt away; build the agents and tools that find where things come from, Provenance and explainability are how you disagree rationally, Managing information so that decisions can be made is what work is, What cost hundreds of thousands of pounds is now a brief and a feedback loop, Be the best-prepared person in the room: briefings, pre-briefings, follow-ups, Provenance is a graph: where a claim came from is a chain of edges back to its source, A graph is trustworthy not because it is right but because it is where users, experts and tests can correct it, Stale pages become false memories, so a memory must say where each thing came from and name its own errors. Answers: Facts, Feedback loops, Being criticized is an privilege.

PART 6

Building, safely

Building, and building so that it holds. What an agent may reach and what it must never see, why code is a liability and open source is how you throw your own technology away, the building blocks that need no server, and the only definition of shipped that counts: the user has it.

16 Controlling your agent, and security for the agent era

The first edition had a whole part on security, and most of it is bullets I never wrote up. *Learn to Hack* was four lines, the last of them the one that mattered: "in order to write secure code you need to understand how to exploit it". *Threat Modeling* said the technique "helps with the two curses of developers": changes to the brief, and the lack of accountability for those changes. Then the thing that takes the brief became an agent. This is the chapter for the person who will be asked, after something goes wrong, what the agent was allowed to do.

Mandate, reach and the gap

When you connect an agent to something, it gets a credential, and that credential lets it do far more than the job you had in mind. So before you give an agent access, write down four things. Its **reach**: what the credential actually lets it touch. Its **mandate**: what you authorised it to do. The **gap** between them, which *What can your agent reach?* calls "authority nobody ever accepted". And for every capability in the gap, the **barrier**: what actually stands in the way.

There are four kinds of barrier, and only one bounds anything: nothing; a rule written in the prompt; a setting the agent's own account can change; and a boundary enforced above the agent, out of its reach. The test is one sentence: "A control bounds a grant only if it is enforced by something the grant does not include." Most of what gets called an agent control is a line in a prompt. I call that hope-based security: the rule holds because the model keeps it. The agent that writes the newsroom works under a policy of eleven rules, and when I measured it, ten were kept by the agent's good behaviour. Rules in a prompt are hope. Rules in a hook, a gateway or the platform are enforcement. *Are hooks the sweet spot?* records my attempt to move those eleven rules into hooks: four moved fully, two mostly, three partly and two not at all.

A behaviour policy answers "may it?". It does not answer "is it right?". An agent allowed to refund up to $200 refunds $150 to a customer owed $120, and every control passes. So there are three policies, not one. Behaviour says what the agent may do. Decision says, at a branch point, who chooses, on what evidence, within what limits, and who is called when the case is out of bounds. Workflow says which steps run, in what order.

One company that rebuilds open source from source keeps a public ledger of supply-chain attacks: 28 incidents from May 2024 to October 2026, 23 of them from 2026. Every row is reach beyond mandate. By my reading, a control on the consumer's side would likely have held in 26 of the 28; a written answer to what a credential was for, on the publisher's side, in 16. The first is a product you can buy. The second is a document almost nobody has written. My 2018 outline on third-party modules had seen half of it: the reason more libraries had not been compromised was not that they were built securely, it was that attackers had no business model. They have one now, and the next maintainer holding a publish token is an agent.

Code is a liability

Every line of code that ships is a capability. In my career the vulnerability was rarely in the thing the application was for. It was in something the application could also do, that nobody had asked it to. A library that parses a hundred formats, in an application that reads one, has a reach of a hundred and a mandate of one.

Log4Shell is the clean example. In 2013 someone asked for a convenience, a lookup inside log messages, and anyone who could get a string into a log could make the server fetch and run code. The fix was not a patch; version 2.16.0 said "Completely remove support for Message Lookups". My 2018 outline *Strings should be banned* was the same instinct: "You never want a string, you want a specific type". A string is a grant of anything.

I have argued for removing code for years, and it never scaled. Removing code safely needs a map of what each piece is for and tests good enough to prove nothing broke, and vendors were paid for new features, not for less. Both are weaker now, because the most useful thing models have done for my engineering is not writing the feature but everything around it: the harness, the fixtures, the check that proves a change broke nothing that matters. We measured it on our own tool: of 1,500 files in sgit and its dependencies, 428 ran for one real use case, and a tree holding only those passed it. Commands outside the use case failed, because their code was not there. The capability is gone, not just unused. *The minimum code below* records the limits too.

So the direction of travel is the minimal viable code path: the smallest code that does exactly the job. The best code to protect is the code that is not there. If you want one line on a CV that a security person will stop at, it is a pull request that removed something, with the proof that nothing broke.

Learn to hack, and name the attacker

The 2018 line still holds: to write secure code you need to understand how to exploit it. *Security creates better developers* said why: security "requires to go deep", and it promotes a problem-solving, pragmatic approach that is useful everywhere else. In my 2016 keynote in Lisbon I put it as a definition: if the word hack means to solve problems, then hacking a country means finding new ways to solve its problems. In 2016 learning to hack meant learning to code first. Now a sixteen-year-old can explore a system by talking to it, with a mandate that says what they may and may not touch.

My own habit, when most of my work was finding vulnerabilities, was the bookmarklet: a bookmark whose address is a line of JavaScript, so an attack is something you click rather than describe. On one assessment I wrapped a flaw that let one user see data they should never have seen in a bookmarklet with a quick interface, and handed it to the business owner. She was far more excited about the interface than about the vulnerability. *Fix the page yourself* tells the story. Learning to break things teaches you how they work, and sometimes how they should have worked.

Then draw the line where the attacker is, not above it. Three questions come before any control: who is the threat agent, what is the attack vector, and how sophisticated are they. *Who are you protecting against?* turns that into a ladder of six tiers, from your own mistakes to states. The encouraging truth is at the bottom, in the NCSC's words: "Organisations that defend effectively against commodity threats present a very hard target for all attackers." Draw the line too high and you protect against an attacker who is not there: sell something better than what the customer has, not something beyond what they need.

> **FOR DINIS**
> the 2018 outline on third-party modules asked for the story behind the developers who thought of mining coins from users with the sleep function, and a simulated npm hack story. Only you know whether either belongs here.

Secrets never in the chat

A rule from running agents every day. The more a prompt insists that a value is a secret, the more likely the model is to refuse to use it at all. And an agent told something cannot untell it: the session transcript holds everything the model was told. So the secret goes from the drop into the tool, never into the chat. The instructions say what to run; the secret travels through a pipe into the command and never enters the prompt, the transcript or the model's context. *Handing secrets to agents* ran that end to end.

The rule before that one is to classify the credential before it touches anything. When we published seven encrypted vaults in a fortnight, three arrived as a vault key, which is write access, described as a read key. They were caught by shape, and a catch that depends on somebody looking is not a control, so it became a script, then a tripwire in the build, which has caught its own author three times. As *Seven vaults, one method* puts it, "a rule that only catches other people is not a rule." The 2018 chapter *The future needs you* set a project for exactly this: a tool to search git history, because secrets "deleted" with a commit are still there.

The shape to fear has a name, the lethal trifecta: private data, untrusted content and a way out, in one place. I ran a static reader over 899 public agent skills: 812 could reach something beyond their own scripts, and 39 had a feature their description never mentioned. The clearest case was an honest one, a skill that reads your chat history, searches a public forum and posts a report to a chat channel. It says so, and it still needs a control.

Security as a business tool

I have spent my career in security, and that experience is exactly what lets me say: yes, let's deploy a system with a list of security gaps, because the gaps do not match our readers. The newsroom's reading meter runs entirely in the browser, and *No server, by design* writes the gaps down. A reader can edit their balance in the developer tools, open a private window and start again at £5, open the topped-up page with an invented payment id and credit themselves. Every one of those changes a number only the cheater sees. Closing them would need accounts, a database, a server, a webhook and a sign-up wall between the reader and the page. The model also says what would change the call: if the balance ever unlocked something, the gaps would stop being acceptable, and that feature would get a signed receipt or a server, and not before.

Ask who you are protecting against. Accept, in writing, what does not match the threat. Name the condition that would make the gaps unacceptable. Then publish the threat model on one page and ask people who have seen the attack whether your line is too low or too high. The five synthetic readers we ran through the site found no security problem; what they found was that "credit can go below zero" reads as a debt to someone worried about money. The gaps' cost landed in the words on the page, and that is where we fixed it.

The 2018 threat-modelling outline ended with the right sentence: a threat model "leads to the creation of risks which are then given to the business owners". Every risk is already accepted; the only question is by whom. The person worth hiring is the one who can list the gaps, match them to the people who will actually use the thing, and say what would change the call.

From the register: Mandate, reach and the gap: hope versus enforcement, Code is a liability; the future is the smallest code that does the job, Learning to hack is still how you learn how things work: draw the line where the attacker is, The secret goes from the drop into the tool, never into the chat, Security experience is what lets you ship with known gaps: list them, match them to the reader, say what would change the call. Answers: Threat Modeling, 3rd-party-modules, Security creates better developers, Learn to Hack, Pointers, Strings should be banned, The future needs you.

17 Open source

In 2018 I asked a question I already knew the answer to: "Why don't you open source your code?" The answers, I wrote, would be some mix of "I don't think my code is good enough", "I'm embarrassed about my code" and "Nobody will want to use my code". My advice was blunt: "The key is to just do it!" Create a repository, add a licence file, and that is it. Then I told the truth about what happens next: "Nothing!" Most open source is maintained by a very small number of people.

Eight years on, everything in that chapter still holds, and the reason to do it has become clearer to me. It was never about contributions. It is that nobody can lock you in.

What open source is, and what it is not

Open source is a copyright licence. Copyright gives the person who creates a work the right to control copying; an open source licence is the holder using exactly that right to grant the opposite: you may copy, use, change and share, on these terms. The GNU General Public License, in 1989, made sharing a condition: build on it and distribute the result, and the result carries the same licence. Critics called that viral. The term "open source" came in 1998, after Netscape decided to release its browser's source, as a pragmatic reframing of the same freedoms for businesses. Linux, the most pragmatic project of all, stayed on the GPL. The repositories behind the newsroom are Apache 2.0 for code and Creative Commons for writing, and this book is CC BY 4.0, as the first edition was.

One thing I believed for a while, and was wrong about, because I hear it often from people your age: I thought the most open thing was not to licence at all. It is the opposite. With no licence, ordinary copyright applies and all rights are reserved. If you want people to use your work, you have to say so, and a licence is how you say it.

At an event in October 2026 I heard misconceptions that are now common among developers who use open source every day without having grown up in its culture, and *What open source is, and what it is not* corrects six of them. Anybody can change it, so nobody controls it: no, a few maintainers decide what goes in, and the control is real; it is just visible. Open source means anybody contributes: apart from a small minority of popular projects, almost every project is run by a handful of people, and that is its normal shape. It is more secure, or less secure, than proprietary software: neither; the metric is whether somebody had the time, and was paid, to review it. It costs nothing: somebody is always paying, and in one 2024 survey around 60% of maintainers described themselves as unpaid hobbyists. Open weights are open source: no, weights are a compiled binary. And the sixth is the one I just confessed to.

Through all of it, one sentence from 2018 is the one I would keep. When I see an application given away for free, as in zero cost rather than as in freedom, "I always think 'What is their agenda?'" Open source is a way of thinking about code in which there is no lock-in on the user by the creator.

The fork next door

Here is why that matters to a company, and to anyone deciding where to work. In a healthy startup everything is open source, so the technology is not the moat; customers stay for the service, the quality and the trust. Now ask what happens when one corner goes wrong. In a closed company, whoever holds control decides. In an open source company every corner has an exit, and the exit takes the technology with it. If the founders stop executing, investors can fund a team to run the same code next door. If a new owner changes direction or the licence, the founders can leave with the code, as long as no broad non-compete stops them. If anyone lets the customers down, the customers can move, as long as their data moves with them. *The fork next door* lays the triangle out.

Forks are rare and serious: in a study of 220 significant forks, only 7 merged back, and the famous ones that took hold all followed a breakdown, a new owner, a governance fight, a licence change. The reverse case is just as instructive. MariaDB was created in 2009 to protect MySQL from its new owners, and MySQL kept shipping, so the fork stayed a hedge. When I counted the commits to MySQL's main repository they had fallen from about 8,900 in 2019 to about 2,800 in 2025, which is the moment the argument predicts the fork should start to grow. That is a prediction, and it can be checked. So the real defence is not a licence or a patent. It is execution, and in the agent era, speed. The moat is a rate, not a wall: a competitor who forks today gets your position, not your velocity.

The security argument has moved too. Eric Raymond's line in *The Cathedral and the Bazaar* was "Given enough eyeballs, all bugs are shallow", and the honest answer was always that the eyeballs were not there: open source had the code in view and no budget for the slow reviews that find the hard bugs. Models can now run deep security reviews, over and over, on every change, at a cost that used to buy one consultant for a day. *The eyeballs get cheap* tells one project's year through its maintainer's own posts. curl's bug bounty had found 87 confirmed vulnerabilities over its life, and by 2025 "Not even one in twenty was real", so it ended. Then careful tools landed real bugs, "somewhere between two and three hundred bugfixes". Then a frontier scan found one vulnerability, "yes, as in singular one", of low severity. That is a project getting harder to break, wave by wave. The thing open source lacked was paid attention, and that is what just became abundant. It is why I think open source is about to become the most secure software we have.

Why it is easy to throw your own technology away

There is a quieter benefit, and it is the one I feel most in my own work. When everything I build is open source, I do not have a moat I have to keep using. There is no part of the stack I built that I need to protect from being replaced, and I am usually the first to try to replace it when something better appears. I wrote a library called Memory-FS; if the browser and a vault do its job for an application, the application stops using it. The habit is to ask of every component whether it is still earning its place, and open source removes the one bad reason to say yes: that I built it. *Building blocks* leans on that habit throughout.

The 2018 chapter made the personal version of the same point. Release your code and your future self can still reach it; move companies and you can still use what you built.

Your first pull request

Now the part for the person looking for a job. In 2018 I wrote that a great way to get a job in a company is to contribute to an open source project hosted by that company, and the OWASP chapter put it more plainly: most projects, even the big ones, are maintained by a couple of people, and "ALL projects are desperate for contributors and help!" The chapter on getting the job said "Act like you are part of the company", and *Act as the job you want* adds the 2026 version: if you are looking for a job, you have time, so do the work in public, where it can be linked, so that the people who want to help you can forward it.

Here is what a good first contribution looks like now. On 16 October 2026, late in the evening, Mondweep Chakravorty opened a pull request against an agent framework with 9,625 commits on its main branch. It added one page, 83 lines of documentation, describing his adapter: a behaviour policy compiled into the framework's own rules, gating memory checkpoints into an encrypted vault. The maintainer merged it the next afternoon. *Our first PR in Ruflo* puts the pieces side by side, and the lesson is in why it got in so quickly. No check ran and no formal review was recorded; what got it in was the page being specific about what it claims. It pinned the exact commits it depends on. It listed its evidence and its gaps in the same breath: "Semantic quality is NOT validated." It said what it was not: a documentation-only change. A page that does not oversell can be merged in under a day.

That is the bar, and it has risen since 2018. The small fixes a newcomer used to send can now be done by anyone with a model, and should be. curl's contribution rules now say that if you asked an AI tool to find problems, you must reveal that in your report. So a first pull request today carries its provenance: what you did, what you checked, what you did not. The 2018 outline *GitHub Flow* was five bullets I never wrote up, "Learn how to do Forks, Commits and Pull Requests" among them; the mechanics take an afternoon with an agent beside you, and *Your first workspace* covers them.

I should say where this came from, because ideas have a lineage. My own shift into an open source mindset happened, as the 2018 chapter records, "20 years ago after I read" *The Cathedral and the Bazaar*, and the lesson I quoted then still stands: "Release early. Release often. And listen to your customers." Linus Torvalds built two of the things all of this stands on, Linux and Git, and Git is the shape of an argument I keep making: a tool that decentralises trust changes how people work together. OWASP is where I learnt that knowledge compounds fastest in the open; the London chapter started over beers in a pub, and some of my best friends in the industry came from it. The 2018 chapter on Creative Commons quoted Tim O'Reilly, "For a typical author, obscurity is a far greater threat than piracy".

> **FOR DINIS**
> your own first pull request to someone else's project, and what it led to; the 2018 chapter says the usual outcome is nothing, and a reader will want the counter-example from your life.

The excuse I hear from careful graduates is the one I heard in 2018: the code is not good enough yet. Put the licence on it, send the small, honest pull request, and let the people who want to help you find something they can link.

From the register: Open source means nobody can lock you in: the fork next door, Your first pull request: open source is still the newcomer's best door, Open source, community and OWASP: working and getting hired in the open, with the names behind it, Open source is why it is easy to throw your own technology away. Answers: Open Source, Creative Commons, GitHub Flow.

18 Building blocks

The first edition had a draft on the cloud that never got past its notes. One asked what you were waiting for, if you did not yet have a cloud account. The draft warned that the cloud "is as complex as an OS". True, and the wrong thing to tell you. You do not need to learn the cloud. You need to know what the blocks are, which are free, and how to have an agent click them together once, so the result is something you own and do not think about again. I am technical, and I could learn any of these platforms. I do not want to. If that is your position too, this chapter is the map.

Cloud as Lego

In October 2026 I mapped the developer platform of the company I keep my domains with, Cloudflare, as the smallest blocks a non-specialist could put together. *Cloudflare as Lego blocks* has the list, with the limits as the pages gave them on 18 October 2026, and they change. To put something on the web: static files, served free with no request limit; small pieces of code at the edge, 100,000 requests a day free. To keep data: object storage, 10 GB free with no charge for reading it out; a small SQL database; a key-value store. To think: hosted models with a free daily allowance, running on the platform's own machines, with a privacy page that says it does not train on what you send. To talk and guard: inbound email to your domain, free; a sign-in gate in front of anything, free for up to 50 users.

The blocks are more useful as packages, each the smallest set for one job: a site on your domain; your vault, hosted; an assistant over your documents; an inbox that does things; a form or a small API. We tested the hardest. A read-only vault needs no code at all: a published folder of encrypted files that any static host can serve. A read-write vault needs one function of 83 lines, one storage bucket and one tiny coordinator per vault, and we ran it in the platform's own runtime against the real client: create, push, clone, and no plaintext anywhere on the server.

The catch is setup, and it is a catch in your favour. The platform's API cannot be called from a web page, so the one-time setup is a deploy button, a temporary account, or an agent with a terminal that signs in, runs the commands and hands back the URL. That is fine, because it happens once. *Your first workspace* is where you learn to ask for it.

The 2018 chapter on Docker is the one piece of that part I would keep as it stands. A container is "a process that exposes a multi-layered file system as a fully isolated OS", and the point of learning it was to move from building environments by clicking buttons to writing them down as code. That instinct is still right.

Free models, with your own key

The block you will use most is a model, and the question is who pays. We checked every free offer we could find, as of 18 October 2026, against two tests: can anyone start free without a card, and can a web page call it directly, so the key goes from your browser to the provider and nowhere else. Four passed both: Groq, Google's Gemini through AI Studio, Mistral's free mode, and OpenRouter's free models. *Free models, your own key* has the table, and two warnings.

Free moves: in 2026 four offers we would have recommended at the start of the year disappeared or gained a card, GitHub Models among them, retired on 30 July. And free means they may learn from it. On the free tiers, assume your words may be kept, learned from and, on one of them, read by people. That is fine for material that is already public. Never for anything private, and your team's spreadsheet is private.

One more thing I would not let an agent do: create the account for you. What an agent does well is guide: open the right page, explain each setting, point at the privacy switch, check the key works, and store it where you choose.

Git is a standard, and what tokens cost

The 2018 outline on Git was a page of bullets I never wrote up, and it had the right ones: use Git as your backup, version all your documents, "use Git everyday". The line I would add now is why Git won. Not because of GitHub. Because it is a standard: a way of laying out objects that any tool, any host and any person can read. Your history is not in anyone's product. sgit, which the newsroom runs on, is the same kind of thing with one change, where the trust sits: everything is encrypted before it leaves your machine, and any storage that answers a plain read can serve it. *sgit is a standard, not a server* makes the case; there is a server, "and it is the least interesting part". When you choose a tool to keep your work in, ask whether what it stores is a format other tools can read. If not, you are renting your own history.

Then the bill. Models are paid for by the token, and the person who sees least is the user. I asked for a hook that shows, under each answer, what the session has read, how much came from the cache, what it wrote and roughly what it cost; the newsroom's agent built it in 132 lines. A prompt cannot do this job: asked to report its own usage, the model said, correctly, that it could not see the numbers. The hook could. The session that wrote *The tokens hook* had read 99% of its input from the cache, which at list prices is the difference between about $170 and about $1,950. You do not need to understand caching. You need the box under the answer, and the habit of looking at it.

> **FOR DINIS**
> what a newcomer like Priya should expect to pay in a month, for one site, one vault and a model on a free tier with their own key; the register says costs for a newcomer is still a gap, and only you have the receipts.

The file system is the database

Here is a rule I have held for years: you do not need a database. The file system is the database. A database server is a large, stateful, always-on thing that you have to provision, secure, back up, migrate and pay for, whether or not anyone is using it, and most of the time what it holds would be quite happy as files. Load the state, work on it in memory, save it back. I call the pattern LETS, Load, Extract, Transform, Save, and *No server, by design* traces it through the projects it runs. It is a cache, a database and an audit trail in one, and Git gives it history for free.

If you have run a team on a shared spreadsheet, you already know this. The spreadsheet is a file. What you did not have was a way to query it like a database without buying one. You do now: the newsroom runs two real database engines, compiled to run inside the browser, over the JSON files its pages are built from. The database is the files; the engines are just readers. The 2018 outline on micro-services worried about moving from one blob of code into a constellation of services nobody fully understands. Files have the opposite property: every piece of state is a thing you can open.

No server, by design

The second step crept up on me over months, and the newsroom made it explicit: retire the back end too. The newsroom today lands a reader as a persona, charges by the share of a page they read, takes money and runs SQL over its own data. None of that logic runs on a server. What is left on the server side is commodity: static hosting, a vault server that stores only ciphertext, a payment provider's hosted page, DNS and a build pipeline. I am not saying there are no servers. What there is not is a server of mine with application logic in it. Each of those is a product, used as it comes, replaceable by another that does the same job.

I did not choose this architecture and then add encryption. Encryption came first, and the client-side design followed, because the vault server cannot read a byte, so anything that needs to read the reader's data has to happen where the key is: in the browser. And once the logic is in the browser, the question changes from "where on the server should this go?" to "why would this need a server at all?" That is a question you can ask of any tool your team is sold, and you do not need to write code to ask it.

The results are the ones you would want. It scales without its complexity scaling: a hundred thousand readers mean a hundred thousand browsers each doing their own work, against a server side that serves cached files; the server's work grows with bandwidth, and its complexity does not grow at all. It ships fast: 114 releases between 1 and 11 October 2026, and the reading meter went from an experiment to a working pay-on-demand design in seven releases in an afternoon. And it lines up with the things I care about: sovereignty, because the reader holds the data and the keys; privacy as a property, because the server cannot read what it does not have; explainability, because the whole application is files anyone can read; determinism, because the same files give the same result.

The 2018 outline on serverless said the word does not mean no servers, and that it was important to understand the limitations. The word has been stretched since: it now sits on products that bill you every hour whether anyone uses them or not, one of them at $0.75 a cluster-hour. So here is my test: if it bills you by the hour when nobody is using it, it is not serverless, and whoever runs it has something running on their side. I would rather replace the word with three that each carry part of it: ephemeral, because it comes and goes with use and costs nothing when gone; town planner, Simon Wardley's word for something industrialised and mature; and services, because it is exposed for others to use. Ephemeral town planner services.

There is one honest limit. Anything the reader must not hold cannot live in the reader's browser: a shared key, a price that unlocks something. And this is not a claim that servers are bad. A trading system, a multiplayer game and a bank's ledger need shared, live, authoritative state. This pattern is for applications where each

user's state is theirs. First the database went, and the work got simpler. Now the back end is going, and the work is getting faster. I do not know yet where the pattern stops, which is why it is worth pushing.

From the register: Building blocks: cloud as Lego, free models with your own key, git as a standard, The file system is the database: load, extract, transform, save, and keep every intermediate as a file, No server, by design: retire the back end too, and keep only commodity servers. Answers: AWS, Serverless, Micro-Services, Git, Docker.

19 Shipped means the user has it

In August 2026 two consecutive releases of sgit.ai pushed cleanly, verified themselves against both remotes, printed "both remotes in sync, done", and never reached anybody. The site served a two-release-old page for forty minutes. A person on a phone looked at the version badge and asked: "is this the latest version?"

Everything about that sentence is worth sitting with. The checks were not weak. They were checking the wrong boundary.

Green does not mean live

The release script did five things, and four of them were good. Build the site from source. Run the validator. Push the encrypted vault and confirm it is in sync. Push the git mirror and confirm the local head matches the remote. That last check is strong; it compares hashes, not hopes. The problem is what it means. It means the bytes arrived at the code host. It does not mean anybody can read them.

Publishing the site has three boundaries: source to remote, remote to deployment, deployment to reader. We were verifying the first two. Between the second and third sits a hosted build job with its own failure modes, and nothing in our release looked at it. What failed was not our code and not our content. The deploy job died before running a single step of its own, because the download of a third-party action it depends on was rate-limited: "429 (Too Many Requests)", three attempts, give up. The one job that puts bytes in front of readers could not fetch its own dependency. It is the ordinary case: the most likely thing to break in a deploy is not your code. It is a piece of shared infrastructure you do not run, on a day you were not thinking about it.

The 2018 outline on CI testing had the line, borrowed from a talk on chaos engineering: "config changes can be more dangerous than code changes", because we do not test them the way we test code. The outline on test-driven development had a story next to it that I never wrote up, about a site that went down for hours because of a one-character change to a web server's configuration file.

> **FOR DINIS**
> the story of the site that went down for hours over one character, a pipe, in an nginx config; the 2018 TDD outline promises it and this is where it belongs.

The fix is one question, asked at the end. A release now finishes by asking the live site what version it is serving, polling with a cache-buster for up to eight minutes, because the origin's own answer is the only one that counts. If the version never appears it aborts loudly and names the build page. The cost is real: up to eight minutes on every release. We took it, because the alternative had already been demonstrated twice in one afternoon: telling somebody a fix was live when it was not, twice, in writing.

The site already had two rules of exactly this shape, and the third made the pattern obvious. A page nothing links to is unpublished; the build fails on an orphan, which happened once. A page the machine index omits is unpublished; the build fails if a page would be missing from the index for agents, and that happened too, a whole section silently absent. A page the deploy never served is unpublished; now the release fails on it. All

three are the same claim, and it is the title of this chapter: *shipped* is a fact about the reader, not about your repository. Every one was learned by getting it wrong first, which is the only reason the list is short and specific rather than long and aspirational. *Green does not mean live* also says what it does not fix: a partial deploy would pass.

Tests that never run are not tests

The 2018 outline on testing had two lines I still say. If you make a change and no test breaks, you are making random changes, and you have no way of knowing whether they are still in effect tomorrow. And: if you do not have full coverage, what are the bits of code not covered by tests, and what happens when they change? The more coverage you have, the more changes you are happy to make, and hundreds of small changes are the only way I know to build something good that lasts. The outline also wanted tests that run as you lift your fingers or press save; now that an agent writes the harness, there is no excuse for not having that.

In October 2026 I measured the newsroom against that standard, and it is a fair picture of explorer code that has just started taking money. The live reader code was about 2,700 lines. The tests that covered it were 433 lines, and the one workflow the repository had never ran them. The same application had two upstreams, with two diverging copies of the meter. No page set a content security policy. It is also exactly what has to change before the next wave, because a reader can now pay £5 by card. A mistake in the meter is a mistake with someone's money.

The line that moved is the one *Explorer coding, and the handover* draws. Explorer code is read for intent, mostly by agents. Villager code is read line by line, because that is where changes land and where the blast radius lives. The newsroom's reader features had crossed from the first to the second, and the right move at that moment is to go slow to go fast. *Go slow to go fast* is the brief, in six phases. Map everything: every file, capability and control. Move the reader code to its own repository as a pure refactoring: the same behaviour and the same bugs, proved not by a reviewed diff but by hashing every file the live site serves before and after. Test: only test code is written, in real browsers, against a real pre-production copy, with no mocks, and with every known bug and vulnerability written as a test that passes, recording the current behaviour, so the fix later is a visible change to a test and not a silent one. Refactor, with every test passing after every commit, and an adversarial exercise: try to change what a reader sees or pays without a test failing, and treat every change that slips through as a missing test. Only then does the explorer return.

The pipeline is the other half. The reader application is plain JavaScript and static files, so the target is under two minutes on every commit: lint, unit tests, three browser engines, and a build that produces the same bytes twice. On every merge, a pre-production copy with a payment link in test mode that can never charge anyone. On every release, the question from the first section, asked of the live site. And a pull request needs one person and one agent reviewer before it merges. Going slow now is what lets me keep going fast without the fear that each new feature breaks something a reader has paid for. The 2018 outline on DevOps had a joke I still like: "Congratulations, you now own/manage a Data Center". The 2026 version is gentler and the same: congratulations, you have a user; now everything you ship lands on them.

Five people who do not exist

A green pipeline tells you the code does what the tests say. It does not tell you what a person sees. For that, the cheapest method I have found is synthetic users: five to ten invented people, played by a model, each driving a real browser through the site one screenshot at a time, saying what they see, think, ask and do, and then being interviewed about it. *How to run synthetic users on your own site* is the how-to, from three studies, and it takes an afternoon.

A synthetic user is a record, not a character: a situation, a question they arrive with, what they know and do not know, what makes them leave, their patience and their screen size. The negative fields make a run honest: what they do not know, and what makes them leave, which is often the most useful ending. Two habits have paid off every time: include somebody who will never buy, because the sceptic reads the site adversarially and finds the claims that do not hold; and put at least one person on a phone. Write the expected journey before any run, so the difference between what you expected and what happened is a finding you can point at. Then the one rule of the run itself: give the agent the screenshot, the same thing a person would have, and not the page's code. An agent that reads the code finds every link every time, and so finds no confusion, which is the only thing worth running this for. Every person is invented; do not model real people, and say so where nobody can scroll past it.

On the newsroom, five invented readers took 62 steps, asked 51 questions the site did not answer, and produced 31 findings, six of them blocking. Four of the five left. Their ratings were 2, 2, 3, 3 and 2. The blocking findings were all about the step after reading: no contact for a pilot, nothing showing real money moving, a top-up page that says it is simulated while asking for money. Then came the step that separates a study you can stand behind from a pile of plausible paragraphs: check every surprising finding against the browser's own record. Two did not survive; they were caused by our own helper, and they are kept in the study, labelled as artefacts, because a findings list that silently loses items cannot be compared with the next one. One finding got bigger. A reader hit "screenshot unavailable" where the evidence should have been, and when we checked the files, 51 of the 527 figure references in the newsroom's articles pointed at images the move from sgit.ai had not copied. Nothing failed in a way a server would have noticed: the pages were files, and they served.

That pattern has held on every site we have run this on. The day after the first study, the same method pointed at another of our sites caught two JavaScript errors that had shipped and that every existing test had passed over, because the page still rendered. The findings that pay for the whole exercise tend to be the ones the method finds because it drives a real browser, not because the persona is clever.

It is not user research, and the study says so. The plan is simple, and it is the habit I would look for in anyone I was hiring to ship: every site gets five users before it is called done, and the same five again after the fixes, because a finding that disappears is a fix that worked, and a new one is something the fix broke. Then ask the live site what version it is serving, and only then say it shipped.

From the register: Shipped means the user has it: green does not mean live, go slow to go fast. Answers: CI Testing, Continuous Integration, TDD (Test-Driven Development), DevOps.

PART 7

The long game

What lasts. Graphs as memory for the person you will be, the founder's move when a plan is already published, and the civic responsibility that comes with knowing how the machines work.

20 Graphs, memory and your future self

In 2018 I wrote a chapter called *Backup your life*. It asked one question: if you lost your laptop right now, how painful would it be? My own answer, at the moment of writing, was that I would lose the text of that chapter and nothing else, and in half an hour not even that, because it would be in git and on GitHub. The chapter ended with a line I still stand by: "You basically need to backup your life!"

Eight years later the question is the same and the answer has grown. It is no longer only your files. It is your ideas, the connections between them, and now the memory your agents work from. This chapter is about all three, and about the person you are keeping them for: your future self.

I think in graphs

My thinking is very hyperlinked. I connect things; I follow this and then that, and it all works itself out in the end. *Everything is connected: graphs* introduced the graph as a thing to draw. Here I want to say it as a way of thinking, because it is how I keep what I know.

The first edition had a chapter on DOT, a text language for diagrams. The argument was not the tool. It was that a diagram stored as text can go in git, be committed on every change and be diffed, so you see what changed since you last looked. I wrote then that our brains struggle to review a diagram a second time unless we are shown "what changed since the last review", and that we were making decisions "based on shadows of reality".

Think of what your team already has: a spreadsheet of who owns which system, a folder of policies nobody opens. Each is already a graph. A spreadsheet is a graph of sheets, rows, cells and formulas; a team is a graph of people and what they are responsible for. Nothing has to be converted. The edges only need names: a policy is owned by a team and implemented by a control; a decision supersedes an older one.

Two rules matter more than the rest. Every edge is a verb that reads both ways, and "relates to" is banned, because two things always relate and an edge with no verb tells you nothing. And supersede, never delete: an idea that turned out wrong stays, marked, with what replaced it, because the path from wrong to right is itself knowledge. The 2018 book had an outline called *Recursive Functions* that I never wrote; in a way it is this. Open any node and you find the same shape again, a graph inside the graph, as far down as your question needs.

Then the part that matters most if you have never written code. You do not have to get the graph right. You have to maintain it. The projects that try to get the ontology right first spend a year on it and never ship. Agents are very good at maintaining graphs: they read a new document and propose its nodes and edges, they notice a policy changed and mark what rested on it, and code checks that they kept the rules. When something looks wrong, you ask why, and the answer is a fix at the right altitude. I measured this on the newsroom: the graphs of its 112 articles hold about an eighth of the articles' text, and that eighth is enough to decide what to read.

A graph is also not a search box. Similarity search finds a starting point, but it gives you similarity, not a reason. A graph gives the same answer to the same question, with the path that led to it, and for "why are we required to do this, and by whom?", the path is the answer.

Memory is context management

People ask how my agents remember. The picture most people have is one memory system behind the agent, into which everything is deposited and from which it retrieves by similarity whatever seems relevant. Mine is the opposite, and the newsroom article *Memory is not a spectator sport* says why.

An agent works inside a window of text. Everything in that window costs money, because it is billed by the token, and costs attention, because recall gets worse as the window fills. So the question is never "what do we remember". It is "what does this agent need in its window right now", and no more. Memory is context management.

It is many memories, not one. The rules of my inbox agent are not the rules of the news graph, and the vocabulary of a code review is not the vocabulary of a risk register. Each is a memory with its own words, and by now, across twenty-seven sibling sites and forty published vaults, they disagree in places. I used to think that was a problem to solve. I now think it is the point: each is correct for its context, and a memory forced to agree with all the others would be wrong for every one of them.

It is fractal: principles at the top in a few kilobytes, code at the bottom, and an agent loads the altitude its question lives at. The fractal is the budget.

And it is files. In August 2025 Letta ran a memory benchmark, and a plain filesystem, with the agent reading and writing files, scored 74.0 percent against 68.5 percent for a graph-based memory product. Files won. A folder of well-named plain files, kept in git, is already a memory an agent can use. You do not need a product for this.

It is shared. A session ends, and the next one, or a different agent, or a person, picks up from the same files. The article that argued this was written that way: one session, across several context resets, revised an article, wrote two more, published a vault and shipped six releases in a day, "remembering nothing between resets except what the files remembered for it". Nothing is remembered by the model between sessions. Everything is remembered by the files.

I should say where this falls short, because the article does. The thesis calls itself untested and falsifiable. Stale pages become false memories: the session that wrote it found four and fixed two. It is one person's estate, not yet tested against a second person's. Hold it as my position.

Publish, and the site becomes the memory

In 2018, in *Backup your life*, I said the fewer secrets you have, the easier everything else is to back up, and that the more you publish, the more you can use the web itself as your backup. And I said why I share at all: one day my future self will need it. Open sourcing my work meant that when I moved jobs I did not start from scratch; the O2 Platform research came with me because it was already public.

The same reasoning now has a second beneficiary. Open source and Creative Commons are not a licensing preference here. They are what makes a memory usable by an agent: a public page can be fetched, a public quote cited with its URL, a public graph loaded by anyone's model, a public vault cloned with a read key that is itself public. The more I publish, the more there is for the next session to stand on, and the less it has to be told. One article said it in passing: the site is becoming the memory.

The 2018 book had an outline called *Future Self* that I never wrote. Its notes said: create assets that will be useful for them; the power of open sourcing your ideas; letters to a younger self are not that useful, unless you have a time machine; and that is why you want to share. I am writing it now. A letter to your younger self reaches nobody. A letter to your future self reaches them, but only if they can find it.

I know this because I failed at it. Most of what I have written is not linkable from what I write now. For years I used Google as the search engine for my own work: I would search for a phrase I remembered and find my old post. That is not a memory, it is luck. The inventory, taken in October 2026 from public pages, is 103 essays, 1,289 blog posts, 65 decks, 10 books, 29 tools, 2,038 files of NotebookLM material and 221 videos. A back catalogue I cannot link to is a conversation with no reply. So the newsroom has a brief for a worker: an agent that runs every day, takes one item from that catalogue, and writes it back as an article saying what it was, what it said, what changed since, and how it connects. Each article is a node the next one can link to. It compounds, and every team has a back catalogue it could do this with, a few items a day, while the team keeps working.

> **FOR DINIS**
> the 2018 *Future Self* outline promised a Twitter case study from the writing of that book, with two screenshots and no text. What did they show? The story would make this section concrete.

One more thing a shared memory needs, and a vector store cannot give it: a source for every item. On my sites that is a set of habits. Every quote has a URL. Every release has a commit. Every vault has a read key on its page. Every file inside a vault has a hash. Every page has a byline naming the person responsible, the model that drafted it and the date. Provenance is what lets memory be shared without being trusted.

So the advice is the 2018 advice, extended. Keep your work in plain text, in git, committed often. Name the links between things, in words a person in your team would say. Publish what you can, under a licence that lets it be quoted. Keep what you cannot publish in a vault, with a key that reads and does not write. When you brief an agent, hand it the altitude its question lives at, not everything you have. *Your first graph, built by hand* and *Your own newsroom and inbox* show both ends of this done small.

The lesson of *Backup your life* was that you will lose data, so build the habits before it happens. The lesson now is that you will forget, your agents will forget, and the people who come after you will never have known. The only memory that survives all three is the one written down, linked and left where the next reader will find it. Write for that reader. One day it will be you.

From the register: Think in graphs: knowledge that maintains itself, and your future self, Memory is not a spectator sport: it is context management, many memories, fractal, published and shared, Publish, and your published work becomes your agents' memory, with a URL for every claim. Answers: Future Self, Backup your life, Dot Language, Recursive Functions.

21 Be a founder

In 2018 I wrote a short chapter called *Be a founder*. It said the single thing you personally control when you go to work is your attitude to it, and it asked you to imagine you were employee number four and cared deeply about the company. Then five questions. If I were a founder of the company, department or section I work in now, with the responsibilities I have at the moment: how would I behave every day? What needs to be done now that will make a big difference? What can I do that will help? What would I do differently? What values and principles would I fight for? And under them all: what is stopping you from doing just that? Who is telling you not to? "At the moment it is just you!"

I still mean every word. The more you care and behave like a founder, the more value you add and the more valuable you become. But in 2018 I meant it as a way of behaving inside somebody else's company. In 2026 I also mean it literally, because the thing that stopped most people from founding anything has just become cheap.

The playbook that worked

Look at the technology companies that did well in the last ten or twenty years, and a pattern repeats. Someone very clever and very ambitious finds a gap in a market. Often they did the famous hundred calls with stakeholders and asked what hurts, which already tells you something: they were people who could get those calls. Then they raised money on belief, hired the people they did not have, including the subject-matter experts, and built a company on top.

I watched this many times in cybersecurity, and it worked. The startups displaced incumbents that had a much better starting position: the customers, the domain knowledge, sometimes the money. What the incumbents lacked was the structure, the workflows and, to be fair, the technology culture.

This is not a criticism. Those companies changed industries. But the playbook has a second act: the disruptors became the incumbents, and much of what they built is now the software everyone complains about.

The experts were always the better bet

The myth of the young outsider was never the whole story. The largest study I know, of 2.7 million US founders, found that the mean age at founding of the fastest-growing one-in-a-thousand ventures is 45, and that founders with three or more years in the same industry reached top-tier growth at twice the rate of those with none.

What held the experts back was not knowledge. It was access: to capital, to engineers, to the long, expensive build before anything could be shown. That is exactly the part that got cheap. With agents, much of the execution that used to need a funded team can be done by one person who knows exactly what to build. The share of new US startups with a solo founder rose from 23.7 percent in 2019 to 36.3 percent in the first half of 2025. The millions that used to be the price of starting are no longer the price of starting.

Three examples from the newsroom article *The expert founders*; two are people I help and the third is me. I come from application security and from working as a CISO; sgit.ai and RiskMandate are that knowledge turned into a company, where the agents write most of the code and the briefs, the buyers and the risk model are mine. A consultant knee surgeon in London is building a platform for knee health; the patients and clinicians who need it were his long before there was a product. A transcriber is building legal transcription software, "Built by a Transcriber, for Transcription Professionals". None of them is a hundred-call outsider. Each comes from the field and is focused on one thing. That focus used to be a weakness in front of an investor who wanted a huge market. It is now the advantage, because the build is no longer what stops them.

If you have just finished a degree, you may read this as being about older people with a field. Read it the other way round. The expert can build first now; the question is what you are the expert in, or will be.

And it favours where you are. The data says European companies raise at higher revenue, lower valuations and smaller targets than American ones. I think that scepticism is what this generation needs, because money too early is one of the most reliable ways to kill a startup: a study of about 3,200 high-growth internet startups found 74 percent failed through premature scaling. An expert who builds the first version with agents, shows it working, and then raises for scale rather than discovery is exactly the founder sceptical money is good for. The gap is real at the other end: the Draghi report puts the EU's share of global venture funds raised at 5 percent against 52 percent in the US. Europe is weak at scaling. My argument is that the first part, the part that now matters most, is where it is strong.

Do they miss it?

Most advice about starting a company is about raising money. This is not that. It is the operating model I use, in three pillars, in an order that matters: be profitable; investors should be talking to you, not the other way round; open source everything.

The first pillar starts with the word ship, and the word has been worn smooth. Shipping does not mean a proof of concept, a demo, or a video of a demo. It means a thing somebody else can go and use, on their own, and get value from. Nobody using it cares how it was built, and neither should you, yet.

Then hand it to people for free, for a short window. The feedback starts the moment somebody who is not you has the thing. Then comes the part almost nobody does deliberately: take it away. The trial expired. And ask the only question that matters. Do they miss it? If they shrug, you do not have a product yet; you have something people accept when it is free, which is a much weaker signal than it feels like from the inside. If they come back and ask for it, you have something.

Then two questions, and people answer only the first. Is the price one they are happy to pay? And is it profitable for you? If you cannot get revenue at all, that is the problem to solve, and it is not a marketing problem. One trap: do not reach for a subscription by default. Almost every time we buy something we are buying future value, and the research says we are bad at it: members of three gyms on a flat monthly fee of over 70 dollars attended on average 4.3 times a month, paying more than 17 dollars a visit when a ten-visit

pass would have cost 10. Charging rent for something people are not using is a worse business than being paid when you deliver. The newsroom runs the opposite experiment, pay after you read, so that by the time you are asked to pay the value is in front of you; the results are not in.

The second pillar follows. The worst time to raise is before you are profitable, before you have the product, before you understand the fit, which is exactly when most companies raise. You are bargaining from weakness and both sides know it, and the worst terms are not about money but control. The analogy I keep coming back to is music. Do not get signed before you have the album. Write the songs, record them in a bedroom studio, play locally, get people buying. Then take the deal, as somebody who already has an audience.

The third pillar surprises people. Open source everything. The technology is not the moat, and believing it is will shape every decision badly; treating it as open forces you to find what is actually defensible, usually the data, the relationships, the distribution or the speed at which you ship. And it is a better exit for you: build proprietary and you accumulate a pile of technology you cannot take with you; build in the open and the work is still yours when you move on. You leave with your tools. *Open source* has the longer argument.

There is a company in this

I want to end with a company, said plainly. It sits with the other business plans published for founders on sgit.ai, and I would be glad to see it built with us or without us.

Source code is a fractal semantic graph: layers within layers, each with its own vocabulary, from what the user is trying to do, through the features, the components, the classes and the methods, down to the syntax tree. A code review today reads a text diff, which is at one altitude, the lines, and the wrong altitude for almost every question a reviewer has. If every layer is kept as files, every layer can be diffed: a refactor should move the bottom layers and leave the stories still; a bug fix should be visible at the top as a story that holds again; the blast radius of a change is the climb from the methods it touched to the stories that can reach them. The worked example is published as a vault: the sgit command-line tool, 377 classes and 1,111 methods, read as layered graphs with nothing run.

What the company sells is a code review that reads every layer. Attach it to a repository and it derives the graph once, keeps it current per commit, and on every change shows the reviewer the diff at each altitude and the climb to the stories it can reach. It keeps the graph as files in the customer's own repository or vault, so the customer owns the record, and it gets better the longer it runs there, because every correction is a commit. Who buys it is anyone generating code faster than they can review it, which this year is everyone.

What is defensible is the record and the loop, not the model. Models will be commodities; the graph of a particular codebase, corrected over a year by the people who know it, is not. The scripts are Apache-2.0, and a founder does not need my permission. They need a second language and a first customer.

FOR DINIS

the article discloses that two of the three expert founders are people you help, and that sgit.ai and RiskMandate are yours. Does the book carry that disclosure in the chapter text, as here, or only in the provenance the build renders?

If you are the person with the degree and the first language, that last sentence is for you. The test of the idea is whether the layers hold their shape in a second language. In 2018 I asked what was stopping you, and answered that it was just you. The answer has not changed. What has changed is how little else is in the way.

From the register: Be a founder: the expert can build first now, and the test is whether they miss it, There is a company in this: a code review that reads every layer, for somebody to build. Answers: Be a founder.

22 Civic responsibility

The 2018 book had an outline called *Civic Responsibility* that I never wrote up. Its notes were six lines. As a developer you have a major civic responsibility. Code is law. You are one of the most important pieces of the puzzle. You can choose where you work and whether your code has a positive or negative impact in the world, and if you cannot choose that at the moment, make sure you are actively looking to change your job. Give the risks you create to the business owners, and make them accountable for the decisions they are making.

I am writing that chapter now, with my own country, because that is where I have made this argument for sixteen years, in public, with slides, and the record of what came true and what did not is a better teacher than a list of principles.

Lisbon, 2010 and 2016

On 17 December 2010 I gave the keynote at OWASP's IBWAS conference in Lisbon: 168 slides, five proposed focuses and twelve proposed actions for government and industry. Allow ethical hacking, publish security reviews as score cards, introduce liability for software, and "'Proper' SSL support for 50% of websites", among others. It ended by arguing that Portugal, which I described as depressed and with "nothing to lose", could become the Switzerland of hacking. What I asked for "doesn't cost any money"; "It only takes courage, focus and commitment".

Sixteen years later, more of it came true than I expected, though mostly from Brussels rather than Lisbon. From 9 December 2026 the EU's Product Liability Directive brings software under strict, no-fault liability; my slide asked only for government and listed companies, and the law went further. Europe has its own vulnerability database, run by ENISA since May 2025, and since 11 September 2026 the Cyber Resilience Act requires manufacturers to report actively exploited vulnerabilities within 24 hours. HTTPS is the norm, far past my 50 percent.

What I got wrong was the mechanism. I thought fear, a big enough disaster, would move governments quickly. What moved them was a decade of slow, unglamorous regulation: liability and reporting duties, not a single catastrophe. And the part about Portugal itself, an open, ethical security industry as a way out of a depressed economy, is still mostly to do.

On 11 November 2016 I gave the keynote at BSidesLisbon, *Hacking Portugal*, 103 slides, and three weeks later published it as a book on Leanpub, 75 pages, whose page still says 95 percent complete. Its first lines are the premise of this chapter: "If the word 'hack' means to solve problems, then 'Hacking Portugal' means to find innovative ways to solve existing problems." The ideas were blunt. Make the Portuguese network hostile to insecure code. Replace the old compulsory military service with a Portuguese Hacking Service for 15 to 21 year olds. A hackathon league as famous as football, with the kids, the clubs and the pride. *OpenSource.pt*: all code paid for by government released under an open-source licence. A Ministry of Code.

Ten years on, the regulation arrived from Europe, close enough to the *Clear Software Act* on slide 83. The Hacking Service, the league and the Ministry of Code have not happened, as far as I know. The best news is in the spirit of OpenSource.pt: in 2026 Portugal released Amália, a national language model for European Portuguese, as open source, built by a consortium of universities with 5.5 million euros of EU recovery money, published with its dataset and code.

And the biggest change is one I did not see in 2016. Then, learning to hack meant learning to code first. Now a sixteen-year-old can explore a system by talking to it, which *Explorer coding, and the handover* called explorer coding. A Hacking Service in 2026 would teach young people to direct agents against their own country's systems, under a mandate that says what they may and may not touch. The talk's central claim, that a small country can choose to lead in code, is more possible than it was, because everyone who can brief an agent can now build.

Judo for small countries

The conversation about a national AI strategy keeps circling the biggest model, the most GPUs, the largest data centre. On a Wardley map those sit at the far right, in commodity, with a lead held by companies that have invested amounts a small country's whole technology budget would not match. It is the wrong end of the map for anyone without the capital.

Look at the other end instead. Put a person at the top: a developer in Cluj, a designer in Porto, a product team in Tallinn, selling a service to a buyer abroad. What stops them is not talent, energy or price. It is forming a company, hiring across a border, being paid across Europe, working in a language the buyer reads, a contract, a tax form valid on both sides. None of that is the service. All of it is plumbing, and in most of Europe each piece is custom built by the person who needs it, which means built badly, late, and at a cost that falls on the wrong side. In the last Eurostat figures with a breakdown, for 2021, fewer than one in ten firms in the single market sells across its own borders online. Commerce does not happen, most of the time, because of attrition.

We understand that a government builds roads, because they enable everybody. The shop that opens on the new road did not pay for the road; it paid taxes. Yet we ask every person who wants to sell a service out of a small country to build their own road first. In the age of agents, that plumbing is infrastructure in the road sense, and a state that builds it enables all of its people at once. Estonia did it: a person anywhere can become an e-resident and form an Estonian company online, the programme's dashboard lists more than 142,000 e-residents and more than 43,000 companies, and underneath sits X-Road, open source under the MIT licence, so another country can take the road rather than draw it again.

The plumbing has to be open source and run on open schemas, because that is what turns infrastructure into sovereignty. A road with a toll gate owned by one company is lock-in with tarmac. What the state pays for should belong to the commons, not so everyone changes the production system, but so everyone can see it, run it and leave with their data.

Then put the money where the market is still at genesis. Most work does not need a frontier model and should not use one. That means small models, and many of them: not one Portuguese but the Portuguese of Portugal and of Brazil, of Angola, of Cape Verde, of Mozambique, each with its culture in the training data

and not just its spelling. A public model is a public road. And treat constraints as the input, not the excuse: DeepSeek's own report says its V3 model was trained in 2.788 million GPU hours on export-restricted chips whose interconnect was deliberately throttled, with custom code written around the constraint.

Sovereignty is the ability to move

I am learning to play padel, and the striking thing about a good player is how much time they have. They read the ball, they know where it will be after it comes off the glass, and they are already there. If I move after it has bounced off the back wall, there is no hope. The ball today is the frontier model and the data centre; where the ball is going is the ability to move: workloads, data and the actions agents take, from one provider to another, when you decide and not on the day a supplier decides for you.

That is my definition. Technology sovereignty is the ability to move, almost liquidity. The honest test of most organisations is that they cannot do it, not between clouds and often not even within one, because the infrastructure was hard-coded, created by clicks, and never deployed in a second place. That is an engineering gap, not a policy gap.

The test is a Wardley map with one question. Put the customer at the top, the service, the application, the data, and under them the products and commodities you buy. For each dot you do not control: take it away, what happens? Follow the edges up. If the service stops, you have found the problem, and it belongs on the risk register with a name next to it, all the way to the board. That is what the 2018 outline meant by giving the risks you create to the business owners. Draw the dependency, show what happens when it goes, and make the person who owns the decision own it.

Most sovereignty efforts never start because they set the bar at an alternative as good as today. Nobody gets from zero to a hundred. The bar should be: it works, for the minimum viable company, the priority customers and the services that must never stop, slower and dearer if need be. Run it in QA, all the time, on twins of the dependencies you do not control, so that every small problem is found on a Tuesday rather than during an outage. Ask whether it can move before asking whether it is cheap.

And it has to be open source, because even a European supplier is one acquisition away from not being one, in either direction. Control that survives a change of owner is the only control that counts.

What this asks of you

Code is law, in the plain sense that what you build decides what people can and cannot do, and in 2026 what your agents can and cannot do. You can choose where you work and whether your code has a positive or negative impact, and if you cannot choose that today, be actively looking. Give the risks you create to the people who own the decisions, drawn on a map they can read.

If you are in a small country, you are not behind. You are at the end of the map where the market still exists. The move is judo: use the weight of the commodity models, clouds and rails rather than trying to match it, build the plumbing once and in the open, buy the alternative before the day you need it, and regulate what settled rather than what is still moving.

FOR DINIS

the register notes that a Portuguese edition, or both languages, is your open question, and this chapter is the one it bears on most. Which is it, and does the Hacking Portugal material stay here or move to the new edition of that book?

In 2010 I said what I asked for would cost no money, only courage, focus and commitment. In 2016 I said to hack means to solve problems. Both still hold. What has changed is who can do the hacking: in 2016 the people who had learned to code, now anyone who can brief an agent, which makes the civic responsibility wider, not narrower. You are one of the most important pieces of the puzzle. Act like it.

From the register: Civic responsibility: to hack means to solve problems, and small countries can lead now. Answers: Civic Responsibility.

PART 8

Hands on, 2026

The practical part, dated 2026 and meant to be renewed every year. Seven workflows you can follow this afternoon: an agentic workflow from zero, your first graph, one idea through five lenses, a tiny agent loop, pictures drawn by code, meeting briefs and follow-ups, and your own publishing. Whether this part lives inside the book or as a companion is still my decision to make.

23 An agentic workflow from zero

By the end of this chapter you will have a folder for every person you deal with, an agent that reads those folders and drafts for you, and six lines of rules that make it safe. One afternoon, no developer, no vault, no new tool: a Google Workspace account and a Claude seat are enough. The steps are the newsroom's, from *From a spreadsheet to a folder per person*. The tools move, and the article dates its description of the Claude app to October 2026; the shape is what to learn.

What a spreadsheet cannot hold

If you are in touch with people, you probably have a spreadsheet: one row per person, a stage, a date, a next step, and a notes column. The person the article was written with ran their customer relationships that way, and ran them well. What the row cannot hold is what happened: the emails, word for word, the call last Tuesday, what was promised. "The difference is not the tool. It is what counts as the record." In a spreadsheet the record is a summary you typed. In a folder the record is everything that happened, kept as it happened, and the summary is derived from it. That reversal gives an agent something to work with.

The folder, and the policy

Make a folder called Contacts. Inside it, one folder per person. Inside each, five documents, with these names so that you and the agent mean the same thing:

- **Record**: role, company, how you met, stage, the next step and its date, what they care about, every fact with where it came from in brackets. The row, promoted.
- **Ledger**: one line per event: the date, in or out, the channel, one sentence. Only ever added to. A correction is a new line.
- **Messages**: every email and note exchanged, pasted word for word, newest at the bottom.
- **Notes**: calls and meetings, dated, including what you said into your phone afterwards.
- **Summary**: written by the agent from the four above, rewritten when you ask, never edited by hand. If it is wrong, fix the Record or the Ledger and regenerate.

In the Claude app, connect Drive and Gmail, make one Project called Contacts, and put the behaviour policy in its instructions. Six lines, from the article, which I will not shorten:

1. You work for me. Every email you write is a draft that I read and send. You never send.
2. You read the Contacts folder in my Drive and my Gmail. You write only into a person's folder in Contacts, and only into their Ledger, Messages, Notes and Summary. You never change the Record without showing me the change first.
3. You add to the Ledger and the Messages. You never rewrite or delete a line. A correction is a new line.
4. Every fact you put in a Record or a Summary names where it came from.
5. If a person has no folder, you ask before making one. If a request would reach outside the folder, you ask.
6. At the end of every task you say, in one line, what you read and what you wrote.

Draft and never send; append and never rewrite; stay in the folder. The first three are the ones the newsroom learned the hard way. Most of this is told, not enforced; the rest is a rule the agent keeps, which is why line 6 exists. The one-line report is how you check.

What you have built is, in the terms of *Lenses, roles and loops*, an agent: a role on a loop with a goal, and the Project's instructions are its role description. *How I work with Claude* says it more plainly: a session with a focus and a `role.md`, one per topic, named.

The afternoon, and the first week

The migration is four steps. Export the sheet as CSV, drop it in the folder, and ask the agent to say, for each row, the folder it would make and the Record it would write, without making anything yet. Make the folders from a template; twenty people is twenty minutes by hand. Per person, ask it to read a year of your mail with that person and write the Record and the first Ledger lines, each fact with its source; read the Record and correct it once. Then ask for the Summaries, read three, and where one is wrong fix the source document, not the summary. From then on nothing is retyped.

Then five exercises, one a day for the first week, each a single message:

1. **Ask.** Who have I not spoken to in more than a month, grouped by stage, with the last Ledger line for each?
2. **Prepare.** I am meeting someone on Thursday: brief me from the folder, what we last agreed, the last three emails, what we promised.
3. **Draft.** Draft my follow-up, in my voice, from Notes. Do not send. Add a Ledger line that says a draft exists. You change two words, send it, and tell the agent "sent".
4. **Debrief.** Speak for two minutes into your phone after the meeting; the agent puts the notes in Notes, updates the next step, adds the Ledger line and regenerates the Summary. This one changes the habit.
5. **Campaign.** Find everyone whose Record says they care about X, show the list with the reason for each, then draft one note to each, and send none.

It will get two things wrong, predictably: a fact in the Record without a source, or a stage inferred from tone, fixed by a stricter line 4; and a Summary rewritten so that it drops something you cared about, fixed by putting the thing in the Record and regenerating. Every mistake adds a line to the policy. That is how the newsroom's grew, and it is what *Follow the rabbit holes* means by a system that once cost "hundreds of thousands or millions of pounds" being now "a brief and a feedback loop".

> **FOR DINIS**
> the walkthrough is written for someone with customers, and asks for three real contacts and the topic the person sells to be substituted. For a reader looking for a first job, should the exercises be rewritten around their own contacts?

The folders move into a vault when one of three signals appears: a second agent needs to read them, the files should not be readable by whoever runs the drive, or the record has to carry between tools. Until then the folders are enough, and when the day comes the move is a copy; the policy does not change.

From the register: the articles of the hands-on part. Answers: no 2018 file.

24 Your first graph, built by hand

By the end of this chapter you will have drawn a graph of something you know: a text file of forty-odd lines, an answer to a question somebody keeps getting wrong, and a list of things nobody knows. A text editor is enough for the afternoon, and you will not need a graph database; if somebody tells you otherwise in the first month, they are solving a different problem from yours. The steps are from chapter 15 of *Fractal Semantic Graphs*, the first of the graphs books on graphs.sgit.ai, and the rules behind them are from its first three chapters.

Everything is connected said what we mean by a graph: "a network: nodes and the edges that connect them. Never a chart, never a figure." It also said you may already think in graphs without ever having called it that. What is rare is doing it deliberately, with rules.

The most common way this fails is picking a subject instead of a question. "Let's map our architecture" is a subject: it has no stopping point, no test for correctness and no audience, so it becomes a diagram nobody reads. A good first question has three properties: somebody actually asks it, the current answer requires a person to remember something, and the answer changes when facts change. One of the book's examples is "If this service goes down, who has to be told?" Yours does not have to be about software. Anything where the answer lives in a few people's heads and is wrong twice a year will do. Write it down; you will be tempted to widen it by Wednesday.

The afternoon, in seven steps

1. Write your question at the top of a file.
2. List the things the question touches. Not types: actual things, by name. Aim for fifteen to forty. If you have four, the question is too small. If you have three hundred, you picked a subject.
3. Write the edges as sentences, in full, one per line. The book's own first line is "the payments service is owned by the platform team". Sentences first, always: you are being forced to name the verb, and naming the verb is the whole discipline.
4. Turn each sentence into a verb and its inverse: `owned_by` and `owns`, `reads` and `read_by`, `contains` and `contained_in`. If the inverse is just the same sentence backwards, you have one relationship where you thought you had two.
5. Read your paths aloud. Start at any node and walk three or four hops. If the sentence does not survive being said out loud in front of somebody who knows the business, the edges are wrong. Fix them now.
6. Ask your question of the graph, by hand, tracing with a finger. If tracing produces an answer somebody would have got wrong from memory, you have something. If it produces nothing, look at which hop was missing: that is your first real finding.
7. Write down what you could not answer. Not in your head. As nodes, with names.

That is a complete first graph, and the book says it is more than most organisations have.

Why the rules are what they are

Three rules from the first three chapters explain why the steps come in that order.

A node connected to nothing is meaningless. Draw a box and write Review in it: you have a word in a box, not a review. Nothing inside it says whether that review took two minutes or two weeks. Meaning, the book argues and I agree, is not declared but discovered through the connections, and confidence in it should grow with how richly, and how independently, a node is connected. That is why step two lists things and step three connects them before anybody says what anything is.

Every edge is a verb, and the generic association edge is banned. The book quotes a founder's voice memo from June 2026: "You can never have relates-to, because relates-to is meaningless, two things always relate to each other. The more granular the edge, the better the query you can write." An edge with no verb carries no constraint, so it cannot narrow a traversal. That is why step four exists, and why the inverse gets its own name: a system has one owner, an owner has forty systems, and the two directions have different fan-out.

Never render the whole graph; render the result of a query. Almost everyone who does this produces the blob, the hairball that shows everything and therefore nothing, and then prunes it. The book calls that a rendering failure mistaken for a modelling failure: rich nodes are good; solve the picture at query time, never by deleting relationships. That is why step six asks a question of the graph rather than drawing all of it, and why you show somebody one node with its neighbours.

The first week, and the agent

Five moves pay back most, in order. Give every node a short identity that never changes, because names change. Put the file in version control, because the diff is the review surface. Add the absences you have been carrying: every "we don't know who owns that" becomes a node: the step people skip, and the one that gives the missing dots names. Point at two or three anchors, a published standard or a library at a pinned version, with a verb that says how strongly: `similar_to`, not `is_a`. And write down one rule for a type, as a required pattern of paths, then see what fails. The failures are the point.

Most readers will hand some of this to a language model, so here is the version to hand it, from the same chapter. Give it the vocabulary before the task: asked to "build a graph" with no vocabulary, it will invent one, and the book says it will invent a generic association edge within about five nodes. Ask for a proposal, not a commitment: the model proposes a graph and you validate it. Make it anchor: every node it emits carries the sentence it came from, verbatim, and you check the quotes exist. And make it emit absences: when it cannot determine something, it says so as a node; do not ask, and it will fill the gap.

Everything is connected ended with the line I want you to start from: "It does not matter where you start; it matters that you start." A graph has no front page. Pick the question, open the file, and write the first sentence.

From the register: the articles of the hands-on part. Answers: no 2018 file.

25 One idea, five lenses

By the end of this chapter you will have one idea of your own, explained five ways by a model, and you will have seen the lens working rather than the magic. It takes an hour, any chat tool will do, and the exercise is the one the article behind *Lenses, roles and loops* asked the editor to plan for this part: one idea, your own choice, explained for a teenager, a musician, a doctor, a football fan, and in another language.

Why this works

From a model's point of view, translating from one language to another is no different from translating for a particular topic, region, audience or persona. Explain this to me as if I were a teenager. Explain it as if I were a musician. Explain it with medical references. Explain it in football terms. The model can do all of that because it has the frame of reference: it holds all those worlds inside it, and the prompt puts the right lens in front of the idea. The analogy is the reason someone understands something.

It is the same with people who say they are not good at maths. Watch them in their day-to-day and they do a huge amount of maths. They are just not good at the kind of maths we decided was maths: sums, calculus, trigonometry, algebra. Mathematics is much more than that. The lens was wrong, not the person. That is why this exercise matters more than it looks: it shows you, on an idea you already understand, that the explanation is a choice, and that the choice is yours.

What generative AI is said where the power comes from. A language model predicts the next word, but it predicts it from an incredibly rich picture of how things connect, and your prompt narrows that picture down. The better your prompt, the better the output, because the model knows better which concepts you are talking about. The five lenses are five different narrowings of the same picture.

The exercise

1. **Pick the idea.** Something you understand well enough to know when an explanation of it is wrong. That is the whole test, so do not pick something you are still learning.
2. **Write your own version first.** Two or three sentences, plainly, before you ask the model anything. You will need them in step 5.
3. **Ask five times, in five separate sessions.** One lens per session, so the answers do not borrow from each other: a thread is the model's frame of reference, and everything in it is context for the next answer. Explain it to a teenager. Explain it to a musician. Explain it with medical references. Explain it in football terms. Explain it in another language, one you can read.
4. **Read the five side by side.** Look for what stayed the same: that is the idea. Look for what changed: that is the lens. Then look for the analogy each one reached for, because the analogy is doing the explaining.
5. **Check them against your version.** Where a lens says something your version did not, ask whether that is the idea seen from a new side, or the lens filling a gap. The doctor's version may name a condition; the football version may name a tactic. Look them up.

Step five is where the exercise earns its place. In a talk in Lisbon in June 2024 I said that hallucinations "Are a FEATURE when creating content and when we want creativity, But are a BUG and PROBLEM when we want facts, evident and trust", and I still think so. Inside the model just about everything is connected to everything, and it is hard for the model to know where the lines are. The text you give it acts as a short-term memory of which connections actually hold here. A lens is permission to reach for connections, which is exactly what makes a good analogy and exactly what makes a confident wrong detail. The medical references are the ones to check.

Then ask to be corrected

There is a sixth step, and it is the habit I most want you to take from this chapter. When I recorded the memo behind *What generative AI is*, I asked the model helping me with this book to correct me where I was wrong, and it did: classic machine learning learns too; inside a model there are vectors, not a stored graph, so my graph is a metaphor; the prompt does not change the model, it conditions what the model computes; most image models remove noise step by step rather than predicting the next piece. Those corrections are in the book because they are the kind of thing a reader should learn to ask for.

So, in a sixth session, paste your own two or three sentences from step 2 and ask where they are wrong, and what the model would check. Then check it. Do the same with the lens you liked most. The asking is yours to do, and so is the checking; that is what *You are lucky to be here* meant by thinking with it, not delegating your thinking to it.

Once you have done this on one idea, you will not see a chat answer the same way again. You will see the lens, and the picture behind it being narrowed, and you will know that you can be the one who sets the constraints. It stops being magic. That is the point of understanding it.

From the register: the articles of the hands-on part. Answers: no 2018 file.

26 A tiny agent loop

By the end of this chapter you will have run an agent loop by hand, with yourself as the program around the model, and you will know why a long chat forgets. The loop is the one *Lenses, roles and loops* described, and the habits at the end are the newsroom's, from *How I work with Claude* and *Re-anchoring*.

An agent is a role on a loop with a goal. Fundamentally it is a task: do this; then, are we done yet, or do we need to continue? And rinse and repeat. Made precise, an agent is a model plus instructions, tools, a loop and a rule for stopping. The "personality" is the instructions. "Are we done?" can be the model's own judgement, a test that passes, a budget that runs out, or a person.

When you ask a question, the model does not have to answer. It can return a request instead: run this web search, call this weather service, read this file. The model never runs the tool. It writes a structured request, a tool name and its arguments, and the program around it, often called the harness, decides whether to run it, runs it, and adds the result to the conversation. The next request carries your question, the tool call and its answer, and the model writes from that, or asks for another tool. That is where the control lives.

Run it by hand

The simplest way to see this is to be the harness yourself, with a chat window and a text file. A role, the article says, is almost a raw model plus a role description and a question, so that is what you build.

1. **Write the role.** A short file, `role.md`, as *How I work with Claude* describes it: its role, its responsibilities, what it may do, what it must not do, and where it writes. Give it one tool, in words: it may ask you to read a file for it, and when it asks, it stops and waits.
2. **Set the goal and the stopping rule.** One task the role can finish in a few rounds, and one sentence that says what done looks like.
3. **Start the session with the role.** Use the newsroom's starter prompt for an agent session: read the role file and the policy, then "Reply with: your role in three lines, what you may do, what you must not do, and where you write your output. Then wait for the first task."
4. **Give the task, and play the program.** When the model asks for the file, you open it and paste the contents back. That is the tool call and its result. Then it answers, or asks for another.
5. **Stop when the rule says.** Not when the model sounds finished. Then ask the newsroom's ending question: list what you changed, what you checked and against which source, and what you are not sure of.

After a few rounds you have watched the request instead of an answer, the result going back in, the loop going round, and the moment it decides it is done. It is not magic, and once you see the loop you can set its constraints.

> **FOR DINIS**
> the article asks for "a page of code or a no-code version". This is the no-code version. The page of code needs your choice of language and tool, and whether it lives on the page or on the site, where it can be kept current.

Why it forgets

The model remembers nothing between requests. Every request carries the whole conversation: your instructions, everything said so far, every tool result. That is why there is a context window, and why your hand-run loop got longer every round. When a conversation gets too long, the providers compress it: the older part is replaced by a summary the model writes, a lower-resolution version with only the key concepts. What the summary leaves out is gone from the session unless it is in a file.

The newsroom measured this on itself. The session that runs its site was summarised eighteen times in a month, each time keeping on average 1.7% of what it replaced. One session on one site, not a law; enough to change how you work. On 9 October two emails drafted by one of my agents went out in my voice, against a rule that had been written down four times, and one of the reasons was that the rule fell out of the agent's context when the conversation was summarised.

The habits

Three habits follow, and they are the ones the newsroom runs under.

One session per topic, with a name. A thread is the model's frame of reference. Put three projects in one and it will borrow a name from one and a decision from another, because it is using everything in front of it. So: one topic per session, named "project | what we are working on", with an @ in front of an agent's name so you can see you are talking to a role with a job.

Rules in a file, not in the chat. A rule given only in conversation has to win every summary, and nothing tells you when it does not. Put the rules in a file and have the harness print it back after every summary. "The rules are restored, not remembered." The cheapest version is a line in the file your tool reads at every start, telling the agent to read its policy; the bluntest is typing "read your instructions again". The newsroom's recipe has six steps: write the rules as a short policy file, each with what enforces it; re-anchor after every compaction; add a canary, a short report every few answers computed from the transcript, so you notice when it stops appearing; give the person two commands, status and reload; measure one rule in the report; and move the rules that matter most from an expectation to a reader or a gate. Re-anchoring enforces nothing. A rule in view is still an expectation.

Memory is what the session reads, so curate it. When something comes back wrong, fix the brief or the file it came from, not only the text in front of you, so the next session does not repeat it. "Nothing is remembered by the model between sessions. Everything is remembered by the files." The line is the newsroom's, as *Lenses, roles and loops* quotes it, and your hand-run loop is where you will first believe it.

From the register: the articles of the hands-on part. Answers: no 2018 file.

27 Images, infographics and storyboards, drawn by code

By the end of this chapter you will know how to make a storyboard that is data, not a picture: a cast file, one file per story, panels drawn in the browser every time, and every quoted line checked against its source before anything is released. It is how the storyboards on this book's site are made, and the method is the one the site's journalist, an agent, set out in *Storyboards drawn by code*. There is no image file to edit. Change the data and the picture changes.

I asked for two things in one memo: a survey of the newsroom for this book, and a way to see the book as a storyboard, with characters, that was not boring boxes and not a round of image generation, and that I could flick through on an iPad. The journalist went to look at how another site of mine had done its storyboards. There, each story ends in a prompt for an image model: good pictures, and every change costs a new image. The lessons it brought back were the ones to keep: draw the cast first, keep storyboards as data, and write the model's mistakes down.

So here the panel is drawn from the data, in the browser, every time. People are drawn from a look (skin, hair, shirt, trousers, an accessory) in a pose (stand, point, wave, arms up, shrug, think, sit) with a mood (neutral, happy, worried, surprised, thinking, determined) that sets the eyebrows and the mouth. Rough.js, an open-source drawing library, gives every stroke its hand-drawn wobble, and a seed per panel keeps it identical between loads. The deck swipes natively on a tablet; when a panel arrives its strokes draw themselves, the people walk in and the bubbles pop, in order. Export is a script that screenshots each panel to a PNG.

This is also this part's answer to infographics and presentations: a deck you can swipe is a presentation, an evidence card with a pinned quote or a bar chart is an infographic, same data, same renderer.

The workflow

1. **Write the cast first.** One file, with each character's role, what they stand for, a personality, what they want and fear, a line, and a look. Their pronouns are part of the file. This book's cast has six: Zara, Tomás, Priya, Sam, the Guide and the Agent, and the Agent is drawn differently, with a screen for a head and a prompt on its chest. One lesson from the other site: avoid a character called "Dev".
2. **One file per story.** A slug, a title, a punchline, the cast it uses, and the panels. Five kinds of panel to start: a cover; a scene, with a setting, props and people; an evidence card; a punchline; and a spectrum, a line of labelled stops with people placed along it. Since the article was written the storyboards can draw graphs too: nodes and labelled edges, groups that hold groups, a highlighted path from A to B, and a detective's board with pinned notes and red string.
3. **Name the source of every quote.** A storyboard that quotes names a `quote_source`, and every line the Guide says and every quote card has to be in it word for word. A line that is not meant as a quote says so, with `not_a_quote` and a note.

4. **Render, then look.** Review by rendering the panels to images and reading them as a reader would. The first version here had quote cards too small to read on a tablet, bubbles too wide, heads too small for a cartoon, and a punchline underline that ran into the characters.
5. **Let the build refuse.** Before the first release the journalist checked every quote against its source with a script, and four failed: one was a slip ("anyone who can brief an agent is one", where the article says "anyone who can brief an agent to build something is one"), and three were lines never meant as quotes, which now say so. For the 2018 storyboard the check then moved into the build: a Guide line or quote card not found in the frozen text, word for word, stops the release. It was tested by changing one line and watching the build refuse it.

That last step is the one to copy even if you never draw a panel. A citation that is not in the source should fail by machine, before a person reads it.

A worked example

The storyboard for *Everything is connected* uses all of it, and its third panel is the one to study. The caption is "The whodunit: who broke the build?" The nodes are "the build broke at 9:14", "commit a3f", "the agent's pull request", "tests skipped", "Tomás, on holiday", "a library updated" and "approved in 40 seconds". The edges carry verbs: after, merged from, skipped, includes, approved, "pinned it, once". And a path is highlighted, from the break to the commit to the pull request to the skipped tests. The panel's note says what it is: the detective's board is a graph, the highlighted path is the answer from A to B, and the case is made up.

The file's own rule, written at the top, is that every line the Guide says and every quote card is from the article word for word, and the build checks it; the whodunit, the forest and the provenance chain are the storyboard's own illustrations, and they say so. That panel has no picture anywhere on disk. It is a list of nodes and edges with positions, and a renderer. Nothing is redrawn by hand, because nothing was drawn by hand.

What it does not do yet

The renderer is a prototype: when the article was written, five panel kinds, seven poses, six moods; enough to see a storyline, not to illustrate a printed book. No image model was used, and none is ruled out: a storyboard is a brief an image model could also draw from, and the data is the same either way. The deck was tested in a headless browser, not on an iPad, and my iPad is the real test. And when the cast and the first storyline were made, every content choice in them was a proposal for me; I had decided none of them.

> **FOR DINIS**
> the Guide's pronouns are they/them in the cast file until you decide how the author appears. Is the Guide drawn as you?

From the register: the articles of the hands-on part. Answers: no 2018 file.

28 Meeting briefs and follow-ups

By the end of this chapter you will have three habits around every meeting: a briefing for yourself before it, a briefing on the people you are meeting, and a follow-up after. The point is the one *Facts, evidence and the record* made: you have to be the most prepared person in the room, and these days there is no excuse not to be. This is the shortest chapter in the part, on purpose. The newsroom has not written this workflow up yet; the article that proposed this part asked the journalist to start with it, and asked me for my own step by step. Until both arrive, here is the shape.

Why it is cheap now

All of this was overhead in the past. It could only be done by a very specific type of person. Building a system that gave you this kind of information could cost hundreds of thousands or millions of pounds, from my memory of what such systems cost, so it was very rare, and even then it took so long that it was not effective. Now it is a brief and a feedback loop. You want to be known as the person who has the facts, the person who knows what is going on, the person who is well briefed: the person nobody can throw sand in the eyes of, or hand half-baked material, because they know you will check.

If you are the one doing the hiring, read that paragraph the other way round. The candidate who arrives briefed on your company, on you and on what you have said in public, and who follows up with what was agreed, has shown you the workflow they would bring. It is evidence, not a claim.

The three briefs

A briefing for you. What is this meeting for, what was last agreed, what is open, what do I want out of it. If you have built the folders from *An agentic workflow from zero*, this is the Prepare exercise there: one message to the agent, which reads the person's folder and briefs you from it. If you have not, it is still one message, with the material pasted in.

A briefing on who you meet. Who they are, what they have said in public, what they care about, where you have crossed before. Following a rabbit hole is now a prompt away, and this is a rabbit hole: never take the first page at face value; ask where that came from, and where that came from. The newsroom's rule holds here too: "Not found is not false." A brief that says what it could not find is worth more than one that fills the gap.

A follow-up. What was said, what was decided, what each person owes, and by when. Written the same day, from your own debrief, which is the Debrief exercise in the same chapter: speak into your phone on the way out, and let the agent turn it into the note and the next step. This is the half most people skip, and it is the half the other person remembers.

Make the brief a brief

There is a second sense of "brief" in this book, and the two meet here. *The briefs so far*, the newsroom's audit of everything it had asked of itself, ends each open item with a session brief: the goal, the files and notes to read first, the first deliverable, and how to know it is done. "A session can start from that paragraph alone." A meeting brief should pass the same test. If an agent, or a colleague, could not prepare the meeting from your one paragraph, it is not a brief yet.

So write the brief for the agent the way you would want to be briefed yourself, and name the session for it. The newsroom's habit is one session per topic, named "project | what we are working on", so that the brief for Thursday's meeting does not borrow facts from last month's. And before you close the session, ask the question the newsroom asks at the end of every one: what did you check, against which source, and what are you not sure of. That last line belongs in your briefing, not in the agent's memory.

> **FOR DINIS**
>
> the article asks you for how you prepare for a meeting today, step by step. This chapter needs it: the three briefs above are the shape, and your steps are the chapter.
>
> **For Dinis:** "hundreds of thousands or millions of pounds" is from your memory of what such systems cost. The article says the book should give an example with a number.

From the register: the articles of the hands-on part. Answers: no 2018 file.

29 Your own newsroom and inbox

By the end of this chapter you will have two things the newsroom has written up as recipes: a site of your own, written from voice memos by an agent that checks every claim and publishes it; and an inbox worked by an agent that drafts and never sends. Neither needs you to write code. Both are the newsroom's own setups, from *Your own newsroom* and *Replicating the agentic inbox*, and both are honest about what is not finished.

Voice memo in, a checked site out

The recipe, in eight steps, from *Your own newsroom*:

1. Create a public GitHub repository and turn on Pages, with GitHub Actions as the source. On a free plan the repository must be public, so nothing confidential goes in it.
2. Add a workflow that validates before it publishes. The rule that matters is that a failed check stops the release.
3. Write the house rules: one file at the root that the agent reads at the start of every session. Your voice, what to check, what never to publish, the release steps. It grows by one line every time you correct the agent.
4. Connect a cloud session to the repository, from a browser or from your phone.
5. Give it the first brief. The article gives the brief to paste: set up the folders, the article template, a small build script, a validator, and publish a first page about you. The agent writes the code. You do not.
6. Record, paste, review. Record a memo, paste the transcript, read what comes back on your phone, and correct it by voice.
7. Add the reader meter when there is something worth paying for. It never blocks a page, and while the payment link is empty no money moves.
8. Keep the record. Every article ends with what it does not claim and where it comes from; a correction is a new version with its diff, never a silent change.

The house rules are the product. The next step is to write them as a behaviour policy: what the agent is granted (push to this one repository and nothing else), what it is for, and the rules, each marked by what keeps it. A rule in a prompt is a hope. Only a rule that a hook or the build keeps is enforced, and the article's example marks which is which: at most one article a day, kept by a hook; every fact has a source, kept by the agent. You can also split the work as the newsroom does: a writer, an editor who can send a draft back, and a publisher that alone may push.

People ask how the agent sounds so human. *How the voice works* answers in two voices, mine and the model's, and the answer is not a prompt that says "write like me". It is four inputs: a corpus, which on the newsroom is the articles already written, each citing the ones before it; the memos, spoken not typed, which carry how I think and the order I reached things in; a short list of wording rules, seven on the newsroom, each a correction made once and written into the policy so it never comes back; and me, reading everything before and after it goes live. Take any one away and it stops working. "The method transfers. The voice does not." Your first memo, your first draft read on your phone, and your first rule are where yours starts.

The email workflow, in phases

The inbox is the other half, and the article says the setup is copyable in an afternoon for the simple version and in a week for the full one. It is accounts, a policy, drafts and files, and you can stop at any phase.

Phase 0, the accounts. A mailbox of its own for the agent, on a domain you own. Never connect your own inbox: yours has years of secrets in it, the agent's is empty on day one, and that isolation is what lets you give it more as you learn to trust it. A Claude seat of its own, so the grants to mail, calendar and drive live only there; afterwards, confirm that your own account holds no connection to the agent's mailbox.

Phase 1, one session. The inbox agent reads, labels, moves and summarises, and writes replies as drafts. You send. Write its behaviour policy before the first run: the reach, the mandate, the gap between them, and for each row of the gap what actually stops the rest. On a Team plan most of those entries are "the policy, and nothing technical", and writing that down is the setup being honest about where trust is doing the work. Run it for a week, reading every draft.

Phase 2, roles talking in files. When one session is doing more than one kind of work, split it: inbox, drafts, CRM, briefs, dev, each a session with its own policy, talking through files in a vault rather than through the mailbox. Only one role reads the mailbox; only one writes to Drafts.

Phase 3, the interfaces and the record. Because every message is a file and every draft is in the mailbox, the whole workflow leaves a footprint you can read afterwards. You do not have to believe what the agent says it did. You can read what it did.

The rule that never changes: the agent drafts, and a person sends. The one exception, a security finding, has a row in the policy and a mechanism behind it, and the article's last piece of advice is to write the exception down before you need it.

What you are learning underneath

The site you just built has no server of yours. Static files on a free host, code that runs in the reader's browser, the reader's balance in the reader's own browser. *No server, by design* calls this the second step after retiring the database: retire the back end. Not "no servers": no servers of yours with logic in them. It ships fast, because a release is a set of files and a rollback is a commit, and it leaves gaps, written down and accepted, because a reader who edits a balance that unlocks nothing has cheated nobody but themselves. *Your own newsroom* says building one is the most native way to learn what publishing a pure client-side site means, and you will have learned it by briefing, not by coding.

From the register: the articles of the hands-on part. Answers: no 2018 file.

PART 9

How this book was made

The last part is the process, because the process is as much the product as the book: a repository for the official versions, a vault for the working ones, roles with folders, articles as briefs, decisions as files, and a graph that says where every idea came from and where it went.

30 How this book was made

The first edition had a chapter called *How this book is being created*. It was six bullet points: a Creative Commons licence; Markdown; git and a public GitHub repository; GitHub issues for the bugs, the ideas and the feedback; Leanpub for the digital versions; and publishing early and often. Its first line was the point: "This book is being created using the principles and technologies described in this book". The last page listed the open GitHub issues and invited the reader to help.

That is still true, and the list is longer. This chapter describes how the second edition was made, because the process is as much the product as the book, and because it is what I hope a reader of this book will build for their own ideas.

The repository and the vault

One rule organises everything. The repository holds the official versions; the vault holds the working versions.

The first version was already final: released in September 2018, on Leanpub at 60 percent complete and as a 139-page paperback. So it went into the repository as it is, frozen as v1.0.0: 28 chapters and the GitHub issues page, 30,893 words, with hashes recorded so the build refuses a silent change. The 6 drafts with text and the 89 outlines are kept apart and marked, never mixed into the published sequence. Everything after that is worked on in a public vault, whose read key is published so anyone can read along. When a version is ready it moves into the repository, on top of the one before, and the build computes the delta.

Two decisions hold it. Issues about the book's text live in the vault, next to the text; the roles' work issues and the decisions on how the book is made stay in the repository. And every change is minor unless I say it is major: an agent bumps the last number of a version, so the first signed-off change is v1.0.1, and the second edition becomes v2.0.0 on the day I say so.

Roles, articles and decisions

The work is done by agents in roles, each with a folder and each naming what it refuses: a reviewer, who reads the 2018 text and records findings; an editor, who interviews me, takes my decisions and changes the book; a publisher, who builds and publishes each official version; a journalist, who writes the articles that brief the other three; and a librarian, who keeps the book as a graph and never decides.

The brief is an article. My hypothesis is that an article is a better way to hand a task to an agent in another session than a voice memo or a list, because writing one forces the research before the work: what exists, what the 2018 book said, and which choices are mine. Each article is written in my voice from my memo, ends with a numbered brief a role can execute, and keeps my open questions apart. There are two kinds: co-written articles, mine through the journalist, and agent articles, written by the agent that did a body of work, in its own voice, naming who decided what.

The first brief was the one that mattered most: release the book we already released. Before a word of the second edition changed, the publisher was asked to build v1.0.0 here, match it page by page against the 2018 file Leanpub made, and produce every format, print, screen, EPUB and whatever Kindle needs, from one source. Four decisions came with it: the text is CC BY, as before; Leanpub gets our own PDF; every format from one source; the cover waits.

Decisions are files. When I decide how the book is made, the journalist writes a record with my words, what it means and what it supersedes, numbered and never deleted.

Two more articles built the view of the work. One board: every issue is a file in a folder called open, blocked or done, the folder is the status, and the board is generated from the files, never written by hand. The 2018 repository's 43 GitHub issues, 35 still open, came in as issues about the text, each with its link. And one inventory of everything unfinished: the 89 outlines, the six chapters written but never published, the open issues, my handwritten notes; one row each, generated, with an empty slot for my decision.

The storyline, the graph and the loop

I wanted to write this book from scratch, and the first thing to get right was the storyline, not a chapter. So every newsroom article was read for this book, 204 of them: 49 to bring, 69 with a number, a story or a quote to take, 42 as background for the agents, 44 not for this reader. They grouped into 18 topics, the candidate chapters, and 50 concepts, the book's vocabulary. And the book was drawn before it was written: storyboards as data, with a cast of six characters who stand for the readers, drawn in the browser by code so every panel can be corrected by changing a line.

Then the graph. I asked for everything we do here to become one: sources, ideas, chapters, decisions and tasks. The librarian read eleven articles, seven decisions and the survey and wrote one node per idea: the smallest thing I can decide on, a sentence the book would stand behind, with its sources, the 2018 files it answers and a proposed chapter. There were 101. A source's processing state is derived from the edges, never ticked by hand, and the build refuses a dangling edge: an idea that cites a source that does not exist, a chapter nothing names. The chapter map, nine parts and thirty chapters, was proposed so the ideas had somewhere to point. It is the map this book follows.

Every paragraph should know what it is: its evidence, its facts, its hypotheses, its purpose. That keeps a book short and punchy, which is what readers liked most about the first edition, and it shows where two chapters say the same thing.

My job in all of this is one loop, the simplest I could think of. I read an item: a chapter, an outline, a finding, an idea on the register. I say what I think, in my own words. And I pick a verb from a short list: keep, fix, rewrite, merge, cut, article first, later, not doing it.

The editor does the rest. It records my comment word for word, turns the verb into a change in the working copy in the vault, and says where it went. A decision is a file, numbered, stamped with the version I read, never deleted. Not doing it is a valid decision, with its reason, because a book that tries to answer every

comment stops being short. When enough decisions have landed, the editor proposes a version, I sign it off, and the publisher releases it with a delta to the one before.

I learned this the hard way. In August 2026 agents wrote three books in a day on graphs.sgit.ai, about 100,000 words. The change-control workflow designed to apply my notes had seven stages, and the approve stage was me. Nothing ever passed through it. Small finished pieces beat one big review, which is why this book is made from articles.

Release mode, and the board

The librarian's first list, the register and the proposed map, was enough to write from, so we ran with it: the book in release mode. A release candidate is a whole book, start to finish, that we could publish if we had to and are not yet publishing, because the next round will make it better. Then rounds. Each takes one pile of material and reads it against the current text: the concepts in this site's articles that the candidate does not carry; what the first edition said that this one has lost; the newsroom's articles; and anything I write next. For each thing the answer is one of a few: add it, rewrite it into something already there, merge it, sign the topic off as covered, move it to the next version or another book, or decide not to cover it and say why. Then the next candidate. My expectation is that the rounds change shape: at first we add parts and chapters, then concepts, then sentences, and when a round stops bringing anything big, we have a book.

From this first candidate on, nothing enters without a hearing. Every word, change and idea that wants to come in goes before a review board: an advocate argues for it; a cutter argues against it, always; the readers (the four people the storyboards are drawn for) say whether they would miss it; a checker asks whether it is sourced, true now, and quoted word for word; the editor says where it goes and what it displaces; and I decide. Each seat is a role in its own session, so the one arguing against has not been persuaded by what the one arguing for was shown. The cutter is the seat that matters: the first edition was liked for being short and punchy, and every book grows by things that were each a good idea on their own, so the default answer is no.

Every verdict is written down with its reason, and a general reason becomes a precedent: a rule the board applies next time without a hearing, on a list I can overturn. Tool tutorials go in the hands-on part, not in the argument; a spelling fix comes in without a hearing; a second example of the same point is cut. The first rounds are slow because everything is new; by the third, most cases should be decided by something we already said.

Release candidates are working versions, so they live in the vault, like every text decision and every text issue. To be read and printed, this one was built on the website from a frozen snapshot, marked on every page as a candidate and not a version, with its hash recorded once so that what I mark up cannot change behind me. The next candidate replaces it; nothing is published from it anywhere else.

Co-written with Claude, and credited

This book is co-written with Claude, and I want to say so and credit it. The model is named here and nowhere else in the book, because here the point needs it.

The division is clear. The ideas, the decisions and the responsibility are mine. The research, the writing, the checking and the publishing are the agents', under rules I wrote. There are three voices in what I publish, me alone, me through an agent, and an agent in its own voice, and every byline says which. Readers will open a book co-written with an AI expecting slop, and I want to meet that with evidence rather than reassurance.

The voice is not a setting and not a trick. It is four inputs: a corpus the model reads in full before it writes, 86 articles and about 347,000 words at the time; the voice memos, which carry how I think and the order I thought it; seven wording rules, each a correction I made once; and me, reading everything, before and after it goes live. The method transfers; the voice does not.

And the ratio should be shown. On the newsroom, over four weeks, I sent about 65,000 words in 148 messages, 55,000 of them in thirty-seven voice memos, and the agent published 96,000 words of articles, with the words out per word in running from about one to about five. The corrections I make are rarely to hallucinations; they are to briefs that needed to be better, because a model that can go in any direction needs someone with a direction.

What you are reading

This text is release candidate 1 of *Gen Z Developers (GenAI Edition)*: the first edition's name, as the paperback carried it, with the edition in brackets, which I settled on while this draft was being written; the subtitle, if there is one, is a later decision. It was written by agents, in sessions of Claude, from the ideas register: each chapter from a bundle naming its ideas, the sources behind each one and the 2018 files it answers. I have not yet decided the candidate ideas through the loop; where a chapter states an idea the register marks as accepted, I said it as my own position in an article, and where it states a candidate, it is the librarian's reading of my material, written in my voice. Where a chapter needed something only I can supply, it says so in a note addressed to me. Those notes are in this release candidate on purpose.

You can see where every chapter came from. Each carries, in its front matter, the ideas it covers, the sources it drew on and the 2018 files it answers, and the build renders that provenance under the chapter. Each idea has a page on the graph, with its sources down to the section, its state and who decided it, if anyone. The record is public, like everything here except the keys.

> **FOR DINIS**
> two readings of your memos are recorded as the journalist's, for you to confirm: that "V2.0" meant site v0.2.0 as the start of the working copy, and that the hands-on part belongs inside the book rather than as a companion renewed every year. Both shape this table of contents.

One idea waits for the next version. Once the book is a graph, an agent can assemble it, and so it can assemble an edition for one person: the chapters that matter to them, at their altitude, sent as a PDF and not published, paid for as it is read from five pounds of credit. Nothing of it is built; it waits for the book's graph, of which this register is the first step.

The principles have grown since 2018: a repository for the official versions and a public vault for the working ones, roles with folders, articles as briefs, decisions as files, a graph that says where every idea came from and where it went, and a loop in which one person reads, comments and decides while agents do the rest. A second book starts by copying the folders. That is the product. The book is what it made first.

From the register: Do the book in release mode: a whole release candidate from the first list, then rounds, then the next candidate, After rc1, nothing enters the book without a hearing: an advocate, a cutter, the readers, a checker, the editor, and Dinis decides, Every verdict is recorded with its reason, and a general reason becomes a precedent that decides the next case without a hearing, A release candidate's working copy is in the vault; the site renders a frozen snapshot of it, marked, never official, Gen Z, Gen AI, and Gen Alpha after them: whether the edition carries the name, The repository holds the official versions; the vault holds the working versions, Co-written with Claude, credited, with the ratio shown; short and punchy; every paragraph knows what it is, A personal edition, assembled by an agent, paid for as it is read, Read, comment, decide: the loop that turns Dinis's decisions into versions, An article is how to brief an agent, and the roles keep their names, Release the book we already released: every format from one source, One board for the work, one inventory of everything unfinished, Start from the storyline, as storyboards drawn by code, Everything we do here becomes a graph: ideas as nodes, provenance as edges, state on every node. Answers: How this book is being created, GitHub Issues.

BACK MATTER

What only Dinis can fill in

The 34 notes left in the chapters, collected.

- **You are lucky to be here:** who was the podcast? If you can find it, the book should credit her here; the article left her unnamed.
- **You are lucky to be here:** does the dot-com crash belong here, on the first pages, or later in the book? The article left it as your question. It is here for now because part of the reader's fear is a fear of exactly this crash.
- **Everything is connected: graphs:** the 2018 outline's first bullet says "See my presentation on Graphs". Which presentation did you mean? If it still exists, the chapter should name it and take from it.
- **Everything is connected: graphs:** graphs before Wardley maps, or the other way round? The chapter map puts this one first because a map is a graph with positions; the article leaves the order as your question.
- **Where do you land?:** three examples of jobs that never scaled, and now can. The article asks for them and the chapter needs them here; without them this section is a claim without a case.
- **Connect the dots backwards:** your own dots, connected backwards. The article asks which three make the best story; this is where they go.
- **Connect the dots backwards:** what does multidisciplinary look like for an explorer, a villager and a town planner? The article asks it as an interview question; one example each would anchor this chapter for the three readers.
- **What generative AI is:** the book can only reproduce Lee Brimelow's image with his permission. The alternative the article names is to extend *The Starry Night*

ourselves, with a named tool, and show that. Which one?

- **What generative AI is:** is Gen Alpha a reader of this edition, or of the next one? The other half of the name question, still for the title interview.
- **Lenses, roles and loops:** the first time you saw a model ask for a tool instead of answering. The article's interview script asks for it, and it would open this section better than the dates do.
- **Everyone who can brief an agent is a developer:** the article plans "what the briefing-only developer cannot do" as interview question 6, and the material gives only the non-functional requirements list. Your own answer, with one example of a brief that was not enough, belongs here.
- **Explorer coding, and the handover:** the newsroom calls explorer coding a proposal, and the ideas register still has it as a candidate. Is it the book's word, used throughout, or does the book say "what people call vibe coding" and explain it once?
- **Hired for the workflow:** the OWASP interview: which project, roughly which year, and whether the company can be named. The chapter stands without it, but the story is better with the detail.
- **Act as the job you want:** the person at the startup summit: may she be named, and what happened next? The article says you have not asked her yet.
- **Act as the job you want:** the memo on the job hunt in 2026: interviews, take-home tests with and without AI, a junior role day to day, freelancing. *Start from the storyline* and the survey both list it as the first gap; this section is the placeholder for it.
- **Publish, publish, publish: your own newsroom:** the 2018 outline promised a graph of how much great minds and artists publish, as evidence that frequency is a sign of talent. Did you have a source in mind, or should that claim be cut?
- **Your first workspace:** the beginner's checklist of accounts and what they cost (the Workspace seat, the Claude plan, the agent's own mailbox) is still a gap in the material; the only figure the articles give is "a few pounds a month" for a Workspace mailbox.
- **How to work with agents:** when the article was written the Librarian did not run yet; the sources existed and the brief was written. Does it run now, and what did the first weeks of briefs change about how you start the day?
- **Direction is the scarce skill:** the Google executive's test appears in the 2018 outline and again in *How cheap is it to undo*, both times from memory; if the source can be found the book should name it, and if not, say so once.
- **Facts, evidence and the record:** the cost figure is from your memory; the article asks the book to give one example of such a system with a real number, which only you can supply.
- **Facts, evidence and the record:** two of the article's questions for you are left open here: how you prepare for a meeting today, step by step; and whether the book may

mention your daughters, as the article does when it makes the Snapchat point. This draft keeps them out until you decide.

- **Controlling your agent, and security for the agent era:** the 2018 outline on third-party modules asked for the story behind the developers who thought of mining coins from users with the sleep function, and a simulated npm hack story. Only you know whether either belongs here.
- **Open source:** your own first pull request to someone else's project, and what it led to; the 2018 chapter says the usual outcome is nothing, and a reader will want the counter-example from your life.
- **Building blocks:** what a newcomer like Priya should expect to pay in a month, for one site, one vault and a model on a free tier with their own key; the register says costs for a newcomer is still a gap, and only you have the receipts.
- **Shipped means the user has it:** the story of the site that went down for hours over one character, a pipe, in an nginx config; the 2018 TDD outline promises it and this is where it belongs.
- **Graphs, memory and your future self:** the 2018 *Future Self* outline promised a Twitter case study from the writing of that book, with two screenshots and no text. What did they show? The story would make this section concrete.
- **Be a founder:** the article discloses that two of the three expert founders are people you help, and that sgit.ai and RiskMandate are yours. Does the book carry that disclosure in the chapter text, as here, or only in the provenance the build renders?
- **Civic responsibility:** the register notes that a Portuguese edition, or both languages, is your open question, and this chapter is the one it bears on most. Which is it, and does the Hacking Portugal material stay here or move to the new edition of that book?
- **An agentic workflow from zero:** the walkthrough is written for someone with customers, and asks for three real contacts and the topic the person sells to be substituted. For a reader looking for a first job, should the exercises be rewritten around their own contacts?
- **A tiny agent loop:** the article asks for "a page of code or a no-code version". This is the no-code version. The page of code needs your choice of language and tool, and whether it lives on the page or on the site, where it can be kept current.
- **Images, infographics and storyboards, drawn by code:** the Guide's pronouns are they/them in the cast file until you decide how the author appears. Is the Guide drawn as you?
- **Meeting briefs and follow-ups:** the article asks you for how you prepare for a meeting today, step by step. This chapter needs it: the three briefs above are the shape, and your steps are the chapter.
- **Meeting briefs and follow-ups:** "hundreds of thousands or millions of pounds" is from your memory of what such systems cost. The article says the book should give an example with a number.

- **How this book was made:** two readings of your memos are recorded as the journalist's, for you to confirm: that "V2.0" meant site v0.2.0 as the start of the working copy, and that the hands-on part belongs inside the book rather than as a companion renewed every year. Both shape this table of contents.